



Desarrollo de aplicaciones empresariales con Java

Manual del curso de Java Enterprise Edition

Material para el alumno

Basado en el proyecto de diccionarios · Sesiones 1 a 9
Julio de 2026

Sobre este manual

Este documento recoge, ordena y desarrolla el contenido de las nueve sesiones del curso de Java Enterprise Edition. No es una transcripción de las clases: es el material de referencia que queda cuando el curso termina.

Todo lo que se explica aquí está construido sobre un mismo proyecto —una aplicación de diccionarios— que a lo largo del curso pasó de ser un puñado de ficheros sueltos a un sistema distribuido con base de datos y tres interfaces de usuario. Los ejemplos no son inventados: son el código que se escribió en clase, y está disponible en el repositorio del curso.

La idea que sostiene todo el curso

Que un programa funcione se da por descontado. Si no funciona, no es un programa: es una colección de ficheros de texto con pretensiones. Lo que marca la diferencia entre un buen sistema y uno malo es que **envejezca bien**: que sea fácil de entender, de modificar y de evolucionar.

Si al terminar de leer este manual sólo se retiene una idea, que sea ésta: **el coste de un producto de software no está en escribirlo, está en mantenerlo**. Todas las decisiones técnicas que se explican en las páginas siguientes —las interfaces, los módulos, los patrones, las pruebas— existen para reducir ese coste.

Cómo está organizado

El manual tiene dos partes y cinco apéndices.

La Parte I explica los fundamentos, organizados por materia. Cada capítulo se puede leer por separado y sirve para consultar un concepto concreto meses después: qué es el classpath, cómo funcionan los códigos de estado HTTP, qué hace exactamente `@Entity`.

La Parte II reconstruye el proyecto paso a paso, desde la primera versión hasta la última. Aquí no se explican conceptos nuevos: se muestra cómo se aplicaron, en qué orden, qué problema resolvía cada cambio y cuánto costó cada uno. Es la parte que da sentido a la primera.

Los apéndices contienen el mapa del temario oficial, un glosario, una referencia rápida de comandos, las soluciones de los ejercicios y la lista de correcciones respecto a las notas tomadas en clase.

Cada capítulo termina con **ejercicios propuestos** sobre el propio proyecto. Las soluciones comentadas están en el Apéndice D. Hacerlos es la diferencia entre haber leído el manual y haber aprendido algo.

Convenciones tipográficas

A lo largo del texto se usan tres marcas:

Las citas enmarcadas recogen las ideas centrales del curso, tal y como se formularon en clase.

Los bloques de código muestran siempre código real del proyecto. Cuando se muestra una versión simplificada para explicar un concepto, se indica expresamente.

⚠ Los avisos señalan errores frecuentes, trampas conocidas y puntos donde la teoría y la práctica no coinciden.

Índice

Parte I · Fundamentos

1. **Java: el lenguaje y su estado actual** Tipado estático y dinámico · Gestión automática de memoria · Compilación e interpretación · La máquina virtual · Historia, versiones y el papel de Oracle · Dónde está Java hoy · Versionado semántico
2. **El entorno de trabajo y Maven** Instalación del JDK y de Maven · Variables de entorno · La estructura estándar de un proyecto · El ciclo de vida y sus fases · Gestión de dependencias · Dependencias transitivas · El repositorio local · Proyectos multimódulo
3. **Principios de diseño de software** Componentes y responsabilidades · Interfaz, clase e instancia · SoC · DRY · Los cinco principios SOLID · Inversión de dependencias en la práctica · La memoria RAM y las cachés · El classpath
4. **Patrones de diseño** Patrones de creación, estructurales y de comportamiento · Factoría · Singleton · Adapter y Proxy · Strategy · Template Method · Repository y DAO · DTO · Fachada de servicio · Front Controller · Programación orientada a aspectos · Antipatrones
5. **Pruebas automatizadas y calidad** Por qué no se prueba a mano · Pruebas unitarias con JUnit 5 · Qué se prueba y qué no · Cobertura · Control de calidad automatizado
6. **Evolución de las arquitecturas empresariales** Del escritorio a los componentes desacoplados · Internet y Web · HTML, XML y JSON · El monolito y sus problemas · Análisis por escenarios de cambio · Análisis de impacto por implicado · El coste total del ciclo de vida
7. **Comunicación entre sistemas: HTTP y REST** Fundamentos del protocolo HTTP · Verbos y códigos de estado · Qué es REST · Diseño de un API REST · El contrato como documento · OpenAPI · Diferencias entre una llamada local y una remota
8. **Tecnologías heredadas y sus equivalentes actuales** RMI · CORBA, IDL, IIOP y RMI-IIOP · JNDI y los servicios de nombres · SOAP · Por qué desaparecieron · Tabla de equivalencias
9. **Spring y Spring Boot** Framework frente a librería · Inversión de control · Inyección de dependencias · Componentes y anotaciones · Controladores REST · El servidor embebido · La magia de Spring: ventajas e inconvenientes
10. **Persistencia: Jakarta EE, JPA e Hibernate** Qué es JEE · De J2EE a Jakarta EE · El modelo entidad-relación · Mapeo objeto-relacional · Entidades y relaciones · Repositorios de Spring Data · Consultas derivadas del nombre · Configuración del acceso a datos · Carga inicial y transacciones
11. **Programación funcional en Java** Paradigmas de programación · Funciones anónimas y referencias a métodos · Streams · El modelo map-reduce · Un caso completo: palabras similares
12. **Inteligencia artificial aplicada al desarrollo** Modelos y herramientas · Del prompt al contexto · Agentes · El coste real · Cómo se usó en este curso · Qué conviene no delegar

Parte II · El proyecto, paso a paso

13. [Versión 1 — Aplicación monolítica de consola](#)
14. [Versión 2 — Modularización y arquitectura cliente-servidor](#)
15. [Versión 3 — Persistencia en base de datos](#)
16. [Versión 4 — Nuevas funcionalidades y gestión de errores](#)
17. [Versión 5 — Tres interfaces de usuario](#)
18. [Balance de la evolución](#)

Apéndices

- [A. El temario oficial, punto por punto](#)
 - [B. Glosario de términos](#)
 - [C. Comandos y referencia rápida](#)
 - [D. Soluciones de los ejercicios](#)
 - [E. Correcciones respecto a las notas de clase](#)
-

Parte I · Fundamentos

1. Java: el lenguaje y su estado actual

Antes de construir nada conviene entender con qué herramienta se está construyendo: qué decisiones tomaron los diseñadores de Java, qué se ganó con ellas y qué se pagó a cambio. Ninguna de esas decisiones es gratuita.

1.1. Tipado estático y tipado dinámico

Todo lenguaje de programación manipula datos, y los datos son de distintos tipos: números, textos, fechas. Lo que cambia de un lenguaje a otro es **qué es una variable**.

En C, C++, Pascal o COBOL una variable es un contenedor: un espacio reservado en memoria donde se guarda un valor. En Java, JavaScript, TypeScript o Python **no es un contenedor**, y entender esta diferencia evita muchos malentendidos posteriores.

Considérese esta línea:

```
String texto = "Hola";
```

Se ejecuta en tres pasos:

1. `"Hola"` — se crea en memoria un objeto de tipo `String` con el valor `Hola`. La memoria RAM funciona como un cuaderno de cuadrícula: se abre por cualquier sitio libre y se escribe ahí la palabra.
2. `String texto` — se crea una variable llamada `texto`. Una variable es como un **pósit**: un marcador con un nombre escrito.
3. `=` — ese pósit se pega al lado del dato.

La diferencia entre familias de lenguajes está en el color de los pósts. En Java, TypeScript o C los pósts son de distintos colores según el tipo de dato al que pueden apuntar: azul para `String`, amarillo para `Integer`, verde para `Boolean`. Una variable, además del dato, **tiene tipo**. Eso es el **tipado estático** o fuerte.

En JavaScript o Python todos los pósts son iguales: cualquier variable puede apuntar a cualquier cosa. Eso es el **tipado dinámico** o débil.

```
String texto = "Hola";
texto = 4;           // Error de compilación en Java
```

```
texto = "Hola"
texto = 4           # Perfectamente válido en Python
```

El tipado estático se considera una ventaja y el dinámico una desventaja. La razón se ve mejor comparando dos declaraciones de la misma función:

```
// Tipado dinámico
funcion generarInforme(titulo, datos)

// Tipado estático
PDFDocument generarInforme(String titulo, List<Integer> datos)
```

La segunda versión dice, sin necesidad de leer una línea de implementación ni de consultar documentación, qué hay que pasarle y qué devuelve. Y si alguien se equivoca, **el error aparece al compilar, no en producción**.

El tipado dinámico es cómodo: se escribe menos. Para un programa pequeño y de un solo autor puede compensar. En cuanto el programa crece o hay varias personas trabajando sobre él, el tipado estático deja de ser una molestia y pasa a ser una red de seguridad.

1.2. Gestión automática de la memoria

Java, JavaScript y Python gestionan la memoria automáticamente. Esto tiene una consecuencia que casi nunca se explica:

```
String texto = "Hola";
texto = "adios";
```

Al ejecutar la segunda línea, Java **no sobrescribe** el dato anterior: crea "adios" en otro lugar de la memoria y despega el pósito de "Hola" para pegarlo al nuevo dato. En ese instante hay **dos** cadenas en memoria. En C o en Pascal habría una sola.

Como a "Hola" ya no apunta ninguna variable, queda marcado como **basura** (*garbage*). En algún momento —y aquí hay poco control— entrará el **recolector de basura** (*garbage collector*), un proceso que se ejecuta en paralelo con el programa, y liberará esa memoria.

Java fue diseñado deliberadamente para hacer un uso ineficiente de la memoria. No es un defecto: es una decisión económica.

El razonamiento es el siguiente. En C o C++ el desarrollador debe reservar memoria para cada dato y **liberarla explícitamente** cuando deja de necesitarla. Si se olvida, el programa acumula memoria hasta agotarla (*memory leak*) y se detiene. Ese trabajo consume horas de desarrollo y es una fuente constante de errores difíciles de encontrar.

Traducido a dinero, con cifras orientativas:

	Java	C / C++
Esfuerzo de desarrollo	300 horas	400 horas
Coste por hora	50 €	60 €

	Java	C / C++
Coste de desarrollo	15.000 €	24.000 €
Hardware adicional (RAM)	1.500 €	—
Total	16.500 €	24.000 €

La memoria RAM es barata; las horas de desarrollador, no. Java cambia una por otra, y en la mayoría de los proyectos empresariales sale a cuenta. En un dispositivo empotrado con 64 KB de memoria, no.

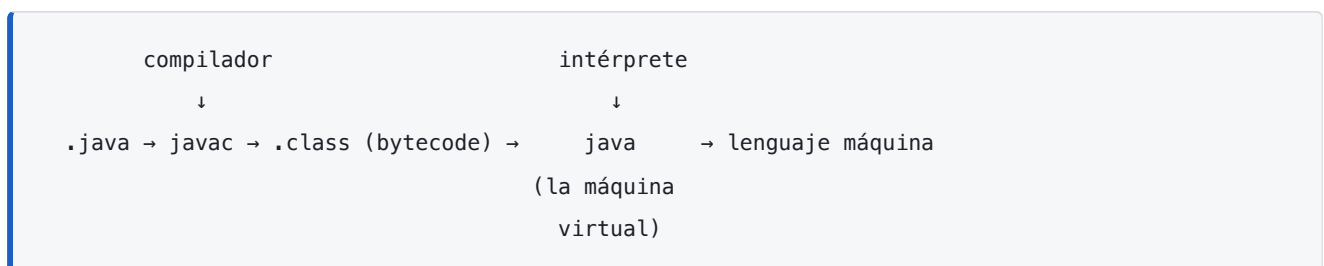
1.3. Compilación e interpretación

Las computadoras no entienden Java, ni Python, ni C: entienden **lenguaje máquina**, que además depende del procesador y del sistema operativo. Hay dos formas de salvar esa distancia.

Compilar es traducir de un lenguaje a otro de nivel de abstracción inferior, antes de ejecutar. Equivale a traducir un libro entero antes de distribuirlo. El inconveniente es que hay que producir una traducción distinta por cada combinación de sistema operativo y procesador.

Interpretar es traducir durante la ejecución, línea a línea. Equivale a leer un libro con un intérprete al lado. Es más lento, pero el mismo original sirve para todos los destinos.

Java hace **las dos cosas**, y ahí está su gran idea original:



El compilador `javac` no genera lenguaje máquina sino **bytecode**: un lenguaje de bajo nivel pero independiente de la máquina. Ese bytecode lo ejecuta la **JVM** (*Java Virtual Machine*), que sí es específica de cada sistema operativo y procesador.

De ahí el lema con el que Java se popularizó: *write once, run anywhere*. Cuando se instala Java, lo que realmente se instala es la máquina virtual.

Una consecuencia práctica: cualquier lenguaje que compile a bytecode puede ejecutarse sobre la JVM y convivir con Java. Es el caso de Kotlin, Scala, Groovy y Clojure.

1.4. Historia, versiones y el papel de Oracle

Java lo crea **Sun Microsystems**; la versión 1.0 se publica en **enero de 1996**. Su recepción fue extraordinaria: se le llamó el lenguaje del futuro y durante casi una década se usó para todo.

	Hacia el año 2000	Hoy
Aplicaciones de escritorio	Java	JavaScript (Electron), C#, C++

	Hacia el año 2000	Hoy
Software empotrado	Java (J2ME)	C, C++, Python, Rust
Aplicaciones web (frontal)	Java (JSP, Servlets)	JavaScript, TypeScript
Aplicaciones web (servidor)	Java	Java + Spring , Node.js, Python, Go
Aplicaciones móviles	Java (Android)	Kotlin, Swift

El nicho principal de Java hoy es el desarrollo de servidor. Y ahí es un lenguaje excelente, sobre todo gracias a Spring. Para casi todo lo demás ha sido desplazado.

Qué ocurrió por el camino. En **abril de 2009 Oracle anuncia la compra de Sun Microsystems**, que se completa en enero de 2010. Con Sun, Oracle adquiere Java. A partir de ahí, la relación de la comunidad con el lenguaje se tensa considerablemente.

Oracle es una compañía que fabrica una de las mejores bases de datos relacionales del mercado y que, en paralelo, tiene un historial notable de proyectos adquiridos que acabaron abandonados o bifurcados por la comunidad:

- MySQL → la comunidad crea **MariaDB**
- OpenOffice → la comunidad crea **LibreOffice**

Con Java el conflicto llegó por dos vías. La primera, un **pleito de once años contra Google** por el uso de las APIs de Java en Android, iniciado en 2010 y resuelto por el Tribunal Supremo de Estados Unidos en 2021 a favor de Google. La segunda, el **cambio de modelo de licencia**: a partir de 2019 Oracle empieza a cobrar por el uso comercial de su propia distribución del JDK, con un modelo de suscripción por procesador en servidores y por usuario en puestos de trabajo.

La reacción del ecosistema fue decisiva y explica la situación actual: **OpenJDK**, la implementación de referencia y de código abierto, se consolidó como la base de múltiples distribuciones gratuitas mantenidas por otras compañías.

- Eclipse Temurin (antes AdoptOpenJDK)
- Amazon Corretto
- Azul Zulu
- Red Hat build of OpenJDK
- Microsoft build of OpenJDK

⚠ Consecuencia práctica: hoy nadie está obligado a pagar por ejecutar Java. En este curso se ha usado **Eclipse Temurin 21**. Pero conviene saber que descargar "Java" de la web de Oracle e instalarlo en los servidores de una empresa **puede tener implicaciones de licencia**. Es un asunto que conviene consultar antes.

El cambio de ritmo. Las versiones de Java salían con cuentagotas:

1996	1.0	2006	6	← 5 años hasta la siguiente
1997	1.1	2011	7	
1998	1.2	2014	8	← 8 años, 2 versiones
2000	1.3	2017	9	
2002	1.4			
2004	5			

A partir de la versión 9 (2017) se adopta una **cadencia fija de seis meses** y se abandona la nomenclatura **1.x**. Cada dos o tres años una versión se designa **LTS** (*Long Term Support*): son las que reciben mantenimiento durante años y las únicas que se usan en producción.

Año	Marzo	Septiembre
2017	—	9
2018	10	11 (LTS)
2019	12	13
2020	14	15
2021	16	17 (LTS)
2022	18	19
2023	20	21 (LTS)
2024	22	23
2025	24	25 (LTS)
2026	26	27

En este curso se ha trabajado con **Java 21**, versión LTS publicada en septiembre de 2023 y ampliamente adoptada en la industria.

El desenlace. Java sobrevivió gracias a dos cosas: la apertura efectiva del ecosistema en torno a OpenJDK y la existencia de **Spring**, el framework que hizo —y sigue haciendo— que construir aplicaciones de servidor en Java sea competitivo.

Google, por su parte, no olvidó el pleito. Adoptó oficialmente **Kotlin** como lenguaje preferente para Android en 2017 —un lenguaje creado por JetBrains por iniciativa propia, que compila a bytecode y se ejecuta sobre la JVM— y desde entonces Java ha ido perdiendo presencia en el desarrollo móvil.

1.5. Versionado semántico

Si un sistema se construye con componentes que evolucionan por separado, hace falta una convención que diga, sólo mirando el número de versión, si actualizar es seguro. Esa convención es el **versionado semántico**:

vMAYOR.MENOR.PARCHE

2 . 4 . 1

Posición	Sube cuando...
MAYOR	Se elimina o se cambia una funcionalidad de forma incompatible (<i>breaking change</i>)
MENOR	Se añade funcionalidad nueva, o se marca una existente como obsoleta (<i>deprecated</i>)
PARCHE	Se corrigen defectos sin cambiar el comportamiento esperado

La utilidad es directa: al actualizar una dependencia de 2.4.1 a 2.5.0 se sabe que hay funciones nuevas y que nada de lo que se usaba ha dejado de existir. Pasar a 3.0.0 obliga a revisar el código.

En el proyecto del curso todos los módulos van por la versión 1.0.0 salvo `diccionario-elfico`, que está en 1.1.0 porque se le añadieron palabras. Ese es exactamente el uso previsto: **cada módulo se versiona por su cuenta**, según lo que le ocurra a él y no al resto del sistema.

Ejercicios del capítulo 1

1.1. Explica, en términos de pósitos y datos en memoria, qué ocurre al ejecutar estas tres líneas y cuántos objetos `String` quedan en memoria al final:

```
String a = "uno";  
String b = a;  
a = "dos";
```

1.2. El equipo propone migrar un servicio de cálculo intensivo de Java a C++ para reducir el consumo de memoria a la mitad. El servicio cuesta 400 € al mes de memoria adicional. Se estiman 250 horas de migración a 55 €/h. ¿Cuántos años tarda en amortizarse la migración? ¿Qué otros factores, además del coste, deberían pesar en la decisión?

1.3. Una biblioteca que usa tu proyecto pasa de la versión 4.2.7 a la 4.3.0. ¿Puedes actualizar sin revisar tu código? ¿Y si pasara a 5.0.0? ¿Y de 4.2.7 a 4.2.9? Justifica cada respuesta.

1.4. ¿Por qué un programa escrito en Kotlin puede utilizar una librería escrita en Java sin ningún tipo de adaptador ni conversión?

2. El entorno de trabajo y Maven

2.1. Instalación del entorno

El entorno del curso consta de tres piezas:

Herramienta	Versión	Qué es
OpenJDK	21 (LTS)	El compilador y la máquina virtual
Apache Maven	3.9 o superior	El gestor del proyecto y de sus dependencias
Visual Studio Code	—	El editor de código

Ni el JDK ni Maven necesitan instalador: basta con descomprimir su ZIP en una carpeta. Una organización razonable:

```
formacionJEE/  
  proyectos/      ← la carpeta que se abre en VS Code  
  java/  
    bin/          ← javac, java  
    lib/  
  maven/  
    bin/          ← mvn  
    lib/
```

Lo único imprescindible es configurar dos **variables de entorno** del sistema operativo, para que se pueda invocar `java` y `mvn` desde cualquier carpeta:

Variable	Valor
PATH	Añadir <code>.../formacionJEE/java/bin</code> y <code>.../formacionJEE/maven/bin</code>
JAVA_HOME	<code>.../formacionJEE/java</code>

`PATH` es la lista de carpetas donde el sistema operativo busca los programas ejecutables. `JAVA_HOME` la consultan Maven y muchas otras herramientas para saber qué instalación de Java deben usar.

Para comprobar que todo está bien, desde una terminal nueva:

```
java -version  
mvn -version
```

2.2. Qué problema resuelve Maven

Maven automatiza las tareas repetitivas de un proyecto Java: compilar, empaquetar, ejecutar las pruebas, generar documentación y desplegar. Existe una alternativa muy extendida, **Gradle**, que hace lo mismo con

otra sintaxis.

Pero su función más importante, con diferencia, es la **gestión de dependencias**.

Al programar no se escribe todo desde cero: se usan librerías de terceros. Cada una de esas librerías es una **dependencia**. Antes de que existieran herramientas como Maven, el procedimiento era buscar cada librería en Internet, descargar el `.jar`, copiarlo a una carpeta del proyecto y repetir la operación cada vez que salía una versión nueva.

El problema no era sólo el número de librerías. Era que **cada librería depende a su vez de otras**, que dependen de otras. Eso son las **dependencias transitivas**. Un proyecto de tamaño medio termina con más de cien librerías en total, de las cuales el equipo sólo ha pedido explícitamente diez o quince.

Reunir a mano las dependencias de un proyecto mediano llevaba días de trabajo, y había que repetirlo con cada actualización. Ese trabajo lo hace hoy Maven en segundos.

Maven descarga las librerías desde un **repositorio de dependencias**. El que viene configurado por defecto es **Maven Central**. Muchas empresas mantienen además su propio repositorio interno —con herramientas como Nexus o Artifactory— para no depender de un servicio externo y para controlar qué librerías se permiten.

Todo lo descargado se guarda en el **repositorio local**, una carpeta oculta en el equipo:

```
~/.m2/repository/          (Linux y macOS)
C:\Users\NOMBRE\.m2\repository\  (Windows)
```

De ahí que la primera compilación de un proyecto tarde y las siguientes sean rápidas: sólo se descarga lo que aún no está.

2.3. La estructura estándar de un proyecto

Maven **impone** una estructura de carpetas. Esto, que puede parecer una imposición arbitraria, es una de sus mayores aportaciones: cualquier proyecto Java del mundo tiene esta misma forma, y cualquier desarrollador sabe dónde mirar.

```

MI-PROYECTO/
  src/
    main/
      java/      ← código de la aplicación
      resources/ ← recursos: ficheros de configuración, datos...
    test/
      java/      ← código de las pruebas automatizadas
      resources/ ← recursos que sólo necesitan las pruebas
  target/       ← todo lo generado. Lo crea y lo borra Maven
    classes/
    test-classes/
  pom.xml       ← la configuración del proyecto

```

Dos reglas que conviene interiorizar:

- **La carpeta `target/` no se toca y no se versiona.** Es material derivado: se puede borrar entera y reconstruir con un comando. Por eso figura siempre en el `.gitignore`.
- **Lo que va en `src/main/resources/` acaba dentro del `.jar`.** En el proyecto del curso, los ficheros de diccionario viven ahí, y por eso se leen como recursos del *classpath* y no como ficheros del disco.

2.4. El fichero pom.xml

`pom.xml` (*Project Object Model*) es el fichero de configuración del proyecto. Lo primero que contiene son las **coordenadas**, los tres datos que identifican unívocamente el proyecto en cualquier repositorio del mundo:

```

<groupId>com.curso</groupId>      <!-- la organización -->
<artifactId>diccionarios-api</artifactId>  <!-- el componente -->
<version>1.0.0</version>          <!-- la versión -->

```

Esas mismas tres coordenadas son las que se usan para declarar una dependencia. Si otro módulo necesita este:

```

<dependency>
  <groupId>com.curso</groupId>
  <artifactId>diccionarios-api</artifactId>
  <version>1.0.0</version>
</dependency>

```

Una buena práctica que se aplicó en el proyecto es declarar las versiones como **propiedades**, para poder cambiarlas en un solo sitio:

```
<properties>
  <version.de.spring>4.1.0</version.de.spring>
</properties>
...
<version>${version.de.spring}</version>
```

2.5. El ciclo de vida y sus fases

Maven ejecuta todas sus tareas a través de **plugins**: hay un plugin para compilar, otro para empaquetar, otro para ejecutar pruebas. Se puede invocar un plugin directamente, pero no es lo habitual.

Lo habitual es usar el **ciclo de vida**, una secuencia de **fases** ordenadas. Al pedir una fase, Maven ejecuta también **todas las anteriores**.

resources	Copia src/main/resources/ a target/classes/
↓	
compile	Compila src/main/java/ y deja los .class en target/classes/
↓	
test	Compila y ejecuta las pruebas de src/test/java/
↓	
package	Empaqueta target/classes/ en un .jar, .war o .ear
↓	
install	Copia el resultado al repositorio local (~/.m2)
↓	
deploy	Lo publica en un repositorio remoto compartido
clean	(fuera del ciclo) Borra la carpeta target/

Un `.jar` no es más que un fichero **ZIP** con las clases compiladas, los recursos y un fichero de manifiesto que indica, entre otras cosas, cuál es la clase que contiene el método `main`.

La fase `install` merece atención porque es la que hace funcionar un proyecto multimódulo: al instalar un módulo, éste queda disponible en el repositorio local para que los demás módulos del equipo lo declaren como dependencia, exactamente igual que si fuera una librería descargada de Internet.

Los comandos más usados:

<code>mvn clean</code>	<code># borra target/</code>
<code>mvn compile</code>	<code># compila</code>
<code>mvn test</code>	<code># compila y ejecuta las pruebas</code>
<code>mvn package</code>	<code># genera el .jar</code>
<code>mvn install</code>	<code># lo instala en el repositorio local</code>
<code>mvn clean install</code>	<code># combinación habitual: partir de cero e instalar</code>

2.6. Proyectos multimódulo

Un proyecto real no es un único bloque de código, sino un conjunto de componentes. Maven lo resuelve con los **proyectos multimódulo**: un `pom.xml` **padre** que no contiene código y que agrupa a los módulos.

El padre se declara con `packaging` de tipo `pom` y enumera sus módulos:

```
<artifactId>diccionarios</artifactId>
<version>1.0.0</version>
<packaging>pom</packaging>

<modules>
  <module>diccionarios-api</module>
  <module>diccionarios-en-ficheros</module>
  <module>servicio-web</module>
  ...
</modules>
```

Y cada módulo declara quién es su padre, del que hereda la configuración común —como la versión de Java del compilador— sin tener que repetirla:

```
<parent>
  <groupId>com.curso</groupId>
  <artifactId>diccionarios</artifactId>
  <version>1.0.0</version>
</parent>

<artifactId>diccionarios-api</artifactId>
```

Desde la carpeta del padre, `mvn install` construye todos los módulos en el orden correcto: Maven analiza las dependencias entre ellos y decide él solo la secuencia. Para trabajar sobre uno concreto existe el modificador `-pl` (*project list*):

```
mvn install                # todos los módulos
mvn -pl servicio-web spring-boot:run  # sólo un módulo
```

Un módulo es la unidad de reutilización, de versionado y de despliegue. Separar un sistema en módulos permite que alguien escriba una implementación nueva sin tocar —ni siquiera recompilar— lo que ya existe.

2.7. Perfiles

Un **perfil** es un bloque de configuración que Maven activa o desactiva según unas condiciones. Sirve para que el mismo `pom.xml` se comporte de forma distinta en entornos distintos, algo habitual para separar

desarrollo, pruebas y producción.

En el proyecto del curso se usaron para resolver un problema concreto: JavaFX distribuye artefactos distintos según el sistema operativo y el procesador, y hay que elegir el que corresponda a la máquina que compila.

```
<profile>
  <id>javafx-windows</id>
  <activation>
    <os><family>windows</family></os>
  </activation>
  <properties>
    <javafx.plataforma>win</javafx.plataforma>
  </properties>
</profile>
```

Con varios perfiles como éste, uno por plataforma, el proyecto compila igual en Windows, en macOS y en Linux sin que nadie tenga que tocar nada.

Ejercicios del capítulo 2

2.1. Un compañero ejecuta `mvn package` y falla con un error de compilación en una clase de pruebas. Sostiene que es un fallo de Maven, porque él sólo quería generar el `.jar` y no ejecutar pruebas. ¿Tiene razón? Explica por qué ocurre y propón dos formas de obtener el `.jar` en ese momento.

2.2. Al añadir una sola dependencia al `pom.xml`, la carpeta `~/m2` crece en más de treinta ficheros `.jar`. ¿Cómo se explica? ¿Qué comando de Maven permitiría ver el árbol completo de lo que se ha descargado?

2.3. En el proyecto de diccionarios, el módulo `servicio-web` depende de `diccionarios-en-bbdd`, que a su vez depende de `diccionarios-api`. Si se modifica una interfaz de `diccionarios-api`, ¿basta con ejecutar `mvn install` dentro de `diccionarios-api`? ¿Qué ocurre si se olvida ese paso?

2.4. Explica por qué los ficheros de diccionario (`es.txt`, `en.txt`) están en `src/main/resources/` y no en una carpeta cualquiera del disco. ¿Qué dejaría de funcionar si se movieran fuera?

2.5. Diseña la estructura multimódulo que tendría un sistema de facturación con estas piezas: el contrato de acceso a datos, una implementación contra PostgreSQL, otra contra un servicio externo, la lógica de cálculo de impuestos, una API REST y una aplicación de línea de comandos. Indica qué módulo depende de cuál.

3. Principios de diseño de software

3.1. Por qué un programa se parte en componentes

La solución más simple a cualquier problema suele ser un único fichero que lo haga todo. Funciona, y para un programa de cincuenta líneas es la respuesta correcta. Deja de serlo en cuanto el programa crece o va a durar años.

No buscamos la solución más simple. Buscamos la solución más mantenible, teniendo en cuenta que el programa va a evolucionar.

La alternativa es construir el sistema con **componentes**, y la regla fundamental es que **cada componente tenga una única responsabilidad**.

La analogía que se usó durante todo el curso es la de un automóvil. Un coche tiene cientos de componentes —ruedas, alternador, batería, bujías, correa de distribución, airbags— que se relacionan entre sí, y cada uno hace una cosa. Que sea así importa por dos razones:

- **Aislamiento de fallos.** Si se estropea un componente que hace una sola cosa, se pierde esa función y el resto del coche sigue andando. Si se estropea un componente que hace cinco cosas, se pierden las cinco.
- **Reemplazo.** Las ruedas se cambian por desgaste, pero también por conveniencia: neumáticos de invierno. El alternador se cambia porque se rompió, pero también porque se ha instalado un equipo de sonido que pide más potencia.

Para que las piezas sean intercambiables hacen falta **estándares**. Una rueda se describe con un conjunto de especificaciones: 17 pulgadas de diámetro, 225 de ancho, perfil 45, código de velocidad V. Cualquier fabricante que cumpla esa especificación sirve.

3.2. Especificación, modelo y objeto: interfaz, clase e instancia

De la analogía anterior salen tres conceptos que se confunden con frecuencia y que conviene separar con precisión:

En el coche	En Java	Qué es
La especificación de la rueda	Interfaz	Abstracta, intangible. Un conjunto de normas. No es una rueda.
El modelo <i>PIRELLI XB17J</i>	Clase	Un modelo concreto que cumple la especificación. Tampoco es una rueda: es un plano.
La rueda montada en el coche	Instancia	Concreta, tangible. Esto sí es una rueda.

Llevado al proyecto del curso:

```
public interface Diccionario {                                // LA ESPECIFICACIÓN
    String cualEsTuIdioma();
    boolean existe(String palabra);
    Optional<List<String>> dameSignificados(String palabra);
}
```

Esta interfaz **no es un diccionario**. Define qué debe poder hacerse con uno. Sobre ella pueden construirse modelos distintos:

```
public class DiccionarioEnFichero implements Diccionario { ... } // UN MODELO
public class DiccionarioEnBBDD implements Diccionario { ... } // OTRO MODELO
public class DiccionarioEnServicioWeb implements Diccionario { ... }
```

Y sólo cuando el programa ejecuta `new DiccionarioEnBBDD("ES", repositorio)` aparece un diccionario de verdad: una **instancia**.

Un diccionario puede estar en un fichero, en una base de datos, en una web... o puede ser un primo que se sabe todas las palabras. Si cumple la especificación, el resto del sistema lo usará igual sin enterarse de la diferencia.

Una nota sobre `Optional`. Cuando una palabra no existe, `dameSignificados` podría devolver una lista vacía, `null`, o lanzar una excepción. Desde Java 8 la respuesta correcta es un `Optional`: una caja que siempre se entrega, y que puede venir llena o vacía. Se le pregunta con `isPresent()`, `isEmpty()` y `get()`.

⚠ Devolver `null` desde un método público se considera hoy una mala práctica. El `Optional` obliga a quien llama a plantearse el caso "no hay resultado", en lugar de descubrirlo con un `NullPointerException` en producción.

3.3. Los principios generales: SoC y DRY

SoC — *Separation of Concerns*, separación de preocupaciones.

Cuando estoy construyendo un componente, me centro en ese componente y me olvido de los demás. Eso será otro problema.

El principio procede del trabajo de **Edsger W. Dijkstra**, uno de los científicos más influyentes de la disciplina, galardonado con el Premio Turing en **1972**. Su conferencia de aceptación, *The Humble Programmer*, es una defensa de la humildad intelectual del programador frente a la complejidad; la formulación explícita de la separación de intereses aparece en un texto suyo posterior, de 1974.

La aplicación práctica en el proyecto: al escribir el controlador REST no hay que preocuparse de dónde salen los diccionarios. El controlador declara que necesita un `SuministradorDeDiccionarios` y se despreocupa del resto.

DRY — *Don't Repeat Yourself*, no te repitas.

Si el código ya existe, en este componente o en otro, no lo vuelvo a escribir: lo reutilizo.

Con un matiz importante que se verá en el capítulo 4: **no toda duplicación aparente es una violación de DRY**. Dos clases con los mismos campos pero que pertenecen a sistemas distintos y evolucionan por separado no son código duplicado, aunque hoy se parezcan.

3.4. SOLID

Cinco principios de diseño orientado a objetos. Respetarlos es una decisión del equipo; lo que está garantizado es que el código resultante será más fácil de mantener.

S — *Single Responsibility Principle*

Una clase, una responsabilidad. Dicho de otro modo: una clase debería tener **una sola razón para cambiar**.

En el proyecto, `CargadorDeDatos` sólo carga datos al arrancar; `NormalizadorDeTerminos` sólo normaliza texto; `DiccionariosRestController` sólo traduce peticiones HTTP en llamadas al negocio. Si cambia el formato de los ficheros de diccionario, sólo hay un sitio donde mirar.

O — *Open/Closed Principle*

Un sistema debe estar **abierto a la extensión y cerrado a la modificación**: debe poder añadirse funcionalidad nueva sin tocar el código que ya funciona.

Es el principio con el efecto práctico más medible. En la sesión 7 se añadió el método `dameIdiomas()` a la interfaz `SuministradorDeDiccionarios`, lo que dejaba sin compilar a los tres módulos que ya la implementaban. En lugar de modificarlos todos, se declaró como método `default`:

```
public interface SuministradorDeDiccionarios {
    boolean tienesDiccionarioDe(String idioma);
    Optional<Diccionario> dameDiccionarioDe(String idioma);

    default List<String> dameIdiomas() {
        throw new UnsupportedOperationException("Este método no está implementado");
    }
}
```

Sólo lo implementó el módulo que lo necesitaba. Dos sesiones después, cuando la interfaz de escritorio necesitó poblar un desplegable de idiomas, se implementó también en el cliente HTTP: un método nuevo, cero cambios en la interfaz, cero recompilaciones ajenas.

⚠ Los métodos `default` en interfaces (Java 8 en adelante) existen para permitir que una interfaz evolucione sin romper a quien la implementa. **No son un sitio donde poner lógica de negocio.**

L — Liskov Substitution Principle

Cualquier implementación debe poder sustituir a su interfaz sin que el sistema se comporte de forma inesperada. Si un método promete devolver la lista de significados, todas las implementaciones deben hacerlo con la misma semántica: no vale que una devuelva lista vacía y otra lance una excepción ante el mismo caso.

I — Interface Segregation Principle

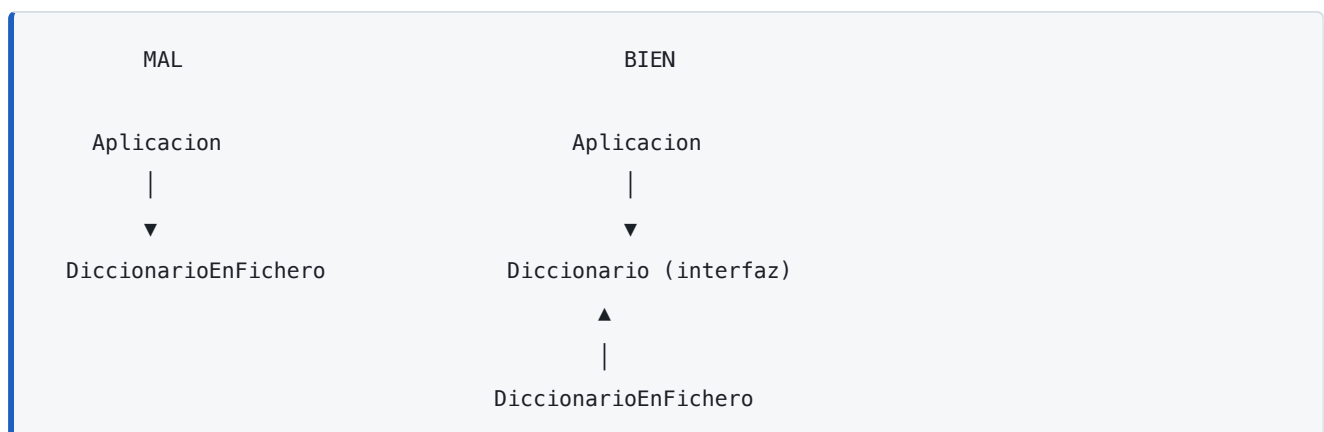
Mejor varias interfaces pequeñas y específicas que una grande que obligue a implementar métodos que no se necesitan.

El proyecto lo ilustra por la vía del contraejemplo. `InterfazDeUsuario` reúne nueve métodos pensados para un programa de terminal —incluidos `recuperarLaPalabraSolicitadaPorElUsuario()` y `mostrarMensajeAyuda()`—. Cuando en la sesión 9 se intentó implementarla con una ventana gráfica, buena parte de esos métodos no encajaban. Una interfaz más segregada habría envejecido mejor.

D — Dependency Inversion Principle

Los componentes de alto nivel no deben depender de implementaciones concretas de componentes de bajo nivel: **ambos deben depender de abstracciones**.

Es el principio que estructura todo el proyecto:



En el diagrama de la izquierda, cambiar de ficheros a base de datos obliga a tocar `Aplicacion`. En el de la derecha, no. Por eso el módulo `diccionarios-api` no contiene ni una sola implementación: sólo las dos abstracciones de las que depende todo lo demás.

Cómo se invierte la dependencia en la práctica. Si `Aplicacion` no construye su diccionario, ¿quién lo hace? Hay dos respuestas, y el curso recorrió las dos:

- El **patrón Factoría**, escrito a mano. Una clase cuya única responsabilidad es decidir qué implementación se construye.
- La **Inyección de Dependencias**, que hace un contenedor como Spring. El componente declara qué necesita y el contenedor se lo entrega.

Ambas se desarrollan en el capítulo 4.

3.5. La memoria y las cachés

Los procesos usan la memoria RAM para varias cosas simultáneamente: almacenar el código del programa, mantener la pila de ejecución —los punteros a las líneas que se están ejecutando—, guardar las variables de trabajo y mantener **cachés**.

Una caché almacena temporalmente datos cuyo origen es lento de consultar. En el proyecto tiene sentido cachear diccionarios: leerlos de fichero o de base de datos cuesta mucho más que tenerlos en memoria.

Un sistema debe poder funcionar **sin** su caché. Una caché es una optimización, nunca un requisito.

Supóngase un sistema con cincuenta diccionarios cacheados que agota la memoria disponible, y llega la petición de cargar uno más. Hay dos comportamientos posibles: que la aplicación falle con un `OutOfMemoryError`, o que vacíe parte de la caché y siga funcionando. Sólo el segundo es aceptable.

En Java, la clase habitual para una tabla en memoria es `HashMap<K, V>`, que guarda **referencias** a los objetos e impide que el recolector de basura los elimine mientras la tabla exista. Para una caché, la clase adecuada es `WeakHashMap<K, V>`: se comporta igual, pero permite al recolector de basura liberar sus entradas cuando la memoria escasea.

3.6. El classpath

Cuando la máquina virtual ejecuta un programa necesita saber dónde están los ficheros `.class` con el código de cada componente. Los busca en las rutas indicadas en el **classpath**.

Conceptualmente es análogo al `PATH` del sistema operativo: allí donde `PATH` dice dónde buscar programas ejecutables, `classpath` dice dónde buscar clases.

El classpath no sirve sólo para clases: también para **recursos** —ficheros de configuración, propiedades y, en este proyecto, los diccionarios—. De ahí que el código no abra los diccionarios como ficheros del disco sino como recursos:

```
Resource archivo = new ClassPathResource("diccionarios/es.txt");
```

⚠ La diferencia es decisiva. Un objeto `File` **no sabe leer dentro de un** `.jar`; un recurso del classpath, sí. Gracias a eso el mismo código funciona en el entorno de desarrollo, donde los `.txt` están sueltos en `target/classes`, y en producción, donde están comprimidos dentro del `.jar`.

En un proyecto Maven no hay que configurar el classpath a mano: lo construye Maven a partir de las dependencias declaradas en el `pom.xml`.

Ejercicios del capítulo 3

3.1. Enumera todas las clases del proyecto de diccionarios que implementan la interfaz `Diccionario`. Para cada una, indica de dónde saca los datos y qué tendría que cambiar en `Aplicacion` para usarla. ¿Qué principio SOLID explica tu respuesta?

3.2. Un compañero propone añadir a la interfaz `Diccionario` un método `int cuantasPalabrasTienes()`. Enumera todos los módulos que dejarían de compilar y propón dos formas de introducir el método sin romper nada, indicando cuál prefieres y por qué.

3.3. Localiza en el proyecto un caso donde se haya sacrificado el principio de segregación de interfaces (ISP). Propón cómo se podría haber dividido esa interfaz en dos o tres más pequeñas, e indica qué implementación se beneficiaría de cada una.

3.4. El equipo decide cachear en memoria los resultados de las búsquedas más frecuentes. Escribe la declaración de la estructura de datos que usarías y justifica la elección. ¿Qué ocurriría si se usara un `HashMap` y el servicio recibiera millones de búsquedas distintas?

3.5. ¿Por qué `SuministradorDeDiccionariosEnFicheros` recibe en su constructor el nombre de una carpeta de recursos (`"diccionarios"`) y no una ruta absoluta del disco (`"C:\datos\diccionarios"`)? ¿Qué dejaría de funcionar con la segunda opción?

4. Patrones de diseño

4.1. Qué es un patrón

Un patrón de diseño **no es código que se copia**. Es una solución con nombre a un problema que se repite. Su valor es doble: ahorra reinventar la solución y, sobre todo, **da un vocabulario común**. Decir "aquí hay una factoría" transmite en tres palabras lo que de otro modo exigiría un párrafo.

Se clasifican tradicionalmente en tres familias —creación, estructurales y de comportamiento— a las que en el mundo empresarial se añaden los patrones de las capas de integración y presentación.

4.2. Patrones de creación

Factoría

Problema. Si cada componente construye sus dependencias con `new`, el sistema queda acoplado a implementaciones concretas y cambiar una obliga a tocar muchos ficheros.

Solución. Centralizar en un único componente la decisión de qué implementación se construye.

Sin factoría, la clase principal del proyecto tendría este aspecto:

```
// MAL: Aplicacion decide qué implementaciones se usan
interfazDeUsuario          = new InterfazDeUsuarioConsola(args);
suministradorDeDiccionarios = new SuministradorDeDiccionariosEnFicheros("diccionarios");
```

El problema es que `Aplicacion` está asumiendo más responsabilidades de las que le corresponden. Es como si para cambiar la batería de un coche hubiera que recolocar el resto de elementos del capó porque no entra.

Con factoría:

```
// Aplicacion.java
interfazDeUsuario          = InterfazDeUsuarioFactory.dameInterfazDeUsuario(args);
suministradorDeDiccionarios = SuministradorDeDiccionariosFactory.dameSuministradorDeDiccionarios();
```

```
// SuministradorDeDiccionariosFactory.java – el único fichero que decide
public class SuministradorDeDiccionariosFactory {
    public static SuministradorDeDiccionarios dameSuministradorDeDiccionarios() {
        //return new SuministradorDeDiccionariosEnFicheros("diccionarios");
        return new SuministradorDeDiccionariosEnServicioWeb("http://localhost:8080");
    }
}
```

`Aplicacion` no sabe —ni necesita saber— si los diccionarios vienen de ficheros, de una base de datos o de la red. Trabaja siempre contra la interfaz. Cambiar el origen de datos de todo el sistema es **modificar una línea**.

Singleton

Problema. Hay componentes de los que sólo debe existir una instancia compartida.

Solución clásica. Constructor privado, variable estática y control de concurrencia. En el proyecto **no se escribió ni una sola vez**, y sin embargo se usa constantemente: **todos los componentes de Spring son singletons por defecto**.

```
@Component
public class SuministradorDeDiccionariosEnBBDD implements SuministradorDeDiccionarios { ... }
```

Con esa anotación, Spring crea **una** instancia y la entrega a todo el que la pida. Es un buen ejemplo de lo que aporta un framework: se obtiene la garantía de instancia única sin escribir el código que tradicionalmente la implementaba, ni sus errores clásicos de concurrencia.

4.3. Patrones estructurales

Adapter (o Gateway)

Problema. Un sistema externo no habla el idioma que espera nuestra aplicación.

Solución. Una clase que implementa nuestra interfaz y traduce por dentro.

```
public class DiccionarioEnServicioWeb implements Diccionario {
    // Por fuera: un Diccionario más.
    // Por dentro: peticiones HTTP y conversión de JSON a objetos Java.
}
```

Para la aplicación cliente, este objeto es indistinguible del que leía ficheros. Ahí está el valor: **la traducción está confinada en un solo sitio**.

Proxy

Problema. Se quiere añadir un comportamiento —medir tiempos, controlar errores, registrar auditoría, comprobar permisos— alrededor de un componente, sin modificarlo.

Solución. Un objeto intermediario que implementa la misma interfaz que el componente real, lo envuelve y añade el comportamiento antes o después de delegar.

Considérese este punto de partida:

```

public interface LibreriaQueHaceAlgoAPI {
    void haceAlgo();
}

public class ClaseQueNecesitaAlguienQueHagaAlgo {
    private LibreriaQueHaceAlgoAPI libreria;

    public ClaseQueNecesitaAlguienQueHagaAlgo(LibreriaQueHaceAlgoAPI libreria) {
        this.libreria = libreria;
    }

    public void trabajo() {
        libreria.haceAlgo();
    }
}

```

Ahora surge la necesidad de **medir cuánto tarda** la librería. ¿Dónde se pone ese código? Hay dos sitios evidentes y los dos son malos:

- **Dentro de la librería.** Puede que no sea nuestra y no podamos modificarla. Y aunque lo fuera, la medición se activaría en todos los sitios donde se use, interese o no.
- **Dentro de la clase que la usa.** Habría que repetir esas líneas en cada punto donde se quiera medir, y activarlas o desactivarlas obligaría a comentar y descomentar código en producción.

La solución es un proxy:

```

public class ProxyQueMideTiempos implements LibreriaQueHaceAlgoAPI {
    private LibreriaQueHaceAlgoAPI libreria;

    public ProxyQueMideTiempos(LibreriaQueHaceAlgoAPI libreria) {
        this.libreria = libreria;
    }

    public void haceAlgo() {
        long tin = System.currentTimeMillis();
        libreria.haceAlgo();
        long tout = System.currentTimeMillis();
        System.out.println("Ha tardado " + (tout - tin) + " ms");
    }
}

```

Y en el momento de construir los objetos se decide si se quiere medir o no:

```
LibreriaQueHaceAlgoAPI libreria = new LibreriaQueHaceAlgoDeUnaFormaConcreta();
LibreriaQueHaceAlgoAPI proxy    = new ProxyQueMideTiempos(libreria);

new ClaseQueNecesitaAlguienQueHagaAlgo(proxy).trabajo(); // con medición
new ClaseQueNecesitaAlguienQueHagaAlgo(libreria).trabajo(); // sin medición
```

Ni la librería ni su consumidor se han modificado. Es el principio Abierto/Cerrado en estado puro.

El objetivo, siempre, es que introducir un cambio consista en **escribir código nuevo** y no en tocar código que ya funciona. Si no se toca lo que funciona, la probabilidad de romperlo es mucho menor; y si algo se rompe, será en lo nuevo, que es mucho más fácil de arreglar.

Proxy remoto

Un caso particular del anterior: un objeto local que representa a otro que vive en otra máquina. Es exactamente lo que hace `DiccionarioEnServicioWeb`: desde la interfaz de usuario se le llama como a cualquier objeto y por debajo la llamada cruza la red.

Conceptualmente es el mismo servicio que prestaban RMI y CORBA (capítulo 8), con una diferencia importante: **aquí el programador sabe que la llamada es remota**.

4.4. Patrones de comportamiento

Strategy

Problema. Existen varias formas de hacer lo mismo y hay que poder elegir una sin que el resto del código se entere.

Solución. Una interfaz común y varias implementaciones intercambiables.

Es la columna vertebral del proyecto. `SuministradorDeDiccionarios` es la interfaz de la estrategia y sus tres implementaciones —ficheros, servicio web y base de datos— son las estrategias concretas. La aplicación elige una al arrancar y el resto del código es indiferente a cuál sea.

Template Method

Problema. El esqueleto de un algoritmo es siempre el mismo y sólo cambian algunos pasos.

Solución. El esqueleto lo pone el framework; nosotros rellenamos los huecos.

```
@Component
public class CargadorDeDatos implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        // Sólo escribimos ESTO. Cuando se ejecuta lo decide Spring.
    }
}
```

Spring define *cuándo* se ejecuta el arranque y en qué orden; nosotros sólo escribimos `run()`. No controlamos el flujo: lo controla el framework. Es la puerta de entrada al concepto de Inversión de Control (capítulo 9).

4.5. Patrones de la capa de integración

La capa de integración es la que habla con los sistemas externos: bases de datos, servicios de terceros, ficheros.

DAO y Repository

DAO (*Data Access Object*) es el patrón clásico de J2EE: aislar el acceso a datos del resto de la aplicación, de forma que cambiar el almacenamiento no afecte a la lógica de negocio.

Su forma actual es el **Repository** de Spring Data JPA, con una particularidad llamativa: **sólo se declara la interfaz**, y el framework genera la implementación —incluida la consulta SQL— deduciéndola del nombre del método.

```
public interface PalabraRepository extends JpaRepository<Palabra, Integer> {
    boolean existsByPalabraAndIdioma_Codigo(String palabra, String codigo);
    Optional<Palabra> findByPalabraAndIdioma_Codigo(String palabra, String codigo);
    List<Palabra> findByIdioma_Codigo(String codigo);
}
```

Data Loader

Inicialización del almacén de datos al arrancar. En el proyecto, `CargadorDeDatos` lee los ficheros de diccionario del classpath y los vuelca en la base de datos, comprobando primero que no haya datos ya cargados.

⚠ En producción esta tarea no se resuelve así, sino con herramientas especializadas en versionado de esquemas de base de datos: **Liquibase** o **Flyway**. Un `if` al arranque cubre la carga inicial, pero no responde a la pregunta de qué ocurre cuando una versión nueva de la aplicación añade palabras a un diccionario que ya estaba cargado.

4.6. Patrones de la capa de presentación

Transfer Object (DTO)

Problema. Atravesar la red es caro. Hacer una llamada remota por cada dato es inviable.

Solución. Un objeto sin lógica cuyo único cometido es transportar datos a través de una frontera, de modo que una sola llamada traiga todo lo necesario.

```
public class RespuestaPalabra {  
    private String palabra;  
    private String idioma;  
    private List<String> significados;  
    private List<String> similares;  
    // sólo getters. Ninguna lógica.  
}
```

En el proyecto existen **dos** clases `RespuestaPalabra`: una en el servidor y otra en el cliente. A primera vista parece una violación de DRY, y no lo es:

Una cosa es lo que el servidor decide enviar y otra lo que el cliente necesita recibir. Puede que el servidor añada un campo que el cliente no use. Hoy son iguales; mañana no tienen por qué serlo, y eso está bien.

Mantenerlas separadas es lo que permite que servidor y cliente evolucionen a ritmos distintos, que es la razón de ser de un sistema distribuido. La aplicación web escrita en Angular tomó la misma decisión de forma independiente: separa el DTO —el formato de cable— del modelo de dominio que consumen sus componentes.

Business Delegate

Un objeto local que oculta a la capa de presentación toda la mecánica de la invocación remota.

`InterfazDeUsuarioConsola` no sabe nada de HTTP, ni de URLs, ni de JSON, ni de códigos de estado: todo eso vive dentro de `SuministradorDeDiccionariosEnServicioWeb` y `DiccionarioEnServicioWeb`.

Service Façade

Exponer una interfaz única y sencilla que oculte la complejidad interna de un subsistema. Toda la funcionalidad del sistema pasa por dos métodos:

```
boolean tienesDiccionarioDe(String idioma);  
Optional<Diccionario> dameDiccionarioDe(String idioma);
```

Detrás puede haber lectura de ficheros, consultas SQL con uniones a tres tablas o peticiones HTTP. El consumidor no lo sabe ni le importa. Es la versión moderna del **Session Façade** de J2EE.

Front Controller

Un único punto de entrada que recibe **todas** las peticiones y las encamina. Este patrón lo aporta Spring: internamente, un componente llamado `DispatcherServlet` recibe absolutamente todas las peticiones HTTP y consulta la tabla de rutas. Lo único que escribimos nosotros son esas rutas:

```
@RestController
public class DiccionariosRestController {

    @GetMapping("/diccionarios/{idioma}")
    public ResponseEntity<Void> existeDiccionarioDe(@PathVariable("idioma") String idioma) { ... }
}
```

4.7. Programación orientada a aspectos

Hay funcionalidades que no pertenecen a ningún componente en particular porque atraviesan todo el sistema: la gestión de errores, la seguridad, el registro de actividad, la auditoría, la medición de rendimiento. Se llaman **preocupaciones transversales**, y meterlas dentro de la lógica de negocio la ensucia y la repite.

La **Programación Orientada a Aspectos (AOP)** consiste en separarlas del negocio y aplicarlas desde fuera. Y su mecanismo habitual es, precisamente, el patrón Proxy.

El caso del proyecto: el servidor debía responder con un código HTTP 500 cuando ocurriera un error no controlado —por ejemplo, si la base de datos deja de estar disponible—. Escrito a mano, exigiría envolver cada método del controlador en un `try/catch`:

```
public ResponseEntity<RespuestaPalabra> existePalabra(String idioma, String palabra) {
    try {
        return controlador.existePalabra(idioma, palabra);
    } catch (Exception e) {
        return ResponseEntity.status(500).body(null);
    }
}
// ...y lo mismo en los otros tres métodos
```

Spring construye ese proxy por nosotros. Basta con declarar una clase con la lógica de tratamiento del error y anotarla:

```

@RestControllerAdvice
public class DiccionariosRestControllerAdvice {

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public String procesarExcepcion(Exception e) {
        // Aquí podría además registrarse en el log, avisar a monitorización...
        return e.getMessage();
    }
}

```

El controlador **no se modificó**. En él sólo se escribe el *happy path* —el camino que sigue el programa cuando todo va bien—, y el tratamiento de errores vive aparte, se activa y se desactiva por sí solo y se puede cambiar en un único sitio.

Antes: Tomcat → Controlador REST
 Ahora: Tomcat → Proxy (Advice) → Controlador REST

4.8. Antipatrones

Un **antipatrón** es una solución que parece razonable, se repite mucho y produce sistemáticamente malos resultados. El curso está construido sobre la corrección de cuatro.

Big Ball of Mud. Un sistema sin estructura reconocible. Fue el punto de partida: diez ficheros `.java` sueltos en una carpeta, sin paquetes ni módulos. Con diez ficheros parece manejable; con cuatrocientos es imposible saber qué hace cada cosa y qué se rompe al tocarla.

Monolito. Un único sistema que lo hace todo, donde cualquier cambio puede impactar en cualquier otra parte. El ejemplo analizado en clase fue una empresa ficticia, *Animalitos Fermín*, con venta de productos, citas veterinarias, peluquería y nóminas de empleados en el mismo sistema. La alternativa: componentes desacoplados.

Ciente pesado con datos embebidos. Meter en cada instalación cliente los datos y la lógica que deberían estar centralizados. Es el problema que motivó todo el rediseño del proyecto, y su análisis por implicado está en el capítulo 6.

Acoplamiento a implementaciones concretas. Dependier de la clase concreta en lugar de la abstracción. Es la violación del principio DIP, y corregirla es lo que permite que el sistema tenga hoy tres orígenes de datos intercambiables.

Ejercicios del capítulo 4

4.1. Escribe un proxy que registre en el log todas las búsquedas de palabras que no se encuentran, para saber qué palabras echan en falta los usuarios. No puedes modificar `DiccionarioEnBBDD` ni

`DiccionariosRestController`. Indica también en qué punto del código habría que "enchufarlo".

4.2. Enumera tres preocupaciones transversales, distintas de la gestión de errores, que tendría un sistema bancario real. Para cada una, explica qué pasaría si se implementara dentro de cada método de negocio en lugar de como aspecto.

4.3. El proyecto mantiene dos clases `RespuestaPalabra` idénticas. Un revisor sostiene que viola DRY y pide unificarlas en un módulo compartido del que dependan cliente y servidor. Argumenta a favor y en contra, y decide. ¿Cambiaría tu respuesta si cliente y servidor los desarrollaran empresas distintas?

4.4. Localiza en el proyecto los tres puntos donde Spring aplica el patrón Proxy sin que aparezca la palabra "proxy" en ninguna parte del código.

4.5. El patrón Repository permite escribir `findByPalabraAndIdioma_Codigo(String palabra, String codigo)` sin implementarlo. Escribe la declaración de los métodos que necesitarías para: (a) buscar todas las palabras de un idioma que empiecen por un prefijo; (b) contar cuántas palabras tiene un idioma; (c) buscar una palabra sin distinguir mayúsculas de minúsculas.

5. Pruebas automatizadas y calidad

5.1. Por qué no se prueba a mano

Terminado un programa, hay que comprobar que funciona. La forma intuitiva es ejecutarlo y probar casos a mano. Es también la forma equivocada, por tres razones:

- **Es lenta.** Cada comprobación consume tiempo de una persona.
- **No es fiable.** Una persona se distrae, se salta un caso o interpreta mal un resultado.
- **No es repetible.** Y esto es lo decisivo.

El tercer punto merece desarrollo. Un programa se modifica muchas veces a lo largo de su vida. Cada modificación obliga a comprobar **no sólo lo que se ha tocado, sino todo lo demás**, porque cualquier cambio puede haber roto algo aparentemente sin relación. Probar a mano el sistema completo tras cada cambio es inviable a partir de cierto tamaño, y lo que ocurre en la práctica es que **se deja de hacer**.

La solución es escribir un programa que pruebe el programa. Y no una prueba grande, sino muchas pequeñas.

La matización sobre el tamaño es importante. Una única prueba que ejecute la aplicación entera aporta poco: cuando falla, informa de que algo va mal pero no de **qué** componente ha fallado. El objetivo es probar cada componente por separado y, después, la integración entre ellos.

5.2. JUnit

La librería estándar para pruebas automatizadas en Java es **JUnit**. No viene con el JDK: se declara como dependencia y Maven la descarga.

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

El `<scope>test</scope>` indica que esa librería sólo se necesita para compilar y ejecutar las pruebas: **no se empaqueta en el .jar final**. Es un detalle que conviene no olvidar, porque de lo contrario se distribuye a producción código que sólo tiene sentido en desarrollo.

⚠ Hoy se usa **JUnit 5**, cuyo nombre técnico es *Jupiter*. Circulan por Internet muchísimos ejemplos de **JUnit 4**, cuya sintaxis y cuyos paquetes son distintos (`org.junit.Test` frente a `org.junit.jupiter.api.Test`). Mezclar los dos es una fuente de errores difíciles de diagnosticar: las pruebas simplemente no se ejecutan y el informe dice que todo ha ido bien.

JUnit busca en cada clase de prueba todos los métodos anotados con `@Test` y los ejecuta uno a uno. Cada uno puede terminar de tres formas:

Resultado	Significado
ok	La prueba pasó
failure	Una verificación no se cumplió: el programa hace algo distinto de lo esperado
error	La prueba reventó con una excepción inesperada

Al terminar genera un informe con el total de pruebas, cuántas pasaron y, de las que no, su nombre y el motivo. Se ejecutan con `mvn test`, y también automáticamente antes de `mvn package`, porque `test` es una fase anterior del ciclo de vida.

5.3. La estructura de una prueba

Toda prueba tiene tres partes, y escribirlas siempre en el mismo orden hace que el código de pruebas sea legible por cualquiera:

1. **Contexto** — *dado que...*: se prepara la situación de partida.
2. **Acción** — *cuando...*: se ejecuta exactamente lo que se quiere probar.
3. **Verificación** — *entonces...*: se comprueba el resultado con aserciones.

Un ejemplo real del proyecto:

```
@Test
@DisplayName("Obtener el idioma de un diccionario")
void cualEsTuIdiomaTest() {
    // 1. Contexto: dado un diccionario de español...
    DiccionarioEnFichero diccionario = new DiccionarioEnFichero("es", null);

    // 2. Acción: cuando le pregunto su idioma...
    String idiomaDevuelto = diccionario.cualEsTuIdioma();

    // 3. Verificación: entonces me responde "es"
    Assertions.assertEquals("es", idiomaDevuelto);
}
```

La anotación `@DisplayName` permite dar a la prueba un nombre legible que aparece en el informe. Es especialmente útil cuando el informe lo lee alguien que no es quien escribió la prueba.

Las verificaciones se hacen con la clase `Assertions`. Las más frecuentes:

Aserción	Comprueba que...
<code>assertEquals(esperado, obtenido)</code>	Los dos valores son iguales
<code>assertTrue(condicion)</code> / <code>assertFalse(condicion)</code>	La condición se cumple o no
<code>assertNull(objeto)</code> / <code>assertNotNull(objeto)</code>	El objeto es o no es nulo

Aserción	Comprueba que...
<code>assertThrows(Tipo.class, () -> ...)</code>	La operación lanza la excepción esperada

5.4. Aislar el componente que se prueba

Para probar un componente por separado hay que poder darle dependencias **controladas**. Si

`DiccionarioEnFichero` leyera el fichero por su cuenta, probarlo exigiría tener ficheros de prueba en disco, y la prueba dependería de ellos.

En el proyecto se resolvió pasándole los datos por el constructor:

```
@Test
@DisplayName("Verificar que existe una palabra que sí está en el diccionario")
void verificarQueExisteUnaPalabraQueSiEstaEnElDiccionarioTest() {
    // Una tabla de datos preparada a mano para la prueba
    Map<String, List<String>> tablaDePrueba = new HashMap<>();
    tablaDePrueba.put("melón", List.of("Fruto de la planta del melón.",
                                        "Persona con pocas luces."));

    DiccionarioEnFichero diccionario = new DiccionarioEnFichero("es", tablaDePrueba);

    boolean existe = diccionario.existe("melón");

    Assertions.assertTrue(existe);
}
```

Esa tabla preparada a mano es lo que en la disciplina se llama un **doblo de prueba** (*test double*): un sustituto controlado de una dependencia real. Según cuánto imite al original recibe nombres distintos — *stub*, *mock*, *fake*— y existen librerías especializadas en construirlos, como **Mockito**.

Y aquí aparece una consecuencia que no es evidente hasta que se ve:

Un componente que no construye sus dependencias, sino que las recibe, es un componente que se puede probar. La inyección de dependencias no es sólo una cuestión de diseño elegante: es lo que hace posible probar el sistema por partes.

Es la razón por la que ninguna clase del servidor ejecuta `new` sobre sus dependencias. En una prueba se le puede pasar un repositorio falso que devuelva exactamente los datos que interesan, sin base de datos alguna.

5.5. Cobertura y control de calidad

La **cobertura de pruebas** es el porcentaje de líneas de código que las pruebas llegan a ejecutar. Se mide automáticamente con herramientas como **JaCoCo**, y se vigila con plataformas de análisis estático como **SonarQube**.

En la industria es habitual exigir entre un **80 % y un 90 %** de cobertura como condición para que el código pase a producción. Y es una comprobación automática:

No hay a quién llorar. Si no se llega al umbral, el despliegue no sale. No es una conversación con nadie, es un programa que dice que no.

⚠ La cobertura mide qué líneas se **ejecutan**, no si se **comprueban**. Es posible alcanzar un 90 % de cobertura con pruebas que no verifican nada. Es una métrica útil como suelo mínimo y engañosa como objetivo: un número alto no garantiza que las pruebas sirvan, pero un número bajo garantiza que hay código sin probar.

5.6. Estado de las pruebas en el proyecto del curso

Conviene ser explícito sobre esto. El proyecto del curso tiene **dos clases de prueba**, ambas en el módulo `diccionarios-en-ficheros`:

```
diccionarios-en-ficheros/src/test/java/com/curso/diccionarios/ficheros/  
    DiccionarioEnFicheroTest.java  
    SuministradorDeDiccionariosEnFicheroTest.java
```

Cubren el módulo que se construyó cuando se explicaron las pruebas. Los módulos posteriores —el servicio web, el acceso a base de datos, el cliente HTTP, las interfaces de usuario— **no tienen pruebas automatizadas**. En un proyecto real esto sería inaceptable y no pasaría el control de calidad descrito en el apartado anterior.

Se señala aquí de forma deliberada: es la deuda técnica más importante del proyecto y varios de los ejercicios de este manual consisten precisamente en empezar a saldarla.

Ejercicios del capítulo 5

5.1. Escribe las pruebas de `NormalizadorDeTerminos.normalizar()`. Considera al menos estos casos: texto en minúsculas, texto con espacios alrededor, texto ya en mayúsculas, cadena vacía y `null`. ¿Cuántas pruebas te salen y por qué no una sola con cinco aserciones?

5.2. `DiccionarioEnBBDD` depende de `PalabraRepository`, que es una interfaz. Explica cómo probarías el método `palabrasSimilares()` sin tener una base de datos en marcha. ¿Qué le pasarías al constructor?

5.3. Escribe una prueba que verifique que `dameSignificados()` devuelve un `Optional` vacío —y no `null`, ni una lista vacía, ni una excepción— cuando la palabra no existe. ¿Qué principio SOLID estarías protegiendo con esa prueba?

5.4. El equipo alcanza un 85 % de cobertura y el análisis de calidad da luz verde. Un mes después aparece un error grave en producción, en una línea que las pruebas sí ejecutaban. ¿Cómo es posible? ¿Qué medida pondrías además de la cobertura?

5.5. El contrato del API dice que un idioma inexistente debe devolver un 404. Con lo aprendido, describe qué tipo de prueba haría falta para verificar eso y por qué no sería una prueba unitaria. Busca en la documentación de Spring qué anotación se usa para este tipo de pruebas.

6. Evolución de las arquitecturas empresariales

6.1. Del escritorio a los componentes desacoplados

La forma de construir software en las empresas ha cambiado varias veces, y cada cambio se explica por un problema concreto que la etapa anterior no resolvía.

Aplicaciones de escritorio. Todo se ejecutaba en un ordenador. El problema era **compartir datos**: si dos personas debían trabajar sobre la misma información, no había forma limpia de conseguirlo.

Arquitectura cliente-servidor. Con la llegada de las redes, parte de la aplicación pasa al servidor. Al principio, en el servidor sólo estaba la base de datos y toda la lógica seguía en el cliente, lo que producía aplicaciones cliente muy complejas. Poco a poco la lógica de gestión de datos se fue trasladando al servidor y en el cliente quedó únicamente la representación.

Para que dos sistemas se comuniquen hacen falta dos acuerdos:

```
APP CLIENTE ←————→ APP SERVIDOR
      Protocolo:  las reglas de la comunicación
      Lenguaje:   el formato de los mensajes
```

La irrupción de la Web. En paralelo, un servicio concreto de Internet cambió todo el panorama.

⚠ **Web e Internet no son lo mismo.** *Internet* es un conjunto descentralizado de redes interconectadas que usan un protocolo común (TCP/IP). La *Web* es **uno de los servicios** que se ofrecen sobre Internet, junto al correo electrónico, la transferencia de ficheros (FTP), el acceso remoto (SSH) o la voz sobre IP.

La Web la propone **Tim Berners-Lee** en 1989, con el primer sitio funcionando en 1991, y en 1993 el CERN la libera al dominio público. Lo que define son dos cosas:

- **HTML** (*HyperText Markup Language*) — el lenguaje de los documentos.
- **HTTP** (*HyperText Transfer Protocol*) — el protocolo para transportarlos.

Al principio los documentos eran **estáticos**. Pronto se quiso que se generaran bajo demanda: ver *mi* listado de pedidos, *mis* facturas. Hizo falta un lenguaje que se ejecutara en el servidor y produjera HTML con datos actualizados de una base de datos.

```
Cliente                      Servidor
(Navegador) —petición—> Servidor de aplicaciones —> Programa —> BBDD
                  <—respuesta—
                  (HTML)
                  (Tomcat, WebSphere, WebLogic, JBoss)
```

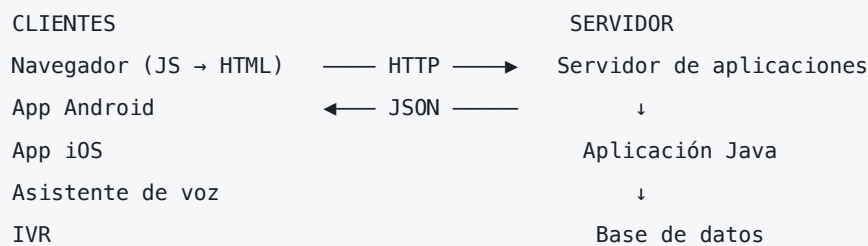
Este modelo funcionó durante años. Hoy está superado, y la razón es que **el navegador dejó de ser la única forma de acceder a los datos**.

6.2. Por qué HTML dejó de ser el formato de intercambio

Hoy conviven muchos frontales distintos: aplicaciones Android e iOS, navegadores, televisores conectados, asistentes de voz, sistemas telefónicos de atención al cliente (IVR).

HTML es un lenguaje orientado a **documentos con representación visual**: dice que un título va en negrita a 12 puntos, que una lista lleva 20 píxeles de margen, que una imagen mide 200 por 300. Eso no le sirve de nada a una aplicación Android, ni a un asistente de voz, ni a un IVR.

La industria se movió hacia formatos orientados **al dato** y no a su presentación: primero **XML** y después **JSON**, mucho más ligero.



El cambio de fondo: **antes el servidor generaba páginas; ahora genera datos**. Si hace falta una web, se escribe un programa en JavaScript que se ejecuta en el navegador, consume esos datos y construye el HTML allí mismo. Es exactamente lo que hace la aplicación Angular del proyecto.

6.3. El monolito

En paralelo, durante años se tendió a construir sistemas muy grandes que lo hicieran todo. El ejemplo analizado en clase:

Animalitos Fermín, una empresa con un único sistema que gestiona la venta de productos para mascotas, las citas veterinarias, la peluquería canina y las nóminas de los empleados.

El problema de estos sistemas no es que no funcionen: es que **cualquier cambio puede impactar en cualquier otra parte**. Tocar el cálculo de las nóminas puede romper la agenda de citas, y no hay forma barata de saberlo de antemano. Además, el sistema entero se despliega de una vez: un cambio pequeño obliga a volver a poner en producción todo lo demás.

La alternativa aprendida con los años es la **arquitectura de componentes desacoplados**: en lugar de un sistema muy grande, muchos sistemas pequeños que se comunican entre sí. Son más sencillos de mantener, y esa es la clave.

6.4. Dos herramientas para evaluar una arquitectura

Una arquitectura no se evalúa en abstracto. Se evalúa **frente a los cambios que sabemos que van a ocurrir**. En el curso se usaron dos herramientas de análisis, y son aplicables a cualquier proyecto.

Análisis por escenarios de cambio

Consiste en enumerar los cambios previsibles y estimar qué costaría cada uno. Los que se plantearon para la aplicación de diccionarios:

- Cambiar la interfaz de usuario.
- Añadir funcionalidad nueva —por ejemplo, sugerir palabras similares cuando no hay coincidencia exacta—.
- Añadir un idioma nuevo.
- Añadir o corregir palabras de un diccionario existente.
- Cambiar el algoritmo de búsqueda —por ejemplo, que deje de distinguir mayúsculas—.

Ninguno de estos escenarios es hipotético: los cinco ocurrieron durante el curso.

Un producto de software es, por definición, un producto sujeto a cambios y mantenimiento. Igual que un coche es, por definición, un producto sujeto a mantenimiento: una vez al año toca cambiar el aceite.

Análisis de impacto por implicado

El segundo análisis es menos habitual y más revelador: **un mismo cambio no cuesta lo mismo a todo el mundo**. Aplicado al escenario "hay que corregir la definición de una palabra" en la arquitectura original — una aplicación instalada en el ordenador de cada usuario, con los diccionarios dentro —:

Implicado	Consecuencia
Desarrollo	Reempaquetar y redistribuir la aplicación entera por corregir una palabra
Usuario	Reinstalar la aplicación. Y el usuario sabe de su negocio, no de instalar software
Operaciones	Publicar la nueva versión y avisar, sin visibilidad de qué versión tiene cada uno
Soporte (CAU)	Imposible reproducir una incidencia sin saber la versión que usa quien la reporta
Negocio	Dos usuarios pueden obtener respuestas distintas a la misma consulta

La última fila es la que suele decidir la discusión. Un sistema en el que dos usuarios obtienen respuestas distintas a la misma pregunta no tiene un problema técnico: tiene un problema de negocio.

Hay además un aspecto organizativo que el análisis técnico no captura: **los diccionarios los mantiene un lingüista, no un programador**. Tenerlos mezclados con el código fuente, dentro de `src/main/resources`, obliga a alguien que no sabe programar a moverse por un árbol de carpetas que no entiende, con riesgo de romper el código por error.

La arquitectura debe reflejar también **quién mantiene cada cosa**. Y el lingüista debe poder aportar su trabajo por una vía trazable y versionada, no por correo electrónico.

6.5. El criterio económico

Todo lo anterior desemboca en un único criterio, que es la justificación de fondo de las decisiones del curso:

El coste de desarrollo inicial de una arquitectura desacoplada es **mayor**. El coste de mantenimiento a lo largo del ciclo de vida es **mucho menor**. Lo que importa no es el coste inicial, sino el **coste total del ciclo de vida** (LCC, *Life Cycle Cost*).

Merece la pena detenerse en la primera frase, porque es honesta y suele omitirse: diseñar bien **cuesta más al principio**. Separar interfaces de implementaciones, montar módulos, escribir pruebas y levantar una arquitectura cliente-servidor lleva más tiempo que escribir diez ficheros en una carpeta.

La apuesta es que ese sobrecoste se recupera con creces durante los años en que el sistema se mantiene. En el proyecto del curso la apuesta se pudo verificar: pasar de ficheros a base de datos costó **un módulo nuevo y una dependencia cambiada**, y añadir una interfaz de escritorio completa costó **un módulo nuevo y una línea modificada**.

⚠ El criterio tiene un límite que también conviene tener claro: si un programa se va a usar tres veces y tirar, diseñarlo así es tirar el dinero. La pregunta correcta no es "¿cuál es la mejor arquitectura?" sino "¿cuánto tiempo va a vivir esto y cuántas veces va a cambiar?".

Ejercicios del capítulo 6

6.1. Aplica el análisis de impacto por implicado al escenario "hay que cambiar el color de fondo de la interfaz de usuario", primero en la arquitectura de la V1 (aplicación instalada en cada puesto) y después en la V5 con la interfaz web. Construye las dos tablas y compáralas.

6.2. *Animalitos Fermín* quiere partir su monolito en componentes. Propón una división en sistemas independientes, indica qué datos necesitaría compartir cada uno con los demás y señala cuál sería el punto de integración más delicado.

6.3. Enumera tres escenarios de cambio que el diseño actual del proyecto de diccionarios resolvería **mal**. Para cada uno, indica qué habría que rediseñar.

6.4. Un responsable rechaza la arquitectura cliente-servidor porque "cuesta el doble y la aplicación hace exactamente lo mismo". Prepara una respuesta de un párrafo apoyada en el coste total del ciclo de vida, con al menos un número concreto.

6.5. ¿Por qué un asistente de voz no puede consumir HTML? Describe qué recibiría y qué tendría que hacer con ello, y compáralo con lo que recibe hoy del servicio web de diccionarios.

7. Comunicación entre sistemas: HTTP y REST

7.1. Las preguntas previas

Antes de elegir una tecnología de comunicación hay que responder a dos preguntas sobre la naturaleza del diálogo.

¿Quién inicia la comunicación? En la inmensa mayoría de los casos —en torno al 90 %— es el cliente: el servidor está a la escucha y responde. Hay excepciones, como un servidor de chat que envía a un cliente los mensajes que escriben otros, y esas excepciones exigen tecnologías distintas (WebSockets, notificaciones push).

¿Qué protocolo se usa? Un protocolo es un conjunto de reglas que define cómo se comunican dos sistemas. A lo largo de la historia ha habido varios; el capítulo 8 los recorre. El que se ha impuesto es **HTTP**, y sobre él, **REST**.

7.2. Fundamentos del protocolo HTTP

HTTP tiene dos propiedades que conviene tener presentes siempre:

- **Unidireccional.** Siempre inicia el cliente, mandando una petición (*HTTP request*); el servidor contesta con una respuesta (*HTTP response*). El servidor nunca habla primero.
- **Síncrono.** El cliente queda esperando la respuesta.

```
graph LR; C[Cliente] -- "HTTP REQUEST" --> S[Servidor]; S -- "HTTP RESPONSE" --> C;
```

Cliente — HTTP REQUEST —> Servidor
← HTTP RESPONSE —

Toda petición y toda respuesta tienen **tres partes**:

Parte	Qué es
URL	La dirección a la que se hace la petición
Cabeceras (<i>headers</i>)	Metadatos: información sobre el envío
Cuerpo (<i>body</i>)	El contenido. Es opcional

La analogía que se usó en clase: una petición HTTP es como un envío por correo. Lo que se manda va dentro de una **caja** (el cuerpo). Por fuera se pega una etiqueta con información adicional (las cabeceras): quién lo envía, cuánto pesa, si es frágil, si es urgente, cuál es el motivo del envío. La caja puede ir vacía, pero la etiqueta va siempre.

De todas las cabeceras, dos son imprescindibles.

El verbo o método, que manda el cliente y es obligatorio. Indica qué se quiere hacer:

Verbo	Intención
GET	Pedir datos

Verbo	Intención
POST	Enviar datos nuevos
PUT	Modificar datos existentes
DELETE	Borrar datos

Un navegador, cuando se escribe una dirección en la barra, usa siempre GET.

El código de estado, que manda el servidor y también es obligatorio. Es un número de tres cifras cuyo primer dígito da la categoría:

Rango	Significado	Ejemplos
2xx	Todo ha ido bien	200 OK, 201 Created
3xx	Redirección: pide en otro sitio	301, 302
4xx	Error del cliente	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Error del servidor	500 Internal Server Error, 503 Service Unavailable

⚠ La distinción entre 4xx y 5xx no es cosmética: dice **de quién es la culpa**. Un 404 significa "lo que pides no existe" y el cliente no debe reintentar. Un 500 significa "no he podido atenderte" y reintentar puede tener sentido. Confundirlos lleva a clientes que reintentan indefinidamente peticiones que nunca van a funcionar.

7.3. Qué es REST

REST (*Representational State Transfer*) se describe a menudo como un protocolo, y no lo es. Es un **conjunto de restricciones sobre HTTP** para comunicar sistemas. Lo formuló Roy Fielding en su tesis doctoral en el año 2000; Fielding fue además uno de los autores de la propia especificación de HTTP.

La idea es aprovechar lo que HTTP ya ofrece en lugar de construir un mecanismo propio encima:

- **Las URL identifican recursos**, no acciones. /diccionarios/es/melón nombra una cosa; no dice qué hacer con ella.
- **El verbo dice qué se quiere hacer** con ese recurso.
- **El código de estado comunica el resultado**.
- **El cuerpo lleva los datos**, habitualmente en JSON, y sólo cuando hace falta.

Muchas veces ni siquiera hace falta cuerpo: si la información que hay que enviar es poca, viaja en la propia URL.

```
GET http://diccionarios.miempresa.com/diccionarios/es/melón
```

```
{
  "idioma": "es",
  "palabra": "melón",
  "significados": [
    "Fruta de la planta Cucumis melo, de pulpa jugosa y dulce.",
    "Persona con pocas luces, torpe o lenta para comprender."
  ]
}
```

Compárese esto con los más de trescientos caracteres de sobre XML que exigía SOAP para transportar dos palabras (capítulo 8) y se entiende por qué REST se impuso.

7.4. Diseñar el contrato antes de escribir código

Una URL tiene partes que cambian y partes que no:

```
http://servidor:puerto/ruta

Desarrollo:  localhost:8080
Producción:  diccionarios.miempresa.com:443  (HTTPS)
```

El servidor y el puerto cambian entre instalaciones. **La ruta no.** Por eso el diseño del API consiste en definir las rutas y, para cada una, qué se responde en cada situación posible.

Éste es el contrato que se definió en el curso, **antes** de escribir el servidor:

Petición	Situación	Estado	Cuerpo
GET /diccionarios	Siempre	200	["EN", "ELFICO", "ES"]
GET /diccionarios/{idioma}	El diccionario existe	200	(vacío)
GET /diccionarios/{idioma}	No existe	404	(vacío)
GET /diccionarios/{idioma}/{palabra}	Ambos existen	200	{"idioma": "...", "palabra": "...", "significados": [...]}
GET /diccionarios/{idioma}/{palabra}	El idioma existe, la palabra no	404	{"idioma": "...", "similares": [...]}
GET /diccionarios/{idioma}/{palabra}	El idioma no existe	404	(sin datos)
cualquiera	Error no controlado	500	El mensaje de error

Ese contrato es lo que se entrega al equipo que va a construir el cliente. A partir de ese momento, los dos equipos pueden trabajar en paralelo sin volver a hablarse.

Ésa es la función real del contrato, y es la razón de que se escriba primero.

OpenAPI. El estándar actual para escribir ese contrato de manera formal se llama **OpenAPI**; sus versiones 1 y 2 se llamaron **Swagger**. Es un fichero —en JSON o YAML— que describe las rutas, los parámetros y las respuestas de forma neutra respecto al lenguaje de programación, y a partir del cual se pueden generar automáticamente documentación y código de cliente.

Conceptualmente, **OpenAPI es el equivalente moderno del IDL de CORBA** (capítulo 8): resuelve el mismo problema —describir un contrato para que sistemas heterogéneos se entiendan— con muchísimo menos esfuerzo.

⚠ En el proyecto del curso el contrato se escribió en prosa y en tablas, no en OpenAPI. Existen librerías que lo generan automáticamente a partir de las anotaciones de Spring (`springdoc-openapi`), pero no se llegaron a incorporar.

7.5. Una llamada local no es una llamada remota

Éste es el punto más importante del capítulo, y el que explica por qué fracasaron las tecnologías del capítulo siguiente.

	Llamada local	Llamada distribuida
Coste	Nanosegundos	Milisegundos: entre 10.000 y 1.000.000 de veces más cara
Datos	Se pasa una referencia en memoria	Hay que serializar , enviar y deserializar
Fallos	Se ejecuta, o lanza una excepción	Puede además no llegar, llegar y no volver, o tardar indefinidamente
Acoplamiento	El compilador verifica los tipos	El contrato es un acuerdo externo: si el servidor cambia el JSON, el cliente no falla al compilar, falla en ejecución
Versionado	Todo se despliega junto	Cliente y servidor evolucionan por separado y deben mantener compatibilidad

De esta tabla se derivan tres decisiones de diseño que se tomaron en el proyecto:

Enviar el resultado completo en una sola respuesta. El DTO `RespuestaPalabra` lleva la lista entera de significados, en lugar de obligar a una llamada por significado. **Minimizar el número de viajes por la red es la regla número uno del diseño distribuido.**

Usar los códigos de estado HTTP para distinguir situaciones, en lugar de excepciones Java, porque las excepciones no cruzan la red.

Reservar el `500` para "no puedo atenderte". En un sistema distribuido, "no encontrado" y "no disponible" son cosas distintas y el cliente tiene que poder diferenciarlas para decidir si reintenta.

La promesa de RMI y CORBA era hacer que una llamada remota **pareciera** local. Esa promesa no se puede cumplir, porque la red no es fiable, no es instantánea y no tiene ancho de banda infinito. La diferencia de REST es que el programador sabe en todo momento que está cruzando la red.

7.6. Herramientas para probar un API

Para probar servicios REST sin escribir código:

- **curl**, disponible en cualquier terminal:

```
curl -i http://localhost:8080/diccionarios/es/casa
```

El modificador **-i** muestra también las cabeceras y el código de estado, que es justo lo que interesa comprobar.

- **Extensiones de navegador** como *Boomerang (SOAP & REST client)*, disponible para Firefox y Chrome, que permiten componer peticiones de forma visual.
- **Postman** o **Insomnia**, aplicaciones especializadas que además guardan colecciones de peticiones y permiten compartirlas con el equipo.

Ejercicios del capítulo 7

7.1. El servicio devuelve **404** tanto cuando el idioma no existe como cuando existe pero la palabra no. Un cliente necesita distinguir los dos casos. ¿Cómo lo hace hoy con el contrato actual? Propón una alternativa que no dependa del cuerpo de la respuesta.

7.2. Diseña el contrato REST completo para añadir tres operaciones nuevas: dar de alta una palabra en un diccionario, modificar sus significados y borrarla. Indica verbo, ruta, códigos de estado de éxito y de error, y cuerpo en cada caso.

7.3. Un compañero propone la ruta `GET /borrarPalabra?idioma=es&palabra=melon`. Enumera todos los problemas que tiene esa propuesta desde el punto de vista de REST.

7.4. El servidor cambia el nombre del campo `significados` a `definiciones`. ¿Cuándo se enterará el cliente Java del proyecto? ¿Y el cliente Angular? ¿Qué estrategia usarías para introducir ese cambio sin romper a los clientes existentes?

7.5. Calcula, con los datos de la tabla del apartado 7.5, cuántas veces más costosa es una búsqueda que hace tres llamadas HTTP frente a una que hace una sola, suponiendo 40 ms por llamada. Traduce el resultado a la espera que percibe un usuario que hace 50 búsquedas en una sesión.

8. Tecnologías heredadas y sus equivalentes actuales

El temario clásico de Java Enterprise Edition dedica sus primeros bloques a las tecnologías con las que se comunicaban los sistemas Java distribuidos entre 1997 y 2006 aproximadamente. Todas han sido desplazadas por REST sobre HTTP.

Conviene conocerlas por dos razones. La primera, práctica: es muy posible encontrárselas en sistemas heredados (*legacy*) y hay que saber qué son. La segunda, formativa: entender **por qué fracasaron** enseña más sobre diseño de sistemas distribuidos que estudiar la tecnología que ganó.

8.1. RMI (Remote Method Invocation)

Qué era. Un mecanismo de Java que permitía invocar métodos de un objeto que en realidad se estaba ejecutando en otra máquina. Desde el código, la llamada parecía local:

```
Diccionario diccionario = (Diccionario) Naming.lookup("rmi://servidor/diccionario-es");
diccionario.dameSignificados("melón"); // Esta llamada viaja por la red
```

Requería definir interfaces que extendieran `java.rmi.Remote`, generar clases intermedias —*stubs* en el cliente y *skeletons* en el servidor— con la herramienta `rmic`, y arrancar un registro de nombres (`rmiregistry`) donde publicar los objetos.

Por qué se dejó de usar.

- **Sólo funciona entre programas Java.** Un cliente Android nativo, una web en JavaScript o un sistema en Python no pueden hablar RMI.
- **Usa puertos propios y serialización binaria**, lo que lo hace difícil de atravesar cortafuegos y servidores intermedios.
- **La serialización nativa de Java** ha sido durante años una de las principales fuentes de vulnerabilidades graves de seguridad en la plataforma.
- **Oculto que la llamada es remota.** Esto suena a ventaja y es precisamente el problema: hace creer al programador que una llamada por red se comporta como una llamada local, cuando —según la tabla del capítulo 7— no se le parece en nada.

Estado actual. RMI sigue existiendo en la JVM, pero no se usa para sistemas nuevos. Su subsistema de activación se eliminó en Java 17.

Qué lo ha sustituido. REST sobre HTTP y, para comunicaciones internas de alto rendimiento entre servicios, **gRPC**.

8.2. CORBA, IDL, IIOP y RMI-IIOP

Qué eran. CORBA (*Common Object Request Broker Architecture*) era un estándar del OMG para hacer lo mismo que RMI pero **entre lenguajes distintos**: un cliente Java podía invocar un objeto escrito en C++ o

en COBOL.

Las piezas que aparecen en el temario clásico encajan así:

Pieza	Función
IDL (<i>Interface Definition Language</i>)	Lenguaje neutro para describir las interfaces remotas. Se escribía un fichero <code>.idl</code> y un compilador generaba el código en Java, C++, etc.
Java IDL / paquete <code>org.omg</code>	La implementación de CORBA incluida en el JDK
ORB (<i>Object Request Broker</i>)	El intermediario que localizaba el objeto remoto y transportaba la llamada
COSNaming (<code>org.omg.CosNaming</code>)	El servicio de nombres: registrar y localizar objetos por nombre
IOP (<i>Internet Inter-ORB Protocol</i>)	El protocolo binario por el que viajaban las llamadas entre ORBs
RMI-IOP	Un puente: escribir con la API de RMI pero transportar por IOP, para interoperar con CORBA. Usaba <code>PortableRemoteObject.narrow()</code> en lugar del cast directo

Por qué se dejó de usar.

- **Complejidad muy alta.** Definir el IDL, compilarlo, generar código, configurar el ORB, gestionar el servicio de nombres... para cada operación.
- **Protocolo binario propio**, muy difícil de depurar y de atravesar cortafuegos.
- **La interoperabilidad entre ORBs de distintos fabricantes** —que era justamente su razón de ser— nunca llegó a funcionar bien del todo en la práctica.
- La llegada de HTTP como transporte universal dejó sin sentido mantener un protocolo específico.

Estado actual. CORBA fue eliminado del JDK en Java 11 (propuesta JEP 320), junto con el compilador `idlj` y el resto del módulo. En Java 21 sencillamente ya no existe.

Qué lo ha sustituido. REST más **OpenAPI** cubre hoy el mismo objetivo —describir un contrato de forma neutra al lenguaje para que sistemas heterogéneos se comuniquen— con una fracción del esfuerzo.

8.3. JNDI y los servicios de nombres

Qué era. JNDI (*Java Naming and Directory Interface*) es una API para localizar recursos por nombre. En las aplicaciones J2EE clásicas el código **no creaba sus dependencias**: las pedía a un directorio central del servidor de aplicaciones.

```
Context contexto = new InitialContext();
DataSource origenDeDatos = (DataSource) contexto.lookup("java:comp/env/jdbc/MiBaseDeDatos");
```

El servidor de aplicaciones —WebSphere, WebLogic, JBoss— tenía configurada la conexión a la base de datos, y la aplicación la localizaba por su nombre lógico. Así, cambiar de base de datos no obligaba a recompilar la aplicación.

Estado actual. JNDI sigue en el JDK y los servidores de aplicaciones clásicos lo soportan. Lo que ha cambiado es que ya casi nunca se escribe código de `lookup` a mano.

Qué lo ha sustituido: la inyección de dependencias. Y aquí hay una continuidad que merece señalarse, porque la idea de JNDI y la de Spring son **exactamente la misma**: que el componente no construya sus dependencias, sino que las obtenga de un directorio central. Lo que ha cambiado es la dirección:

Modelo	Cómo funciona	Nombre del patrón
JNDI (antiguo)	El componente va a buscar lo que necesita	<i>Service Locator</i>
Spring (actual)	El contenedor le entrega al componente lo que necesita	<i>Inyección de Dependencias</i>

Compárese el ejemplo de JNDI de arriba con lo que se escribió en el proyecto:

```
public DiccionariosRestController(SuministradorDeDiccionarios suministradorDeDiccionarios) {  
    this.suministradorDeDiccionarios = suministradorDeDiccionarios;  
}
```

El controlador no busca nada, no conoce ninguna cadena de texto y no depende de ninguna API de directorio. **Sólo declara qué necesita**, y el contenedor se lo entrega. La diferencia es sustancial: con JNDI, si el nombre lógico estaba mal escrito, el fallo aparecía en ejecución; con inyección de dependencias, si falta un componente, la aplicación no arranca.

En sistemas distribuidos actuales, la parte de "localizar dónde está el servicio" la resuelve el *service discovery* de la plataforma: el DNS interno de Kubernetes, Consul o Eureka.

8.4. SOAP

Qué era. El paso intermedio entre CORBA/RMI y REST: mensajes XML enviados dentro de peticiones HTTP, con el contrato descrito en un fichero **WSDL**.

Que viajara sobre HTTP fue su gran acierto —atravesaba cortafuegos— y lo que le permitió desplazar a CORBA. Su problema fue el peso:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:dic="http://www.diccionarios.com">
  <soap:Header/>
  <soap:Body>
    <dic:buscarPalabra>
      <dic:palabra>melón</dic:palabra>
      <dic:idioma>es</dic:idioma>
    </dic:buscarPalabra>
  </soap:Body>
</soap:Envelope>
```

Para transmitir siete caracteres útiles se envían más de trescientos. Eso es un sinsentido.

Y la respuesta tenía la misma estructura y el mismo peso. Súmese que programarlo era notablemente más complejo que hacer una petición HTTP corriente.

Estado actual. JAX-WS, la API de Java para SOAP, también se eliminó del JDK en Java 11 y hay que añadirla como dependencia externa. No se arrancan proyectos nuevos con SOAP salvo por imposición de un tercero, pero **sigue muy presente en producción** en banca, seguros y administración pública.

⚠ De todas las tecnologías de este capítulo, SOAP es con diferencia la que más probabilidades hay de encontrarse en un puesto de trabajo.

8.5. Tabla de equivalencias

Resumen de todo el capítulo, y del recorrido del temario clásico al estado actual:

Concepto del temario original	Equivalente actual
RMI, CORBA / IIOP	REST sobre HTTP; gRPC en comunicaciones internas
IDL (definición de interfaces)	OpenAPI (antes Swagger)
Servicio de nombres JNDI / COSNaming	Inyección de dependencias (contenedor IoC de Spring)
Localizar un servidor remoto	<i>Service discovery</i> (DNS de Kubernetes, Consul, Eureka)
XML como formato de intercambio	JSON
Servidor de aplicaciones instalado (WebSphere, WebLogic, JBoss)	Tomcat embebido dentro del propio <code>.jar</code>
Descriptores XML (<code>web.xml</code> , <code>ejb-jar.xml</code>)	Anotaciones (<code>@RestController</code> , <code>@Entity</code>) y <code>application.properties</code>
EJB <i>Session Bean</i>	Componente de Spring (<code>@Component</code> , <code>@Service</code>)
EJB <i>Entity Bean</i>	Entidad JPA (<code>@Entity</code>)

Concepto del temario original	Equivalente actual
DAO escrito a mano con JDBC y SQL	Repositorio de Spring Data JPA
J2EE (<i>Java 2 Enterprise Edition</i>)	Jakarta EE (paquetes <code>jakarta.*</code> en lugar de <code>javax.*</code>)

8.6. Qué lección queda

Hay un patrón común en los cuatro fracasos de este capítulo, y es la lección que merece la pena llevarse:

Las tecnologías que intentaron **ocultar** la complejidad de la red fracasaron. La que triunfó es la que la deja a la vista.

RMI y CORBA prometían que una llamada remota se pareciera a una local. SOAP prometía que la interoperabilidad se resolvería con un contrato exhaustivo y un tipado estricto. En ambos casos la complejidad no desapareció: se desplazó a la configuración, a las herramientas de generación de código y a los errores en tiempo de ejecución.

REST no oculta nada: hay una URL, un verbo, un código de estado y, si acaso, un JSON. Se puede depurar con `curl` y leer con los ojos. Esa transparencia, que técnicamente parece una limitación, es la razón de su éxito.

Ejercicios del capítulo 8

- 8.1. Un sistema heredado expone su funcionalidad por RMI y hay que consumirlo desde una aplicación web moderna. Describe la arquitectura que propondrías sin tocar el sistema heredado. ¿Qué patrón de los del capítulo 4 estarías aplicando?
- 8.2. Compara el tamaño en bytes de la petición SOAP del apartado 8.4 con la petición REST equivalente del capítulo 7. Calcula el sobrecoste porcentual y estima cuántos megabytes de tráfico adicional supondría en un servicio con un millón de peticiones diarias.
- 8.3. Explica con tus palabras la diferencia entre *Service Locator* e *Inyección de Dependencias*, y por qué el segundo se considera preferible. Pon un ejemplo de error que el primero detecta en ejecución y el segundo al arrancar.
- 8.4. El temario original dedica un bloque a los EJB. Busca qué eran un *Session Bean* y un *Entity Bean* e identifica, en el proyecto de diccionarios, qué clase concreta cumple hoy cada uno de esos dos papeles.
- 8.5. ¿Por qué la eliminación de CORBA del JDK en Java 11 no rompió ninguna aplicación moderna, mientras que un cambio equivalente en la API de colecciones sería impensable? Relaciona la respuesta con el versionado semántico del capítulo 1.

9. Spring y Spring Boot

9.1. Framework frente a librería

La distinción es la más importante del capítulo y se enuncia en dos frases:

Una **librería** la integro yo en mi programa: yo llamo a su código cuando lo necesito.

Un **framework** no lo integro: **construyo mi programa alrededor de él**, y es él quien llama a mi código.

Un ejemplo de librería: una que genera informes en PDF. En el punto del programa donde hace falta un informe se llama a su función, se le pasan los datos y devuelve el fichero. El programa manda; la librería obedece.

Con un framework la relación se invierte. El framework trae librerías, pero además **impone una forma de hacer las cosas**: dónde va cada pieza, cómo se llaman, cuándo se ejecutan. A cambio de aceptar esas reglas, escribe por nosotros la mayor parte del sistema.

Spring es un framework de Java: más de doscientas librerías que cubren los tipos de aplicación empresarial más habituales. Como casi todo el mundo construye básicamente los mismos tipos de aplicación, Spring los estandariza.

Spring Boot es una de esas doscientas librerías, y resuelve un problema muy concreto: hasta que apareció, **configurar un proyecto Spring para empezar a trabajar era una tarea ingente**. Spring Boot elimina esa configuración inicial tomando decisiones razonables por defecto.

9.2. Los starters

Un **starter** es un paquete de dependencias que cubre un tipo de proyecto completo. En lugar de averiguar qué treinta librerías hacen falta para exponer un servicio web y qué versiones son compatibles entre sí, se declara una sola:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
  <version>${version.de.spring}</version>
</dependency>
```

Algunos de los más usados:

Starter	Para qué
spring-boot-starter-webmvc	Servicios web y controladores REST
spring-boot-starter-data-jpa	Acceso a bases de datos relacionales

Starter	Para qué
<code>spring-boot-starter-security</code>	Autenticación y autorización
<code>spring-boot-starter-test</code>	Pruebas automatizadas

9.3. Inversión de control

Spring es lo que se denomina un **contenedor de inversión de control** (*IoC container*). La idea, formulada de la manera más directa posible: **el flujo de la aplicación deja de escribirlo el programador**.

El flujo de una aplicación es, básicamente, lo que se pone en el método `main`. En una aplicación Spring ese método existe, pero tiene **una sola línea**, y es la misma en todas las aplicaciones Spring del mundo:

```
@SpringBootApplication
public class ServicioWeb {
    public static void main(String[] args) {
        SpringApplication.run(ServicioWeb.class, args);
    }
}
```

Traducido: "*Spring, arranca mi aplicación*". A partir de ahí, quién se construye, en qué orden y cuándo se ejecuta cada cosa lo decide el framework.

Esto es desconcertante la primera vez: uno lee el código fuente completo del proyecto y no encuentra escrito en ninguna parte por qué el programa hace lo que hace. **Ésa es exactamente la diferencia entre un framework y una librería.**

9.4. Inyección de dependencias

Si el programador no construye los objetos, ¿cómo obtiene un componente lo que necesita? Lo **declara** en su constructor, y el contenedor se lo entrega:

```
@RestController
public class DiccionariosRestController {

    private final SuministradorDeDiccionarios suministradorDeDiccionarios;

    public DiccionariosRestController(SuministradorDeDiccionarios suministrador) {
        this.suministradorDeDiccionarios = suministrador;
    }
}
```

Esta clase no ejecuta `new` sobre nada, no sabe de dónde salen los diccionarios y no menciona ninguna implementación concreta. Es el principio de inversión de dependencias (capítulo 3) aplicado por el framework.

¿Y quién decide qué implementación concreta se entrega? Una anotación sobre la clase que se quiere ofrecer:

```
@Component
public class SuministradorDeDiccionariosEnBBDD implements SuministradorDeDiccionarios { ... }
```

Con eso basta: cuando alguien pida un `SuministradorDeDiccionarios`, Spring entregará una instancia de esta clase. Y será **siempre la misma instancia**, porque los componentes de Spring son *singletons* por defecto.

⚠ Si dos clases distintas implementan la misma interfaz y ambas están anotadas como componentes, Spring no sabrá cuál entregar y la aplicación **fallará al arrancar**. Se resuelve con `@Primary` o con `@Qualifier`. Que falle al arrancar y no en mitad de una petición es, precisamente, la ventaja frente al modelo antiguo de JNDI.

La alternativa explícita. Spring permite también declarar la construcción a mano, con una clase de configuración. Es el equivalente exacto de la factoría del capítulo 4:

```
@Configuration
public class SuministradorDeDiccionariosConfiguration {

    @Bean
    public SuministradorDeDiccionarios dameSuministrador() {
        return new SuministradorDeDiccionariosEnFicheros("diccionarios");
    }
}
```

En el proyecto se escribieron las dos versiones y se conserva la primera comentada, para poder compararlas. `@Component` es más cómodo; `@Bean` es imprescindible cuando la construcción requiere lógica o cuando la clase no es nuestra y no se le puede poner una anotación.

9.5. El controlador REST

Traducir peticiones HTTP en llamadas a Java se reduce a dos anotaciones:

```

@RestController
public class DiccionariosRestController {

    @GetMapping("/diccionarios/{idioma}")
    public ResponseEntity<Void> existeDiccionarioDe(@PathVariable("idioma") String idioma) {
        if (suministradorDeDiccionarios.tienesDiccionarioDe(idioma)) {
            return ResponseEntity.ok().build(); // 200
        } else {
            return ResponseEntity.notFound().build(); // 404
        }
    }
}

```

Elemento	Qué hace
@RestController	Marca la clase como componente que define rutas HTTP
@GetMapping("/")	Asocia un método al verbo GET y a una ruta
{idioma}	Parte variable de la ruta
@PathVariable	Extrae esa parte variable y la pasa como argumento
ResponseEntity<T>	La respuesta HTTP: código de estado y, opcionalmente, cuerpo de tipo T

Con eso, Spring se encarga de arrancar el servidor, registrar las rutas, extraer los parámetros de la URL, invocar el método correcto y **convertir a JSON** el objeto devuelto —de esto último se ocupa una librería llamada **Jackson**—.

Si no hay cuerpo que devolver se usa `ResponseEntity<Void>`.

9.6. Dónde busca Spring los componentes

`@SpringBootApplication` hace que Spring busque componentes automáticamente, pero sólo **en el paquete de la clase principal y sus subpaquetes**. En un proyecto multimódulo esto deja de ser suficiente.

En el proyecto del curso, la clase principal está en `com.curso.diccionarios.servicioweb` y el componente que hay que encontrar está en `com.curso.diccionarios.bbdd`, que no es un subpaquete del anterior. Por eso hizo falta ampliar el ámbito de búsqueda:

```

@SpringBootApplication(scanBasePackages = {"com.curso.diccionarios"})
@EnableJpaRepositories(basePackages = {"com.curso.diccionarios.bbdd.repositorios"})
@EntityScan(basePackages = {"com.curso.diccionarios.bbdd.entidades"})
public class ServicioWeb { ... }

```

⚠ Este es uno de los errores más frecuentes al modularizar una aplicación Spring: el código compila perfectamente y la aplicación falla al arrancar diciendo que no encuentra un componente que está claramente ahí. La causa casi siempre es ésta.

9.7. Configuración externa

Spring lee al arrancar un fichero `application.properties` (o `application.yml`) en `src/main/resources`. Cualquier valor de ese fichero se puede inyectar en un campo:

```
@Value("${diccionarios.carpeta:diccionarios}")
private String carpetaDeDiccionarios;
```

La sintaxis dice: usa la propiedad `diccionarios.carpeta` y, si no existe, usa `diccionarios` como valor por defecto. Éste es el equivalente moderno de la configuración que antes vivía en el servidor de aplicaciones, y el mecanismo que permite que el mismo `.jar` funcione en desarrollo y en producción con configuraciones distintas.

9.8. El servidor embebido

Un **servidor de aplicaciones** es el programa que aloja la aplicación, abre un puerto de red, recibe las peticiones HTTP y las encamina. Los clásicos de J2EE se instalaban aparte y en ellos se desplegaba un `.war` o un `.ear`: **WebSphere** (IBM), **WebLogic** (Oracle), **JBoss/WildFly** (Red Hat) y **Apache Tomcat**.

El modelo actual es el inverso: **Tomcat embebido**. Spring Boot mete el servidor *dentro* del `.jar` de la aplicación. No hay nada que instalar ni configurar en la máquina de destino: basta con Java y ejecutar el `.jar`.

```
Modelo clásico:  instalar WebSphere → desplegar mi-app.war dentro
Modelo actual:   java -jar mi-app.jar    (el servidor va dentro)
```

La consecuencia va más allá de la comodidad: es lo que hace posible empaquetar una aplicación en un contenedor y desplegarla en cualquier plataforma sin preparar la máquina.

9.9. La magia de Spring: las dos caras

Conviene ser explícito sobre el precio de todo lo anterior.

A favor. Desarrollo mucho más rápido, muchísimo menos código propio y un estándar de facto: cualquier desarrollador reconoce la estructura de un proyecto Spring en cualquier empresa, lo que permite cambiar de proyecto sin aprender nada nuevo.

En contra. Hace mucha "magia" por debajo.

Si no se entiende qué está haciendo el framework, se llega a un punto en el que se lee el código fuente y no se entiende por qué el programa funciona, porque una gran parte del comportamiento no está escrita en ningún fichero del proyecto.

De ahí el enfoque seguido en este curso: enseñar primero cómo se hacía a mano —la factoría, el script SQL, el proxy de gestión de errores— y sólo después qué parte de ese trabajo asume el framework. Saber qué hay debajo es lo que separa a quien usa Spring de quien depende de él.

Ejercicios del capítulo 9

- 9.1.** Se añade al proyecto una segunda clase anotada con `@Component` que también implementa `SuministradorDeDiccionarios`. Describe qué ocurre al arrancar, en qué momento exacto, y dos formas de resolverlo indicando cuándo usarías cada una.
- 9.2.** Escribe la clase de configuración con `@Bean` que entregaría un `SuministradorDeDiccionariosEnServicioWeb` cuya URL se lea de `application.properties`, con `http://localhost:8080` como valor por defecto.
- 9.3.** Explica por qué `DiccionarioEnBBDD` **no** lleva la anotación `@Component`, mientras que `SuministradorDeDiccionariosEnBBDD` sí. Fíjate en quién crea cada uno.
- 9.4.** Un compañero mueve la clase `ServicioWeb` al paquete `com.curso.arranque` y la aplicación deja de encontrar los componentes. Explica la causa y da dos soluciones posibles, una tocando la anotación y otra tocando la estructura de paquetes.
- 9.5.** Enumera cinco cosas que Spring hace por nosotros en este proyecto y que, en la época de J2EE, había que escribir o configurar a mano. Para cada una, indica qué fichero o herramienta lo hacía entonces.
-

10. Persistencia: Jakarta EE, JPA e Hibernate

10.1. Qué es JEE

Antes se llamaba **J2EE** (*Java 2 Enterprise Edition*). Hoy se llama **Jakarta EE**, tras la cesión de la plataforma por parte de Oracle a la Fundación Eclipse en 2017. Ése es el motivo de que los paquetes se llamen ahora `jakarta.persistence.*` y no `javax.persistence.*`.

⚠ Ese cambio de nombre de paquete es la causa de una de las incompatibilidades más molestas al actualizar proyectos antiguos: código que compilaba perfectamente deja de hacerlo, y la única solución es cambiar los `import` uno a uno.

JEE **no es una librería, sino una colección de estándares** que definen cómo debe construirse una aplicación empresarial en Java. El que se ha usado a fondo en este curso es **JPA** (*Java Persistence API*), el estándar de persistencia en bases de datos relacionales.

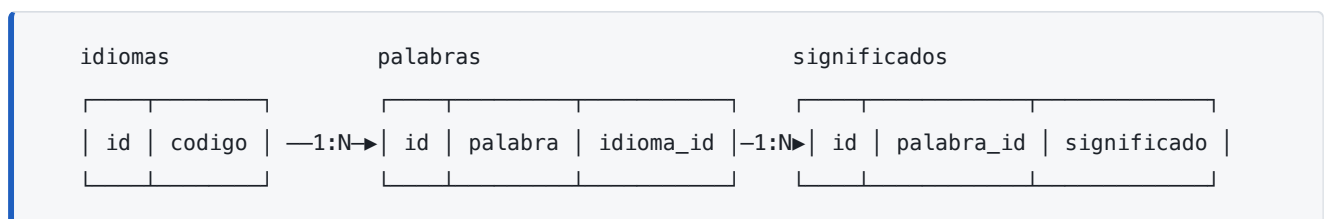
La distinción entre las tres piezas es importante y se confunde con frecuencia:

	Qué es
JPA	El estándar : define las anotaciones (<code>@Entity</code> , <code>@Id</code> , <code>@ManyToOne</code>) y el comportamiento esperado
Hibernate	Una implementación concreta de ese estándar
Spring Data JPA	Una capa por encima que elimina el código repetitivo de los repositorios

Es exactamente la misma relación que hay entre una interfaz y sus implementaciones, aplicada a escala de industria.

10.2. El modelo entidad-relación

Antes de escribir código, el modelo de datos. Para el proyecto de diccionarios:



Las relaciones se leen en los dos sentidos, y ambos importan:

Desde...	Relación
El idioma	Un idioma tiene muchas palabras → <i>one-to-many</i>
La palabra	Muchas palabras pertenecen a un idioma → <i>many-to-one</i>
La palabra	Una palabra tiene muchos significados → <i>one-to-many</i>

Desde...	Relación
El significado	Muchos significados pertenecen a una palabra → <i>many-to-one</i>

Y las **restricciones de integridad**, que son parte del diseño y no un detalle:

- `codigo` es único en `idiomas`: no puede haber dos idiomas con el mismo código.
- La combinación (`palabra`, `idioma_id`) es única: la misma palabra puede existir en varios idiomas, pero no dos veces en el mismo.
- La combinación (`significado`, `palabra_id`) es única: un significado no puede repetirse dentro de la misma palabra.

Las restricciones no son una molestia burocrática: son la última línea de defensa de la coherencia de los datos. Si el programa tiene un error, la base de datos impide que ese error se convierta en datos corruptos.

10.3. El trabajo que ya no hacemos

Antiguamente, con el modelo terminado, tocaba escribir a mano el script de creación y de carga:

```
CREATE TABLE Idiomas (
    ID INT PRIMARY KEY,
    codigo VARCHAR(10) NOT NULL UNIQUE
);

CREATE TABLE Palabras (
    ID INT PRIMARY KEY,
    palabra VARCHAR(100),
    idioma_id INT,
    UNIQUE (palabra, idioma_id),
    FOREIGN KEY (idioma_id) REFERENCES Idiomas(ID)
);

CREATE TABLE Significados (
    ID INT PRIMARY KEY,
    palabra_id INT,
    significado VARCHAR(255),
    UNIQUE (palabra_id, significado),
    FOREIGN KEY (palabra_id) REFERENCES Palabras(ID)
);

INSERT INTO Idiomas (ID, codigo) VALUES (1, 'es'), (2, 'en'), (3, 'elfico');
```

Y después, cada consulta del programa era una sentencia SQL escrita a mano dentro del código Java.

Todo ese trabajo lo hace hoy Hibernate. No se escribe una sola línea de SQL en todo el proyecto. Hibernate es un **ORM** (*Object Relational Mapping*): una librería que traduce entre objetos Java y tablas relacionales en ambos sentidos.

Ese es el patrón que el temario clásico llama **Domain Store**: que el programador trabaje con objetos y no con filas y columnas.

10.4. Las entidades

Una **entidad** es una clase Java cuyos objetos se guardan en una tabla. Se declara con anotaciones:

```
@Entity
@Table(name = "idiomas")
public class Idioma {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "codigo", nullable = false, unique = true)
    private String codigo;

    // getters y setters
}
```

Anotación	Qué indica
@Entity	Esta clase se persiste en la base de datos
@Table(name = ...)	En qué tabla
@Id	Cuál es la clave primaria
@GeneratedValue	Que el valor lo genera la base de datos
@Column(...)	Nombre de columna y restricciones: nullable, unique

Las relaciones se declaran en los dos extremos:

```
// En Palabra
@ManyToOne
private Idioma idioma; // muchas palabras, un idioma

@OneToMany(mappedBy = "palabra", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Significado> significados;
```

Dos atributos merecen explicación:

- `mappedBy` indica cuál de los dos extremos es el "dueño" de la relación, es decir, en qué tabla está la columna que la materializa. Aquí, en `Significado`.
- `cascade = CascadeType.ALL` propaga las operaciones: al guardar una palabra se guardan también sus significados, sin necesidad de guardarlos uno a uno.

⚠ Cuando se usan relaciones bidireccionales hay que **enganchar los dos lados** antes de guardar. En el proyecto, `CargadorDeDatos` asigna a cada `Significado` su `Palabra` y a la `Palabra` su lista de significados; sólo entonces hace un único `save()`.

10.5. Los repositorios

Una cosa son los datos y otra las operaciones que se pueden hacer sobre ellos. Ese segundo componente es el **repositorio**, y aquí ocurre algo que rompe con todo lo visto hasta ahora en el curso:

Hasta este punto siempre definíamos una interfaz y escribíamos una o varias clases que la implementarían. Con los repositorios **sólo se declara la interfaz**: la implementación la genera Spring Data JPA al arrancar.

```
public interface IdiomaRepository extends JpaRepository<Idioma, Integer> {
    boolean existsByCodigo(String codigo);
}
```

Al heredar de `JpaRepository`, la interfaz recibe gratis unas treinta operaciones:

Método	Qué hace
<code>save(entidad)</code>	Crea o actualiza
<code>findById(id)</code>	Busca por clave primaria
<code>findAll()</code>	Devuelve todos
<code>delete(entidad)</code>	Borra
<code>count()</code>	Cuenta
<code>existsById(id)</code>	Comprueba existencia

Consultas derivadas del nombre. Cuando hace falta algo que no está entre las operaciones estándar, no se implementa: **se declara con un nombre que siga la convención**, y Spring deduce de él la consulta SQL.

```
boolean existsByCodigo(String codigo);
Optional<Palabra> findByPalabraAndIdioma_Codigo(String palabra, String codigo);
List<Palabra> findByIdioma_Codigo(String codigo);
```

Se lee de izquierda a derecha: `findBy` + campo + `And` + campo. El guion bajo (`Idioma_Codigo`) permite navegar a un campo de la entidad relacionada.

Existen además sufijos que modifican la consulta, como `IgnoreCase`, `StartingWith`, `Containing`, `OrderBy` o `Between`:

```
Optional<Palabra> findByPalabraIgnoreCaseAndIdioma_CodigoIgnoreCase(String palabra, String codigo);
```

Y si la convención no da para tanto, siempre se puede escribir la consulta a mano con `@Query`:

```
@Query("SELECT p FROM Palabra p JOIN p.idioma i " +
        "WHERE LOWER(p.palabra) = LOWER(:palabra) AND LOWER(i.codigo) = LOWER(:codigo)")
Optional<Palabra> buscar(@Param("palabra") String palabra, @Param("codigo") String codigo);
```

10.6. Configuración del acceso a datos

Spring busca al arrancar la configuración de la base de datos en `application.properties`:

```
# Ejemplo para Oracle
spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
spring.jpa.database-platform=org.hibernate.dialect.Oracle10gDialect
```

```
# Ejemplo para MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/mi_bbdd
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Cada base de datos tiene su **driver JDBC** y su **dialecto** de SQL, y Spring necesita saber ambos para generar las consultas correctas.

H2 en desarrollo. Para desarrollar y probar no hace falta instalar un motor de base de datos: basta con declarar la dependencia de **H2**, una base de datos que se ejecuta **en memoria** y desaparece al parar la aplicación.

Al no encontrar configuración de ninguna otra base de datos, Spring detecta H2, la arranca y la configura solo. **No hace falta ni crear el fichero** `application.properties` —y de hecho el proyecto del curso no lo tiene—.

La ventaja: cada ejecución parte de un entorno limpio, que es justo lo que interesa para probar. El inconveniente, evidente: nada de lo que se guarde sobrevive al reinicio.

10.7. Carga inicial y transacciones

Si la base de datos se crea vacía en cada arranque, hay que llenarla. Spring ofrece un tipo de componente que se ejecuta automáticamente al arrancar:

```
@Component
public class CargadorDeDatos implements CommandLineRunner {

    @Override
    @Transactional
    public void run(String... args) throws Exception {
        if (idiomaRepository.count() > 0) {
            return; // ya hay datos: no hacer nada
        }
        // leer los ficheros del classpath y volcarlos en la BBDD
    }
}
```

La anotación `@Transactional` merece un apartado propio, porque su efecto se midió en clase y fue espectacular.

Sin ella, cada `save()` provoca su propia confirmación de escritura (*commit*) en la base de datos. Un *commit* es una operación costosa: obliga a garantizar que el dato está escrito de forma duradera. Con un diccionario de cientos de miles de palabras, eso son cientos de miles de *commits*.

Con `@Transactional`, todas las inserciones ocurren dentro de **una única transacción** y se confirman de una vez al terminar el método. En SQL sería la diferencia entre esto:

```
INSERT INTO palabras VALUES ('abanico'); -- commit
INSERT INTO palabras VALUES ('acariciar'); -- commit
```

y esto:

```
BEGIN TRANSACTION;
INSERT INTO palabras VALUES ('abanico');
INSERT INTO palabras VALUES ('acariciar');
...
COMMIT;
```

El resultado medido en clase: la carga pasó de unos 130 segundos a unos 65. La mitad del tiempo, por una anotación.

⚠ Esa medición se tomó cuando el diccionario de español contenía 646.612 entradas. El diccionario que se distribuye hoy con el proyecto tiene 20.000 palabras, así que el arranque es mucho más rápido y la diferencia, aunque sigue existiendo, es menos espectacular. La lección no cambia: **el coste no estaba en insertar, estaba en confirmar.**

Qué se hace en producción. Un `if` al arranque resuelve la carga inicial, pero no responde a la pregunta importante: ¿qué pasa cuando una versión nueva de la aplicación añade un diccionario o corrige palabras de uno existente? Ese trabajo lo hacen herramientas especializadas en **versionado de esquemas y datos**, que llevan la cuenta de qué cambios se han aplicado ya a cada entorno. Las más usadas son **Liquibase** y **Flyway**.

Ejercicios del capítulo 10

10.1. Escribe la entidad `Sinonimo`, que relaciona una palabra con otra del mismo idioma. Decide qué tipo de relación necesitas, escribe las anotaciones y añade la restricción de integridad que evite que una palabra sea sinónimo de sí misma.

10.2. Declara los métodos de repositorio que resolverían: (a) todas las palabras de un idioma ordenadas alfabéticamente; (b) cuántas palabras tiene cada idioma; (c) las palabras que contienen una subcadena; (d) las palabras sin significados. Indica cuáles se pueden derivar del nombre y cuáles necesitan `@Query`.

10.3. Se quita la anotación `@Transactional` de `CargadorDeDatos`. Además de tardar el doble, ¿qué ocurre si falla la carga a mitad? Compara el estado en que queda la base de datos con y sin la anotación.

10.4. El proyecto pasa de H2 a PostgreSQL en producción. Enumera todo lo que hay que cambiar. ¿Cuánto código Java se ve afectado? Relaciona la respuesta con el patrón DAO del capítulo 4.

10.5. Explica por qué la combinación (`palabra`, `idioma_id`) es única pero `palabra` por sí sola no. Da un ejemplo concreto de dos filas que serían válidas y dos que la base de datos rechazaría.

11. Programación funcional en Java

11.1. Los paradigmas de programación

Un **paradigma** es una forma de usar un lenguaje para escribir código. No son excluyentes: Java admite los cuatro.

Paradigma	En qué consiste
Imperativo	Escribir secuencias de instrucciones que la máquina ejecuta en orden
Procedural	Agrupar esas secuencias bajo un nombre y reutilizarlas: funciones, procedimientos, métodos
Orientado a objetos	Crear tipos de datos propios que extienden los que trae el lenguaje
Funcional	Que una variable pueda apuntar a una función , y poder ejecutarla a través de esa variable

La última es la que interesa aquí. Java incorporó soporte funcional en la **versión 1.8**, con dos operadores nuevos:

Operador	Nombre	Para qué
<code>-></code>	Flecha	Crear funciones anónimas (<i>lambdas</i>)
<code>::</code>	Referencia a método	Apuntar a un método que ya existe

```
// Una función anónima que recibe una palabra y dice si es larga
palabra -> palabra.length() > 10

// Una referencia a un método que ya existe
PalabraPuntuada::getDistancia
```

Junto con esos operadores llegó un paquete nuevo: `java.util.stream`.

11.2. El modelo map-reduce

Un **Stream** es un flujo de datos sobre el que se pueden encadenar operaciones. La idea, tomada del mundo del procesamiento masivo de datos, se llama **map-reduce**:

```
Colección —transformación 1—> Colección 2 —transformación 2—> ...
                                     ...-> REDUCCIÓN —> Resultado
```

Una **transformación** produce otro flujo; una **reducción** convierte el flujo en otra cosa que ya no es un flujo: una lista, un número, un texto.

Las operaciones más usadas:

Operación	Qué hace
<code>filter(predicado)</code>	Deja pasar sólo los elementos que cumplen una condición
<code>map(función)</code>	Transforma cada elemento en otra cosa
<code>sorted(comparador)</code>	Ordena
<code>limit(n)</code>	Se queda con los primeros <code>n</code>
<code>distinct()</code>	Elimina duplicados
<code>collect(...)</code>	Reducción: convierte el flujo en una colección
<code>count()</code>	Reducción: cuenta
<code>anyMatch(predicado)</code>	Reducción: ¿hay alguno que cumpla?

Hay más de cincuenta en total. Un **predicado** es simplemente una función que devuelve un booleano.

⚠ Las transformaciones son **perezosas**: no se ejecuta nada hasta que llega una reducción. Un flujo sin operación final no hace absolutamente nada, y es un error frecuente al empezar.

11.3. Un caso completo: las palabras similares

El problema real del proyecto: cuando una palabra no existe, devolver las diez más parecidas del diccionario, ordenadas de más a menos parecida.

PALABRA BUSCADA: manana IDIOMA: español

Respuesta esperada: mañana, manzana, manada, manzano, ...

La distancia de Levenshtein

Antes de programar nada hay que responder a una pregunta: **¿cómo se mide lo diferentes que son dos palabras?**

La respuesta la dio Vladimir Levenshtein en 1965. La **distancia de Levenshtein** entre dos palabras es el número mínimo de operaciones necesarias para convertir una en la otra, donde cada operación puede ser:

- **insertar** un carácter,
- **borrar** un carácter,
- **sustituir** un carácter por otro.

Cuanto menor es la distancia, más parecidas son.

Par de palabras	Distancia	Por qué
<code>manana</code> → <code>manana</code>	0	Son la misma palabra
<code>manana</code> → <code>mañana</code>	1	Sustituir la <code>n</code> por <code>ñ</code>

Par de palabras	Distancia	Por qué
manana → manzana	1	Insertar una <code>z</code>
manana → manzano	2	Insertar una <code>z</code> y sustituir la <code>a</code> final por <code>o</code>

El algoritmo, paso a paso

Con esa medida, el procedimiento es directo: calcular la distancia entre la palabra buscada y todas las del diccionario, ordenar de menor a mayor y quedarse con las diez primeras.

El problema es el coste. El diccionario grande de español tiene **20.000 palabras**, y calcular una distancia de Levenshtein no es una operación barata. Ordenar 20.000 elementos, tampoco.

Aquí entra una optimización que se razonó en clase y que es un buen ejemplo de cómo pensar antes de programar:

La distancia entre dos palabras es **como mínimo** la diferencia entre sus longitudes. Para pasar de `Mar` a `Margarita` hay que añadir seis caracteres, así que su distancia es al menos 6.

De ahí se deduce que si dos palabras difieren mucho en longitud, no hace falta ni calcular la distancia: se descartan directamente. Y una palabra descartada no sólo ahorra el cálculo, **también ahorra ordenarla después**.

La secuencia completa:

```

COLECCIÓN 1  todas las palabras del diccionario (20.000)
|
├─ map      quedarme sólo con el texto de cada palabra
|
├─ filter   descartar las de longitud muy distinta ← el gran ahorro
|
├─ map      convertir cada palabra en (palabra + distancia)
|
├─ filter   descartar las de distancia demasiado grande
|
├─ sorted   ordenar por distancia, de menor a mayor
|
├─ map      quedarme sólo con la palabra, tirar la distancia
|
├─ limit    las 10 primeras
|
└─ collect  convertirlo en una lista

```

El código

Y ahora, la traducción literal de ese diagrama a Java:

```

public List<String> palabrasSimilares(String palabraObjetivo) {

    List<Palabra> palabrasDelDiccionario = palabraRepository.findByIdioma_Codigo(idioma);

    return palabrasDelDiccionario.stream()
        .map(    palabra -> palabra.getPalabra())
        .filter(palabra -> Math.abs(palabra.length() - palabraObjetivo.length())
                    <= DISTANCIA_MAXIMA_ADMISIBLE)
        .map(    palabra -> new PalabraPuntuada(palabra,
                    DistanciaLevensthein.distance(palabraObjetivo, palabra)))
        .filter(puntuada -> puntuada.getDistancia() <= DISTANCIA_MAXIMA_ADMISIBLE)
        .sorted(Comparator.comparingInt(PalabraPuntuada::getDistancia))
        .map(    puntuada -> puntuada.getPalabra())
        .limit(10)
        .collect(Collectors.toList());
}

```

`PalabraPuntuada` es una clase auxiliar mínima que sólo empareja una palabra con su distancia, para poder ordenar por ella y descartarla al final.

Esto se podría hacer con programación imperativa, pero serían páginas de código, más difíciles de leer y de mantener. Y además se ejecutaría más despacio, porque el flujo puede paralelizarse.

El último punto merece una nota: cambiar `.stream()` por `.parallelStream()` reparte el trabajo entre los núcleos disponibles sin tocar una línea más. No siempre compensa —hay un coste de coordinación— pero para un conjunto de decenas de miles de elementos es exactamente el caso en que sí.

La constante mágica

Una decisión de diseño pequeña pero significativa:

```

public static final int DISTANCIA_MAXIMA_ADMISIBLE = 2;

```

Dar nombre al número evita el llamado *número mágico*: un `2` suelto en medio del código no dice nada, mientras que la constante explica por qué está ahí y permite cambiarlo en un solo sitio.

⚠ En el código actual esa misma constante se usa para dos cosas distintas: el filtro de longitud y el filtro de distancia. Funciona, pero son dos conceptos diferentes que podrían querer valores distintos. Es un buen candidato a separarse en dos constantes.

Ejercicios del capítulo 11

11.1. Reescribe el método `palabrasSimilares()` con programación imperativa: bucles, `if` y una lista auxiliar. Compara el número de líneas y decide cuál te parece más fácil de leer dentro de seis meses.

11.2. Escribe, usando *streams*, el código que devuelva: (a) cuántas palabras del diccionario empiezan por "a"; (b) la palabra más larga; (c) la lista de todos los significados de todas las palabras, sin duplicados. Pista para la última: busca la operación `flatMap`.

11.3. Calcula a mano la distancia de Levenshtein entre `casa` y `caso`, entre `casa` y `causa`, y entre `melón` y `manzana`. Comprueba con el filtro de longitud del código cuáles de esas comparaciones se llegarían a calcular.

11.4. El filtro de longitud descarta palabras antes de calcular la distancia. Estima cuántas de las 20.000 palabras sobreviven al filtro para una palabra de seis letras, suponiendo que la longitud media es de ocho y que la mayoría está entre cinco y once. ¿Qué porcentaje del trabajo se ahorra?

11.5. `DISTANCIA_MAXIMA_ADMISIBLE` vale 2 y se usa en dos filtros distintos. Sepárala en dos constantes con nombres adecuados y razona qué valor le darías a cada una y por qué.

12. Inteligencia artificial aplicada al desarrollo

Este capítulo recoge la sesión monográfica del curso. Es el contenido que más deprisa envejece de todo el manual, así que se ha escrito procurando separar lo que es estructural —cómo funcionan estas herramientas y qué implica usarlas— de lo que es coyuntural: nombres de productos, versiones y precios.

12.1. Modelo y herramienta no son lo mismo

Es la confusión más extendida y conviene deshacerla primero.

ChatGPT no es una inteligencia artificial. Es un **chat**: una herramienta de conversación que usa una inteligencia artificial para responder.

A esas inteligencias artificiales las llamamos **modelos**. Lo mismo ocurre con Copilot, Gemini o Claude: son herramientas, y por debajo hay un modelo haciendo el trabajo de procesar el lenguaje y generar la respuesta.

La prueba está en que muchas de esas herramientas **permiten elegir el modelo**. Las familias principales hoy son GPT (OpenAI), Gemini (Google), Claude (Anthropic) y Llama (Meta), cada una con varias versiones y tamaños: los modelos grandes razonan mejor y cuestan más; los pequeños son rápidos y baratos.

⚠ Los nombres y números de versión concretos cambian cada pocos meses. Cualquier lista de modelos escrita hoy estará desactualizada en un año. Lo que no cambia es la distinción entre herramienta y modelo, y el hecho de que elegir el modelo adecuado para cada tarea es parte del oficio.

12.2. Del prompt al contexto

La forma inicial de trabajar, que sigue siendo válida para muchas cosas:

HUMANO —pregunta→ Chat → Modelo —respuesta de texto→ HUMANO

Se empezó usando estas herramientas como una alternativa al buscador. Y para eso funcionan bien.

Pero en desarrollo de software la forma de trabajar ha cambiado:

```
HUMANO —orden→ Herramienta integrada en el entorno de desarrollo
|
|→ Modelo → respuesta de texto → HUMANO
|
|→ Ejecuta programas:
    · escribe código directamente en el proyecto
    · ejecuta las pruebas
    · arranca la aplicación y comprueba el resultado
    · lee ficheros, busca en el repositorio
```

Eso es lo que llamamos un **agente**. Por debajo sigue habiendo un modelo, pero hay una diferencia decisiva en **cómo le llega la información**:

Cuando sólo se hace una pregunta a un chat, toda la información viaja en el **prompt**. Cuando se trabaja sobre un proyecto de software con un agente, el prompt vale muy poco: la mayor parte de la información llega por el **contexto**.

El contexto es el código del proyecto, la estructura de carpetas, los ficheros de configuración, el historial del repositorio, la salida de los comandos que el agente ha ejecutado. Un agente que ha leído treinta ficheros del proyecto antes de proponer un cambio toma decisiones muy distintas de un chat al que se le pega una función suelta.

La consecuencia práctica es contraintuitiva: dedicar mucho esfuerzo a redactar el prompt perfecto rinde poco. Rinde mucho más asegurarse de que el agente tiene acceso al contexto correcto —el proyecto ordenado, las convenciones escritas, la documentación al día—.

Y de ahí una idea que cierra el círculo con el resto del curso: **un proyecto bien estructurado no sólo es más fácil de mantener para las personas, también es más fácil de entender para un agente**. La modularidad, los nombres claros y las interfaces explícitas rinden dos veces.

12.3. Las herramientas y su coste

GitHub Copilot viene integrado en Visual Studio Code. Requiere una cuenta de GitHub, que es gratuita. Sin pagar, se accede a modelos básicos y a un número reducido de peticiones al mes. Las suscripciones de pago dan acceso a modelos más capaces y a más volumen de trabajo, con distintos escalones de precio. Superado el límite, o se espera a que se restablezca o se paga por ampliarlo. Lo fabrica Microsoft.

Claude, de Anthropic, es la alternativa principal. Anthropic desarrolla además sus propios modelos, considerados de los más capaces disponibles. Teniendo Copilot se pueden usar modelos de Anthropic dentro de Visual Studio Code sin contratar Anthropic aparte; lo que cambia es la herramienta que los envuelve.

La diferencia práctica entre ambas, tal y como se planteó en clase:

Con las dos se pueden hacer muchas cosas. Claude es **más autónoma**: se le pueden encargar trabajos de mayor envergadura y se hace cargo de ellos.

⚠ **Sobre los precios.** Cambian constantemente y no tiene sentido fijarlos en un manual. Lo que sí conviene retener es el orden de magnitud y quién lo paga: para un particular estas herramientas son **caras**; para una empresa, comparadas con el coste por hora de un desarrollador, salen **baratas**. Una suscripción que a una persona le parece un gasto considerable se amortiza en la empresa con unas pocas horas ahorradas al mes.

12.4. Cómo se usó la IA en este curso

La sesión monográfica no fue teórica: se construyeron dos interfaces de usuario completas delante de los alumnos, y el contraste entre las dos es la parte interesante.

La aplicación web (Angular). Se generó con Copilot **sin darle apenas indicaciones**. El resultado fue notablemente bueno: separó el DTO del modelo de dominio, definió un contrato abstracto para el suministrador de diccionarios y configuró la inyección de dependencias de Angular para elegir la implementación. Es decir, **aplicó por su cuenta los mismos principios de diseño que se habían enseñado en el curso**.

La aplicación de escritorio (JavaFX). Se construyó con Claude en dos actos deliberados:

- El **Acto 1** implementó al pie de la letra el contrato `InterfazDeUsuario` que ya existía. Sirvió para comprobar que el diseño de la primera sesión funcionaba: cambiar la interfaz de usuario de todo el sistema costó **una línea**. Pero el resultado era una aplicación de escritorio inservible, que sólo permitía una búsqueda.
- El **Acto 2** diagnosticó por qué, y construyó la aplicación de verdad consumiendo directamente la fachada de negocio.

El hallazgo. Angular, sin que nadie se lo indicara, había tomado exactamente la misma decisión que el Acto 2: **no implementar** `InterfazDeUsuario`, sino consumir `SuministradorDeDiccionarios`. Dos herramientas distintas, dos lenguajes distintos, la misma conclusión sobre el diseño.

De las dos abstracciones que se diseñaron el primer día, la que ha sobrevivido a tres clientes distintos —consola, web y escritorio— es la de **negocio**, no la de interfaz de usuario.

Ése es un resultado que no estaba planificado y que salió de leer el código. Es también un buen ejemplo de para qué sirven realmente estas herramientas: no sólo escriben, también permiten **probar una hipótesis de diseño en una tarde** en lugar de en una semana.

12.5. Qué conviene no delegar

El curso entero descansa sobre una idea que conviene enunciar sin ambigüedad:

El trabajo hoy no es escribir código: eso lo escriben en buena medida los agentes. El trabajo es **entender qué componentes necesita el sistema, definir sus especificaciones y decidir cómo encajan**. Después se le pide a la IA que escriba cada pieza, y se prueba y se integra.

De ahí se derivan cuatro cosas que siguen siendo responsabilidad de la persona:

El diseño. Decidir qué abstracciones existen y dónde están las fronteras. Un agente rellena bien un hueco bien definido; no decide por nosotros qué huecos debe haber. En este proyecto, la razón de que la IA produjera código que encajaba es que **las abstracciones ya estaban puestas**.

La verificación. El código generado hay que probarlo y leerlo. Un modelo produce con la misma confianza código correcto y código plausible pero equivocado, y la diferencia sólo se ve ejecutándolo. Todo lo del capítulo 5 —las pruebas automatizadas— vale más, no menos, cuando parte del código no lo has escrito tú.

El criterio sobre el contexto. Saber qué información necesita el agente y qué le sobra. Y saber qué **no** debe salir del entorno: código propietario, datos de clientes, credenciales.

Entender lo que se acepta. Un agente resuelve en segundos un error críptico. Si uno se queda con la solución y no con el porqué, la próxima vez que el error cambie un poco vuelve a estar perdido. Ocurrió en clase con el error de JavaFX sobre componentes del *runtime* que faltaban: el mensaje era falso, y entender la causa —la frontera entre el *classpath* y el sistema de módulos de Java 9— vale más que la línea que lo arregla.

⚠ Y una advertencia sobre el riesgo profesional: la parte del trabajo que estas herramientas automatizan bien es precisamente la que tradicionalmente se aprendía haciendo. Quien delega desde el principio la escritura de código sin haber pasado por la fase de escribirlo corre el riesgo de saber pedir sin saber juzgar. Y juzgar es lo único que no se puede delegar.

Ejercicios del capítulo 12

12.1. Toma una clase del proyecto que no tenga pruebas —por ejemplo `NormalizadorDeTerminos` o `DiccionarioEnServicioWeb`— y pide a un agente que las escriba. Revisalas después: ¿cubren los casos límite? ¿Hay alguna que pase siempre, verifique lo que verifique?

12.2. Formula la misma petición de dos maneras: primero pegando una clase suelta en un chat, después pidiéndosela a un agente con acceso al proyecto completo. Compara los resultados y explica la diferencia en términos de contexto.

12.3. Un compañero propone subir todo el repositorio a una herramienta de IA externa para que "lo entienda mejor". Enumera qué preguntarías antes de aceptar y en qué casos te negarías.

12.4. La aplicación Angular decidió por su cuenta no implementar `InterfazDeUsuario`. Escribe en un párrafo por qué esa decisión era la correcta y qué te dice sobre la calidad del diseño original del proyecto.

12.5. Busca en el código generado durante el curso —la aplicación web o la de escritorio— una decisión que **no** compartas, y argumenta qué habrías hecho distinto. El objetivo del ejercicio no es encontrar un error, es practicar el juicio.

Parte II · El proyecto, paso a paso

Esta segunda parte no introduce conceptos nuevos: recorre cómo se aplicaron. Cada capítulo cubre una versión del sistema y responde siempre a las mismas cuatro preguntas: **qué problema había, qué se decidió, qué se construyó y cuánto costó.**

La última pregunta es la importante. Todo el curso sostiene que un buen diseño abarata los cambios futuros; en estas páginas esa afirmación se puede comprobar con números.

13. Versión 1 — Aplicación monolítica de consola

13.1. El encargo

Un programa ejecutable desde una terminal que busque una palabra en un diccionario:

```
c:\> buscarPalabra "melón" "es"
Aplicación de Diccionarios v1.1.0
La palabra melón existe en el diccionario de español, y significa:
- Fruto del melonero
- Persona con pocas luces: "Eres un melón!"
Gracias por usar nuestra aplicación de diccionarios.
```

Con tres casos más que atender: palabra que no existe, idioma que no se tiene, y parámetros insuficientes.

La solución más simple sería un único fichero con dos diccionarios dentro. Y se descartó desde el principio:

No buscamos la solución más simple. Buscamos la solución más mantenible, teniendo en cuenta que el programa va a evolucionar.

Que iba a evolucionar no era una suposición. Se sabía que los diccionarios pasarían de ficheros a una base de datos, que la interfaz de terminal acabaría siendo web o móvil, y que llegarían idiomas nuevos.

13.2. Identificar los componentes

El primer trabajo no fue escribir código, sino decidir **de qué piezas se compone el sistema y cuál es la responsabilidad de cada una**:

Componente	Responsabilidad
Diccionario	Saber si una palabra existe y devolver sus significados
SuministradorDeDiccionarios	Saber qué diccionarios hay y suministrarlos
InterfazDeUsuario	Interactuar con el usuario: pedir datos y mostrar resultados
Aplicacion	Coordinar a los demás y ejecutar el algoritmo

Los tres primeros se definieron como **interfaces**, por la razón del capítulo 3: un diccionario puede estar en un fichero, en una base de datos o al otro lado de una red, y quien lo usa no debería enterarse.

13.3. El algoritmo

Con los componentes definidos, el flujo de la aplicación se escribió en lenguaje natural antes de programarlo:

Al arrancar:

- Pedir a la InterfazDeUsuario que muestre el mensaje de bienvenida
- Preguntarle qué palabra quiere buscar el usuario
 - Si no hay palabra → mostrar mensaje de ayuda
 - Si hay palabra:
 - Preguntarle qué idioma quiere
 - Si no hay idioma → mostrar mensaje de ayuda
 - Si hay idioma:
 - Preguntar al Suministrador si tiene diccionario de ese idioma
 - Si no lo tiene → avisar de idioma desconocido
 - Si lo tiene:
 - Pedirle el diccionario
 - Preguntar al Diccionario si la palabra existe
 - Si no existe → avisar de palabra desconocida
 - Si existe → pedir los significados y mostrarlos
 - Pedir que muestre el mensaje de despedida

Ese pseudocódigo se tradujo casi línea a línea a `Aplicacion.java`, y ha sobrevivido sin cambios hasta hoy: es el mismo algoritmo en la versión 5.

13.4. El formato de los datos

Los diccionarios se guardaron como ficheros de texto, una línea por palabra, con los significados separados por barras verticales:

```
abanico=Herramienta para mover el aire.|Conjunto de opciones entre las que elegir.  
melón=Fruto del melonero|Persona con pocas luces: "Eres un melón!"
```

Una decisión pequeña con una justificación importante: **ese formato lo puede mantener un lingüista**, que es quien realmente va a escribir el contenido, sin saber programar.

13.5. Lo que quedó bien y lo que quedó mal

Bien: las abstracciones. `Diccionario` y `SuministradorDeDiccionarios` se definieron aquí y siguen intactas cinco versiones después. Eso es lo que ha permitido todo lo demás.

Mal: todo lo estructural. El resultado fueron **diez ficheros** `.java` **sueltos en una carpeta**, sin paquetes ni módulos, con los diccionarios mezclados con el código y una aplicación que se instalaba entera en el ordenador de cada usuario.

Diez ficheros no parecen muchos, y no lo son. Pero esto es una aplicación de ejemplo. En un sistema real se acaba con cientos, y sin estructura es imposible saber qué hace cada cosa y qué se rompe al tocarla.

Esa primera versión se conserva sin tocar en la carpeta `diccionarios/` del repositorio, como punto de comparación.

14. Versión 2 — Modularización y arquitectura cliente-servidor

14.1. El diagnóstico

La aplicación funcionaba. El problema era todo lo demás, y se hizo explícito con las dos herramientas del capítulo 6.

Escenarios de cambio previsibles: cambiar la interfaz, añadir la sugerencia de palabras parecidas, añadir un idioma, corregir palabras y cambiar el algoritmo de búsqueda.

Análisis por implicado del escenario más frecuente —corregir la definición de una palabra—:

Implicado	Consecuencia con la V1
Desarrollo	Reempaquetar y redistribuir la aplicación entera
Usuario	Reinstalar. Y el usuario sabe de su negocio, no de instalar software
Operaciones	Publicar y avisar, sin saber qué versión tiene cada uno
Soporte	No puede reproducir una incidencia sin conocer la versión del cliente
Negocio	Dos usuarios pueden obtener respuestas distintas a la misma consulta

La última fila cerró la discusión.

14.2. Primera decisión: modularizar

Antes de cambiar la arquitectura se reorganizó el proyecto **sin tocar funcionalidad**. A eso se le llama **refactorizar**: cambiar la estructura del código conservando exactamente su comportamiento.

De diez ficheros sueltos se pasó a un proyecto Maven multimódulo:

Módulo	Contenido
<code>diccionarios-api</code>	Los contratos: <code>Diccionario</code> , <code>SuministradorDeDiccionarios</code>
<code>ui-api</code>	El contrato <code>InterfazDeUsuario</code>
<code>diccionarios-en-ficheros</code>	La implementación que lee ficheros
<code>ui-consola</code>	La implementación de terminal
<code>diccionario-es</code> , <code>-en</code> , <code>-elfico</code>	Los datos, un módulo por idioma
<code>aplicacion-completa</code>	El <code>main</code> , las factorías y la orquestación

Cada idioma en su propio módulo puede parecer excesivo, y tiene una razón concreta: **cada diccionario tiene su propio ciclo de vida y su propia versión**. Añadir palabras al diccionario élfico es un cambio suyo, no del sistema. Por eso hoy va por la `1.1.0` mientras el resto sigue en la `1.0.0`.

14.3. Segunda decisión: cliente-servidor

Con el proyecto ordenado, el cambio de arquitectura: mantener un cliente que sólo sea interfaz, y llevar los diccionarios y la lógica de búsqueda a un servidor central.

ANTES

Ordenador del usuario

Aplicación → Java → SuministradorDeDiccionariosEnFicheros → ficheros

AHORA

Ordenador del usuario

Servidor central

Aplicación → ...EnServicioWeb → HTTP → Controlador REST → ...EnFicheros → ficheros

Lo que se gana es exactamente lo que costaba caro en la tabla anterior: corregir una palabra pasa a ser un despliegue en un único sitio, del que se benefician todos los usuarios a la vez y sin reinstalar nada. Y desaparece la posibilidad de que dos usuarios obtengan respuestas distintas.

Lo que **no** se resuelve: un cambio en la interfaz de usuario sigue obligando a distribuir una versión nueva del cliente. Para eso habrá que esperar a la V5.

14.4. El contrato antes que el código

Antes de programar el servidor se definió el API REST —las rutas, los códigos de estado y los cuerpos de respuesta—, tal y como se explica en el capítulo 7. Ese documento es lo que permite que el equipo del servidor y el del cliente trabajen en paralelo.

14.5. Lo que se construyó

El servidor, con Spring Boot, resultó ser sorprendentemente poco código: una clase de arranque con un `main` de una línea, un controlador REST con dos métodos, un DTO y una clase de configuración. Todo lo demás —el servidor Tomcat, el enrutamiento, la conversión a JSON— lo aporta el framework.

El cliente nuevo: un módulo `diccionarios-en-servicio-web` con tres clases que implementan las interfaces de siempre pero hablando HTTP por dentro.

Y aquí está el resultado que importa:

Del cliente antiguo se reutilizó **todo**: la interfaz de usuario, la lógica de la aplicación, los contratos y los diccionarios. Lo único que se modificó fue **una línea** en `SuministradorDeDiccionariosFactory`.

```
//return new SuministradorDeDiccionariosEnFicheros("diccionarios");  
return new SuministradorDeDiccionariosEnServicioWeb("http://localhost:8080");
```

14.6. El balance de la V2

Concepto	Cantidad
Módulos nuevos	2 (<code>servicio-web</code> , <code>diccionarios-en-servicio-web</code>)
Módulos reutilizados sin tocar	7
Líneas modificadas en código existente	1

¿Qué probabilidad hay de haber roto algo de lo que ya funcionaba? Ninguna. No se ha tocado nada: sólo se han añadido proyectos nuevos.

Y un detalle que suele pasar desapercibido: **la aplicación monolítica anterior seguiría funcionando** si se quisiera, reutilizando esos mismos componentes. Las dos arquitecturas conviven sobre la misma base.

El coste, dicho con honestidad. El desarrollo fue más complejo y llevó más tiempo. Tampoco mucho más —son cuatro ficheros con poco código— pero más. La apuesta es que ese sobrecoste se recupera durante los años de mantenimiento, y las dos versiones siguientes permiten comprobar si la apuesta era buena.

15. Versión 3 — Persistencia en base de datos

15.1. Por qué cambiar

Los diccionarios en ficheros de texto funcionaban, pero tenían un techo evidente: no hay forma razonable de consultar, actualizar o indexar cientos de miles de palabras en un fichero plano, ni de que varias personas trabajen sobre él a la vez.

El cambio afecta **sólo al servidor**. El cliente no se entera, porque nunca supo de dónde salían los datos.

15.2. Diseñar antes de programar

Primero el modelo entidad-relación, con sus tres tablas y sus restricciones de integridad (capítulo 10).

Después, la comprobación de qué trabajo desaparece: el script `CREATE TABLE` y los `INSERT` que antiguamente habría que escribir a mano los genera Hibernate a partir de las anotaciones.

15.3. Lo que se construyó

Un único módulo nuevo, `diccionarios-en-bbdd`, con cuatro tipos de pieza:

Pieza	Ficheros	Función
Entidades	<code>Idioma</code> , <code>Palabra</code> , <code>Significado</code>	El mapeo objeto-relacional
Repositorios	<code>IdiomaRepository</code> , <code>PalabraRepository</code> , <code>SignificadoRepository</code>	El acceso a datos. Sólo interfaces
Implementación del API	<code>DiccionarioEnBBDD</code> , <code>SuministradorDeDiccionariosEnBBDD</code>	Los contratos de siempre, hablando con la base de datos
Carga inicial	<code>CargadorDeDatos</code>	Vuelca los ficheros en la base de datos al arrancar

Obsérvese que las dos clases de la tercera fila implementan **exactamente las mismas interfaces** que las versiones de ficheros y de servicio web. Ésa es la tercera implementación intercambiable de la misma abstracción.

15.4. El cambio en el sistema existente

Sustituir un origen de datos por otro consistió en cambiar **una dependencia** en el `pom.xml` del servicio web:

```
<dependency>
  <groupId>com.curso</groupId>
  <!--<artifactId>diccionarios-en-ficheros</artifactId-->
  <artifactId>diccionarios-en-bbdd</artifactId>
  <version>1.0.0</version>
</dependency>
```

Y añadir tres anotaciones a la clase de arranque, porque los componentes nuevos viven en un paquete que Spring no exploraba (capítulo 9):

```
@SpringBootApplication(scanBasePackages = {"com.curso.diccionarios"})
@EnableJpaRepositories(basePackages = {"com.curso.diccionarios.bbdd.repositorios"})
@EntityScan(basePackages = {"com.curso.diccionarios.bbdd.entidades"})
```

Nada más. Ni el controlador REST, ni el cliente, ni la interfaz de usuario, ni los contratos se tocaron.

15.5. La lección de rendimiento

Al cargar el diccionario grande —que en aquel momento contenía 646.612 entradas— la carga inicial tardaba unos **130 segundos**. El diagnóstico: cada `save()` provocaba su propia confirmación de escritura en la base de datos.

Una anotación, `@Transactional`, agrupó todas las inserciones en una sola transacción y **el tiempo bajó a unos 65 segundos**. La mitad.

⚠ El diccionario que se distribuye hoy con el proyecto tiene 20.000 palabras —seleccionadas por frecuencia de uso y con las definiciones tomadas del Wikcionario, según se documenta en el `NOTICE.md` que acompaña a los ficheros—, así que el arranque actual es mucho más rápido que el que se midió en clase.

Merece la pena retener el orden de las cosas: primero se midió, después se diagnosticó y sólo entonces se cambió el código. Optimizar sin medir es adivinar.

16. Versión 4 — Nuevas funcionalidades y gestión de errores

Esta versión no cambia la arquitectura: añade funcionalidad. Y es la que más decisiones de diseño interesantes contiene, porque en tres de los cuatro casos había más de un sitio donde poner el cambio.

16.1. Listar los idiomas disponibles

Qué se pedía. Un endpoint que devolviera los códigos de los idiomas del servidor:

```
GET /diccionarios → ["EN","ELFICO","ES"]
```

El problema. Añadir `dameIdiomas()` a la interfaz `SuministradorDeDiccionarios` dejaba sin compilar a los tres módulos que ya la implementaban.

La decisión. Declararlo como método `default` que lanza una excepción, de forma que sólo lo implemente quien lo necesite. Es el principio Abierto/Cerrado del capítulo 3, y su rentabilidad se cobró dos sesiones más tarde.

16.2. Búsquedas sin distinguir mayúsculas

Qué se pedía. Que `melón`, `Melón` y `MELÓN` den el mismo resultado.

Dónde ponerlo. Aquí hubo una discusión de diseño real, con dos alternativas.

Alternativa A: en el repositorio. Se le puede pedir a Spring que genere la consulta ignorando mayúsculas, ya sea cambiando el nombre del método o escribiendo la consulta a mano:

```
Optional<Palabra> findByPalabraIgnoreCaseAndIdioma_CodigoIgnoreCase(String palabra, String codigo);
```

Funciona y es poco código. **Y se descartó**, por esta razón:

Esa funcionalidad debe ser independiente de dónde se almacenen los diccionarios. Si mañana cambio la base de datos, o vuelvo a ficheros, tendré que reimplementar el comportamiento en el nuevo sitio. Y eso no es bueno.

Alternativa B: normalizar en la frontera. Da igual cómo vengan las palabras: se guardan siempre en mayúsculas, y se convierte a mayúsculas cualquier término que llegue del usuario antes de buscarlo.

```
public class NormalizadorDeTerminos {  
    public static String normalizar(String termino) {  
        if (termino == null) return null;  
        return termino.trim().toUpperCase();  
    }  
}
```

Se eligió la B. La regla de decisión es generalizable: **una regla de negocio no debe vivir en la capa de infraestructura**, porque la infraestructura es precisamente lo que está pensado para poder cambiar.

⚠ Toda decisión tiene efectos secundarios, y ésta también: las palabras se almacenan en mayúsculas, así que las sugerencias que devuelve el servidor llegan como `MAÑANA`. Las interfaces gráficas lo corrigen al presentarlas. Es una consecuencia asumida, no un descuido, pero conviene conocerla.

16.3. Devolver un error 500 cuando algo falla

Qué se pedía. Que si el servidor no puede atender una petición —por ejemplo, porque la base de datos no responde— conteste con un `500` y no con un error opaco.

Dónde ponerlo. Escribirlo a mano significaría envolver cada método del controlador en un `try/catch`, repitiendo el mismo bloque cuatro veces y ensuciando el controlador con código que no tiene que ver con su trabajo.

La decisión. Aplicar programación orientada a aspectos con `@RestControllerAdvice` (capítulo 4). Spring construye el proxy y el controlador **no se modifica**: en él sólo vive el *happy path*.

16.4. Sugerir palabras parecidas

Qué se pedía. Que si la palabra no existe, el servidor devuelva las diez más parecidas.

Es la funcionalidad más ambiciosa de la versión y está desarrollada por completo en el capítulo 11: la distancia de Levenshtein como medida, el filtro por longitud como optimización previa y la programación funcional con *streams* como forma de expresarlo.

Con esta funcionalidad el sistema deja de ser sólo un diccionario y se convierte en la base de un corrector ortográfico.

17. Versión 5 — Tres interfaces de usuario

La última versión no toca el servidor en absoluto. Añade dos formas nuevas de usar el sistema y, de paso, somete el diseño original a su prueba más dura.

17.1. La interfaz web (Angular)

Una aplicación Angular independiente, que vive fuera del proyecto Maven y se compila con sus propias herramientas. Se comunica con el servidor por HTTP, y en desarrollo un *proxy* redirige las peticiones para evitar los problemas de origen cruzado.

Su estructura interna reproduce, en TypeScript, exactamente las mismas ideas del proyecto Java:

Pieza en Angular	Equivalente en el proyecto Java
<code>SuministradorDeDiccionarios</code> (clase abstracta)	El módulo <code>diccionarios-api</code>
<code>SuministradorDeDiccionariosHttp</code>	<code>diccionarios-en-servicio-web</code>
<code>RespuestaPalabraDto</code>	El DTO <code>RespuestaPalabra</code>
<code>ResultadoBusqueda</code>	El modelo de dominio
La configuración de proveedores	La factoría

Lo relevante es que **esa correspondencia no se pidió**: la aplicación se generó con Copilot sin apenas indicaciones y llegó a ella por su cuenta.

17.2. La interfaz de escritorio (JavaFX), en dos actos

Se construyó deliberadamente en dos pasos para responder a una pregunta pendiente desde la primera sesión.

Acto 1: cumplir la promesa. El diseño del primer día prometía que `InterfazDeUsuario` admitiría una implementación de escritorio. Se escribió esa implementación al pie de la letra y se cambió una línea en la factoría. Funcionó: el sistema entero pasó a tener interfaz gráfica sin tocar `Aplicacion`, ni los contratos, ni el servidor.

Y el resultado era inservible. La aplicación abría una ventana, permitía una búsqueda y se quedaba muerta. Las causas, todas del contrato y ninguna de JavaFX:

- `recuperarLaPalabraSolicitadaPorElUsuario()` es una pregunta que se hace **una vez**: está pensada para leer argumentos de línea de comandos. Implementarla con una ventana obliga a **congelar el programa** hasta que el usuario pulse un botón.
- `Aplicacion.main()` es un guion que se ejecuta de arriba abajo y termina. En una terminal está bien; en escritorio el usuario espera seguir usando la ventana.
- El contrato **no tiene forma de preguntar qué idiomas hay**, porque no conoce la capa de negocio. Por eso el idioma había que escribirlo a mano en lugar de elegirlo en un desplegable.

- Con los campos vacíos, `Aplicacion` llama a `mostrarMensajeAyuda()`, que explica cómo se usan los argumentos de línea de comandos... en una ventana gráfica.

El contrato `InterfazDeUsuario` no describe "una interfaz de usuario". Describe "una interfaz de usuario **de consola, gobernada por un guion**". Es una abstracción con la forma de su primera implementación.

Acto 2: invertir el control. En escritorio manda el usuario: busca, mira, pulsa una sugerencia, cambia de idioma, vuelve a buscar. El flujo se invierte, que es exactamente la Inversión de Control del capítulo 9 vista desde el otro lado: ya no hay un guion propio llamando a la interfaz, hay un framework que nos llama cuando el usuario hace algo.

La aplicación definitiva **no implementa** `InterfazDeUsuario`: consume directamente `SuministradorDeDiccionarios`, con selector de idioma, sugerencias pulsables y trabajo en segundo plano para que la ventana no se congele mientras el servidor calcula.

17.3. El hallazgo

Las dos interfaces nuevas se construyeron por separado, con herramientas distintas y en lenguajes distintos. Y las dos tomaron la misma decisión: **ignorar el contrato de interfaz de usuario y consumir el de negocio**.

De las dos abstracciones diseñadas el primer día, la que ha sobrevivido a tres clientes es la de **negocio**, no la de interfaz de usuario.

Ése es el resultado más valioso de todo el proyecto, y merece la pena entender por qué no es un fracaso del diseño original. `InterfazDeUsuario` cumplió su función durante cuatro versiones y permitió sustituir la implementación de consola por una gráfica en una línea. Lo que ocurre es que **una abstracción extraída de un solo caso de uso tiende a heredar la forma de ese caso**. Sólo el segundo caso revela cuáles de sus supuestos eran esenciales y cuáles accidentales.

18. Balance de la evolución

18.1. Lo que costó cada salto

Salto	Módulos nuevos	Cambios en lo existente
V1 → V2 (cliente-servidor)	2	1 línea en una factoría
V2 → V3 (base de datos)	1	1 dependencia en un <code>pom.xml</code> + 3 anotaciones
V3 → V4 (funcionalidades)	0	Métodos añadidos; ninguna firma existente modificada
V4 → V5 (escritorio)	1	1 línea en una factoría
V4 → V5 (web)	proyecto aparte	ninguno

En el salto a la interfaz de escritorio se midió el cambio con precisión sobre el control de versiones: **121 líneas añadidas y 2 borradas** en todo el sistema. Y una de las dos era una línea en blanco.

18.2. Lo que se reutilizó

De la V1 a la V5 se reutilizaron **sin modificarlos**: `diccionarios-api`, `ui-api`, `ui-consola`, los tres módulos de diccionarios y la lógica de `Aplicacion`.

El algoritmo que se escribió en pseudocódigo en la primera sesión sigue siendo, línea por línea, el que ejecuta el cliente de consola hoy.

Ése es, en una frase, el objetivo de todo lo que se ha visto: **que un cambio grande en el sistema se traduzca en un impacto pequeño en el código existente.**

18.3. Lo que no está bien

Un balance honesto tiene que incluir la deuda. Estas son las limitaciones conocidas del sistema tal y como queda:

ES.GRANDE es un idioma fantasma. El código de idioma se deduce del nombre del fichero, así que `es.grande.txt` se carga como un idioma distinto de `ES` y aparece en los desplegables. Una convención de nombres de fichero se convirtió en contrato de datos sin que nadie lo decidiera.

El cliente de consola hace tres viajes por la red para resolver una búsqueda, porque `tienesDiccionarioDe`, `existe` y `palabrasSimilares` abren cada uno su propia petición. Cuando la palabra no existe, el servidor calcula las distancias de Levenshtein **dos veces**. Contradice la regla número uno del capítulo 7, y las interfaces web y de escritorio lo resuelven con una o dos llamadas.

`palabrasSimilares()` carga el diccionario entero en memoria en cada búsqueda fallida. Con H2 en memoria se aguanta; contra una base de datos real, no.

Faltan pruebas. Sólo el módulo `diccionarios-en-ficheros` tiene pruebas automatizadas. El servidor, el acceso a base de datos, el cliente HTTP y las interfaces de usuario no tienen ninguna. Es la deuda más

grave.

El contrato documentado no coincide del todo con el comportamiento real. Se documentó que un idioma inexistente devolvería `{}` y el servidor devuelve los cuatro campos a `null`. Y el campo `idioma` se devuelve tal y como lo escribió el cliente, no normalizado.

Otros: H2 1.4.200 es de 2019 y arrastra vulnerabilidades conocidas; CORS está sin resolver para producción; la versión que imprime la aplicación (`v1.1.0`) no coincide con la de los `pom.xml` (`1.0.0`); y elegir interfaz de usuario obliga a recompilar, porque la decisión vive en una factoría en lugar de en una propiedad de configuración.

Que esta lista exista y esté escrita es parte del trabajo. La deuda técnica no desaparece por no documentarla: sólo se vuelve invisible, que es peor.

Ejercicios de la Parte II

II.1. Toma el escenario "hay que añadir el idioma alemán" y descríbelo paso a paso en la V1 y en la V5. Cuenta cuántas personas intervienen y cuántos despliegues hacen falta en cada caso.

II.2. Resuelve el problema del idioma fantasma `ES.GRANDE`. Propón al menos dos soluciones —una tocando los ficheros y otra tocando el código— y decide cuál elegirías justificando el impacto de cada una.

II.3. Elimina los tres viajes por la red del cliente de consola. Describe qué métodos cambiarías y qué impacto tendría en la interfaz `Diccionario`. ¿Se puede hacer sin romper a los otros dos clientes?

II.4. Diseña la V6 del sistema: los diccionarios dejan de estar en una base de datos propia y pasan a consultarse contra un servicio externo de terceros que cobra por consulta. Indica qué módulos crearías, cuáles reutilizarías y cuántas líneas tocarías del código existente. Añade una caché y justifica dónde la pondrías.

II.5. Escribe la lista de las cinco tareas que abordarías primero para saldar la deuda técnica del apartado 18.3, ordenadas por relación entre riesgo evitado y esfuerzo. Justifica el orden.

Apéndice

Apéndice A · El temario oficial, punto por punto

Este apéndice relaciona cada punto del programa formativo oficial con el capítulo de este manual donde se estudia y con el fichero del proyecto que lo ilustra. Sirve para dos cosas: comprobar la cobertura del temario y localizar rápidamente el ejemplo de código de cualquier concepto.

Punto del temario	Capítulo	Ejemplo en el proyecto
2 · El modelo de factorías	4.2	<code>aplicacion-completa/.../factorias/SuministradorDeDiccionariosFactory.java</code> <code>aplicacion-completa/.../factorias/InterfazDeUsuarioFactory.java</code>
4.4 · Principios de diseño OO	3	<code>diccionarios-api/.../Diccionario.java</code> <code>diccionarios-api/.../SuministradorDeDiccionarios.java</code>
4.5 · Patrones de creación, estructurales y de comportamiento	4	Factoría, Singleton (<code>@Component</code>), Adapter y Proxy (<code>diccionarios-en-servicio-web</code>), Strategy (las tres implementaciones), Template Method (<code>CargadorDeDatos</code>)
4.6 · Bloques de constitución arquitectónicos	2.6, 14	<code>diccionarios_v2/pom.xml</code> y los doce módulos
5.1 · Patrones para la capa de integración	4.5, 10	<code>diccionarios-en-bbdd/.../repositorios/</code> <code>diccionarios-en-bbdd/.../entidades/</code> <code>CargadorDeDatos.java</code>
5.2 · Patrones presentation-to-business	4.6	DTO: las dos clases <code>RespuestaPalabra</code> Business Delegate: <code>DiccionarioEnServicioWeb</code>
5.3 · Patrones intra-business	4.6, 9.4	Fachada: <code>SuministradorDeDiccionarios</code> Inyección de dependencias: constructores del servidor
5.4 y 5.5 · Capas de micro y macro presentación	4.6, 9.5	Front Controller: el <code>DispatcherServlet</code> de Spring <code>DiccionariosRestController.java</code> <code>ui-api</code> / <code>ui-consola</code>
5.6 · Antipatrones	4.8	La carpeta <code>diccionarios/</code> (V1) como <i>Big Ball of Mud</i>
6.1 · Guías y heurísticas de arquitectura	6.4, 6.5	Análisis por escenarios de cambio y por implicado
6.2 · Proceso de desarrollo del software	2, 5	Maven, ciclo de vida, versionado semántico, JUnit, cobertura
6.3 · Desarrollo local frente a distribuido	7.5	Las tres versiones del sistema como transición
7.3, 7.4, 7.5 · Servidores y tecnología Jakarta EE	9, 10	<code>ServicioWeb.java</code> , Tomcat embebido, JPA e Hibernate

Punto del temario	Capítulo	Ejemplo en el proyecto
7.7 y 7.8 · Diseño software y arquitectura en capas	15, 18	El modelo entidad-relación y la arquitectura final
Tecnologías del temario hoy obsoletas	8	RMI, CORBA/IDL/IIOP, JNDI, SOAP y sus equivalentes

Apéndice B · Glosario de términos

Agente. Herramienta de IA integrada en el entorno de desarrollo que, además de generar texto, ejecuta programas: escribe ficheros, lanza pruebas, arranca la aplicación.

Anotación. Marca que empieza por `@` y se coloca antes de una clase, método o campo para darle un significado especial al compilador o a un framework.

API. Conjunto de operaciones que un componente ofrece a otros. En este proyecto, tanto las interfaces Java como las rutas HTTP del servicio.

AOP (*Aspect-Oriented Programming*). Forma de programar que separa de la lógica de negocio las preocupaciones transversales: errores, seguridad, registro, auditoría.

Bytecode. Lenguaje de bajo nivel al que compila Java. No es lenguaje máquina: lo ejecuta la máquina virtual.

Caché. Almacén temporal de datos costosos de obtener. Un sistema debe poder funcionar sin ella.

Classpath. Lista de rutas donde la máquina virtual busca las clases y los recursos de un programa.

Cobertura de pruebas. Porcentaje de líneas de código que las pruebas automatizadas llegan a ejecutar.

Commit (base de datos). Confirmación de que un conjunto de cambios queda escrito de forma duradera. Es una operación costosa.

Contexto (IA). Toda la información que un agente tiene disponible además de la pregunta: el código, la estructura del proyecto, la salida de los comandos.

CORS. Mecanismo del navegador que restringe las peticiones de una página web a un servidor de origen distinto.

DAO (*Data Access Object*). Patrón que aísla el acceso a datos del resto de la aplicación.

Dependencia. Librería de terceros que un proyecto necesita.

Dependencia transitiva. Dependencia de una dependencia. Maven las resuelve automáticamente.

DI (*Dependency Injection*). El componente declara qué necesita y un contenedor se lo entrega, en lugar de construirlo él.

Dialecto SQL. Variante del lenguaje SQL propia de cada motor de base de datos.

DTO (*Data Transfer Object*). Objeto sin lógica cuyo único cometido es transportar datos a través de una frontera.

Endpoint. Cada una de las rutas que expone un servicio web.

Entidad. Clase Java cuyos objetos se guardan en una tabla de la base de datos.

Framework. Conjunto de librerías que además impone una forma de construir el programa. A diferencia de una librería, es él quien llama a nuestro código.

Garbage collector. Proceso que libera automáticamente la memoria de los objetos a los que ya no apunta ninguna variable.

H2. Base de datos que se ejecuta en memoria y desaparece al parar la aplicación. Se usa para desarrollo y pruebas.

Happy path. El camino que sigue la ejecución cuando todo va bien, sin errores.

Hibernate. Implementación concreta del estándar JPA. Traduce entre objetos Java y tablas relacionales.

HTTP. Protocolo de la Web. Unidireccional y síncrono: el cliente pregunta, el servidor responde.

IoC (*Inversion of Control*). El flujo de la aplicación lo decide el framework, no el programador.

Jakarta EE. Nombre actual de J2EE/JEE. Colección de estándares para aplicaciones empresariales Java. Sus paquetes empiezan por `jakarta.`

JDBC. API estándar de Java para conectarse a bases de datos relacionales. Cada motor aporta su *driver*.

JNDI. API clásica para localizar recursos por nombre en un servidor de aplicaciones. Antecesora conceptual de la inyección de dependencias.

JPA (*Java Persistence API*). El **estándar** de persistencia de Jakarta EE. Define las anotaciones y el comportamiento esperado.

JSON. Formato ligero de intercambio de datos. Sustituyó a XML.

JUnit. Librería estándar de pruebas automatizadas en Java. La versión actual es la 5, también llamada *Jupiter*.

JVM (*Java Virtual Machine*). Programa que ejecuta el bytecode. Es específica de cada sistema operativo y procesador.

Lambda. Función anónima. En Java se escribe con el operador `->`.

LCC (*Life Cycle Cost*). Coste total de un producto a lo largo de toda su vida, no sólo el de su construcción.

Levenshtein (distancia de). Número mínimo de inserciones, borrados o sustituciones de caracteres para convertir una palabra en otra.

LTS (*Long Term Support*). Versión de Java con mantenimiento prolongado. Son las que se usan en producción: 8, 11, 17, 21, 25.

Map-reduce. Modelo de procesamiento que transforma una colección mediante operaciones encadenadas y termina reduciéndola a un resultado.

Maven Central. Repositorio público desde el que Maven descarga las dependencias.

Modelo (IA). La inteligencia artificial propiamente dicha, por debajo de una herramienta como un chat o un agente.

Módulo. Subproyecto con su propio `pom.xml`. Unidad de reutilización, versionado y despliegue.

ORM (*Object Relational Mapping*). Librería que traduce entre objetos y tablas.

POJO (*Plain Old Java Object*). Clase Java sencilla, sin lógica ni dependencias de ningún framework.

pom.xml. Fichero de configuración de un proyecto Maven.

Predicado. Función que devuelve un booleano.

Proxy. Objeto intermediario que implementa la misma interfaz que otro, lo envuelve y añade comportamiento.

Recurso (classpath). Fichero no compilado que viaja dentro del `.jar`: los diccionarios `.txt`, los ficheros de configuración.

Refactorizar. Cambiar la estructura del código conservando exactamente su comportamiento.

Repositorio (Spring Data). Interfaz que declara las operaciones sobre una entidad. La implementación la genera el framework.

Repositorio local (Maven). Carpeta `~/.m2` donde se guardan las dependencias descargadas y los módulos instalados.

REST. Conjunto de restricciones sobre HTTP para comunicar sistemas: las URL identifican recursos, el verbo dice qué hacer y el código de estado, cómo ha ido.

Serializar. Convertir un objeto en una secuencia de bytes o de texto para poder enviarlo por la red. Deserializar es la operación inversa.

Singleton. Patrón que garantiza una única instancia compartida. En Spring es el comportamiento por defecto.

SoC (*Separation of Concerns*). Al construir un componente, centrarse en él y olvidarse de los demás.

SOLID. Los cinco principios de diseño orientado a objetos: SRP, OCP, LSP, ISP, DIP.

SonarQube. Herramienta de análisis de calidad que puede bloquear el paso a producción si no se alcanzan ciertos umbrales.

Starter. Paquete de dependencias de Spring Boot que cubre un tipo de proyecto completo.

Stream. Flujo de datos sobre el que se encadenan operaciones de transformación y reducción.

Test double. Sustituto controlado de una dependencia real, usado en pruebas. Según su fidelidad se llama *stub*, *mock* o *fake*.

Tomcat embebido. Servidor de aplicaciones que Spring Boot incluye dentro del propio `.jar`, en lugar de instalarse aparte.

Transacción. Conjunto de operaciones sobre la base de datos que se confirman o se descartan como una sola unidad.

Versionado semántico. Convención `MAYOR.MENOR.PARCHES` que indica, sólo mirando el número, si actualizar es seguro.

Apéndice C · Comandos y referencia rápida

C.1. Maven

Comando	Qué hace
<code>mvn clean</code>	Borra la carpeta <code>target/</code>
<code>mvn compile</code>	Compila el código fuente
<code>mvn test</code>	Compila y ejecuta las pruebas
<code>mvn package</code>	Genera el <code>.jar</code>
<code>mvn install</code>	Instala el artefacto en el repositorio local
<code>mvn clean install</code>	Combinación habitual: partir de cero
<code>mvn install -DskipTests</code>	Instala sin ejecutar las pruebas
<code>mvn -pl MODULO ...</code>	Ejecuta el objetivo sólo sobre un módulo
<code>mvn dependency:tree</code>	Muestra el árbol completo de dependencias
<code>mvn -version</code>	Comprueba la instalación

C.2. Arrancar el proyecto de diccionarios

```
mvn clean install                # compilar todo

mvn -pl servicio-web spring-boot:run    # el servidor (puerto 8080)

mvn -pl aplicacion-completa exec:java -Dexec.args="casa es"  # cliente consola
mvn -pl aplicacion-completa exec:exec@act02                  # cliente escritorio
cd ui-web && npm install && npm start                          # cliente web (4200)
```

C.3. Probar el API con curl

```
curl http://localhost:8080/diccionarios/test    # prueba de vida
curl http://localhost:8080/diccionarios         # lista de idiomas
curl -i http://localhost:8080/diccionarios/es   # -i muestra el código de estado
curl http://localhost:8080/diccionarios/es/casa # significados
curl -i http://localhost:8080/diccionarios/es/manana # 404 con sugerencias
```

C.4. Anotaciones de Spring

Anotación	Dónde	Qué hace
<code>@SpringBootApplication</code>	Clase principal	Arranque y búsqueda automática de componentes
<code>scanBasePackages</code>	Atributo de la anterior	Amplía dónde buscar componentes
<code>@Component</code>	Clase	La registra como componente (singleton)
<code>@Configuration</code> + <code>@Bean</code>	Clase y método	Construcción explícita de un componente
<code>@RestController</code>	Clase	Componente que define rutas HTTP
<code>@GetMapping("/ruta")</code>	Método	Asocia el método a una ruta y al verbo GET
<code>@PathVariable</code>	Parámetro	Extrae una parte variable de la URL
<code>@RestControllerAdvice</code>	Clase	Tratamiento centralizado de excepciones
<code>@ExceptionHandler</code>	Método	Qué excepción atiende
<code>@ResponseStatus</code>	Método	Código de estado que se devuelve
<code>@Value("\${prop:defecto}")</code>	Campo	Injecta una propiedad de configuración
<code>@Transactional</code>	Método	Agrupar las operaciones en una sola transacción

C.5. Anotaciones de JPA

Anotación	Qué indica
<code>@Entity</code>	La clase se persiste en la base de datos
<code>@Table(name=...)</code>	En qué tabla
<code>@Id</code>	Clave primaria
<code>@GeneratedValue</code>	El valor lo genera la base de datos
<code>@Column(name=, nullable=, unique=)</code>	Nombre y restricciones de la columna
<code>@ManyToOne</code>	Muchos de estos pertenecen a uno de aquellos
<code>@OneToMany(mappedBy=, cascade=)</code>	Uno de estos tiene muchos de aquellos
<code>@UniqueConstraint(columnNames=...)</code>	Unicidad sobre varias columnas
<code>@EnableJpaRepositories</code>	Dónde están los repositorios
<code>@EntityScan</code>	Dónde están las entidades

C.6. Consultas derivadas de Spring Data

```
boolean existsByCodigo(String codigo);
Optional<Palabra> findByPalabraAndIdioma_Codigo(String palabra, String codigo);
List<Palabra> findByIdioma_CodigoOrderByPalabraAsc(String codigo);
List<Palabra> findByPalabraStartingWith(String prefijo);
long countByIdioma_Codigo(String codigo);
Optional<Palabra> findByPalabraIgnoreCase(String palabra);
```

Prefijos: `findBy`, `existsBy`, `countBy`, `deleteBy`. Conectores: `And`, `Or`. Sufijos: `IgnoreCase`, `StartingWith`, `EndingWith`, `Containing`, `Between`, `OrderBy...Asc/Desc`. Navegación a entidad relacionada: guion bajo (`Idioma_Codigo`).

C.7. Operaciones de Stream

Operación	Tipo	Qué hace
<code>filter(p)</code>	transformación	Deja pasar los que cumplen el predicado
<code>map(f)</code>	transformación	Transforma cada elemento
<code>flatMap(f)</code>	transformación	Transforma y aplana colecciones anidadas
<code>sorted(c)</code>	transformación	Ordena
<code>distinct()</code>	transformación	Elimina duplicados
<code>limit(n)</code> / <code>skip(n)</code>	transformación	Toma o descarta los primeros
<code>collect(...)</code>	reducción	Convierte en colección
<code>count()</code>	reducción	Cuenta
<code>anyMatch</code> / <code>allMatch</code> / <code>noneMatch</code>	reducción	Comprobaciones booleanas
<code>findFirst()</code>	reducción	Devuelve el primero, en un <code>Optional</code>
<code>reduce(...)</code>	reducción	Combina todos en un solo valor

C.8. Aserciones de JUnit 5

```
Assertions.assertEquals(esperado, obtenido);
Assertions.assertTrue(condicion);
Assertions.assertFalse(condicion);
Assertions.assertNull(objeto);
Assertions.assertNotNull(objeto);
Assertions.assertThrows(IllegalArgumentException.class, () -> metodo());
```

C.9. Códigos de estado HTTP más frecuentes

Código	Significado
200 OK	Todo bien
201 Created	Se ha creado un recurso
204 No Content	Todo bien, sin cuerpo que devolver
400 Bad Request	El cliente ha mandado algo que no se entiende
401 Unauthorized	Falta autenticación
403 Forbidden	Autenticado, pero sin permiso
404 Not Found	No existe
500 Internal Server Error	Fallo del servidor
503 Service Unavailable	El servidor no puede atender ahora

C.10. Git

```
git status          # qué ha cambiado
git add .           # preparar los cambios
git commit -m "mensaje" # confirmarlos
git log --oneline    # historial resumido
git diff            # ver los cambios sin confirmar
git diff --stat      # resumen de líneas cambiadas por fichero
```

Apéndice D · Soluciones de los ejercicios

Algunos ejercicios tienen una respuesta única y otros son de diseño o de criterio. En los segundos no se da "la" solución, sino los puntos que una buena respuesta debería tocar.

Capítulo 1 · Java

1.1. Se crean **dos** objetos. `a` se pega a `"uno"`; `b` se pega **al mismo dato**, no a una copia; después `a` se despega y se pega a `"dos"`. Al final `a` apunta a `"dos"` y `b` sigue apuntando a `"uno"`, así que **ninguno de los dos es basura**. Matiz: los literales de texto viven además en un almacén interno de la JVM (*string pool*), por lo que no se liberan aunque nadie los referencee.

1.2. Ahorro: $400 \text{ €/mes} = 4.800 \text{ €/año}$. Coste: $250 \text{ h} \times 55 \text{ €} = 13.750 \text{ €}$. Amortización: $13.750 / 4.800 \approx 2,9$ años. Otros factores que deberían pesar más que ese número: si el equipo sabe C++, el riesgo de introducir errores de memoria, el coste de mantener dos tecnologías, la pérdida de librerías del ecosistema Java y si el servicio va a seguir vivo dentro de tres años.

1.3. De `4.2.7` a `4.3.0` sube la **MENOR**: hay funcionalidad nueva y nada de lo que usabas ha desaparecido; puedes actualizar sin revisar. A `5.0.0` sube la **MAYOR**: hay cambios incompatibles y hay que revisar. A `4.2.9` sube el **PARCHE**: sólo correcciones; es la actualización más segura y la que más conviene aplicar siempre.

1.4. Porque Kotlin **compila a bytecode**, igual que Java. Ambos acaban siendo el mismo lenguaje para la máquina virtual, así que una clase Kotlin puede llamar a una clase Java directamente, sin conversión ni adaptador.

Capítulo 2 · Maven

2.1. No tiene razón. `test` es una **fase anterior** a `package` en el ciclo de vida, y pedir una fase ejecuta todas las anteriores. Formas de obtener el `.jar` en ese momento: `mvn package -DskipTests` (compila las pruebas pero no las ejecuta) o `mvn package -Dmaven.test.skip=true` (ni siquiera las compila). Las dos son parches: lo correcto es arreglar la prueba.

2.2. Son las **dependencias transitivas**: la librería declarada necesita otras, que necesitan otras. Se ven con `mvn dependency:tree`.

2.3. No basta. Hay que instalar `diccionarios-api` y volver a construir todo lo que depende de él, porque los demás módulos usan el `.jar` del repositorio local, no el código fuente. Si se olvida, se compila contra la versión antigua y aparecen errores desconcertantes en ejecución. Lo seguro es `mvn install` desde el padre.

2.4. Porque así acaban **dentro del** `.jar` y se leen como recursos del *classpath*. Si se movieran a una carpeta del disco, la aplicación funcionaría desde el IDE y fallaría al ejecutarse empaquetada: un objeto `File` no sabe leer dentro de un `.jar`.

2.5. Una división razonable: `facturacion-api` (contratos), `facturacion-en-postgres` y `facturacion-en-servicio-externo` (implementaciones, dependen del api), `facturacion-impuestos` (lógica de negocio, depende del api), `facturacion-rest` (API REST, depende del api y de impuestos) y `facturacion-cli`. Regla que debe cumplirse: **las implementaciones dependen del contrato, y nadie depende de una implementación** salvo el módulo que decide cuál se usa.

Capítulo 3 · Principios de diseño

3.1. `DiccionarioEnFichero` (ficheros `.txt` del classpath), `DiccionarioEnBBDD` (base de datos vía JPA) y `DiccionarioEnServicioWeb` (peticiones HTTP). En `Aplicacion` **no hay que cambiar nada**: sólo la factoría. El principio es el **DIP**: la aplicación depende de la abstracción, no de las implementaciones.

3.2. Dejarían de compilar los tres módulos que implementan `Diccionario`. Dos formas: (a) declararlo como método `default` con una implementación por defecto o que lance `UnsupportedOperationException`; (b) crear una interfaz nueva `DiccionarioContable` `extends Diccionario` que sólo implementen los que puedan. Preferible la (b) si el método no tiene sentido para todas las implementaciones —es ISP—; la (a) si es un añadido temporal para mantener la compatibilidad.

3.3. `InterfazDeUsuario` mezcla tres responsabilidades: **obtener datos** del usuario (`recuperar...`), **mostrar resultados** (`mostrarSignificados`, `mostrarQueLaPalabraNoExiste`) y **mostrar mensajes de sistema** (bienvenida, ayuda, despedida). Se podría dividir en `EntradaDeUsuario`, `PresentacionDeResultados` y `MensajesDeAplicacion`. La implementación de escritorio sólo necesitaría las dos últimas, y no se vería obligada a fingir la primera con un semáforo.

3.4. `WeakHashMap<String, ResultadoBusqueda>`, porque permite al recolector de basura liberar entradas cuando falta memoria. Con un `HashMap`, millones de búsquedas distintas harían crecer la tabla sin límite hasta provocar un `OutOfMemoryError`: el sistema se caería **por culpa de una optimización**.

3.5. Porque los diccionarios son **recursos del classpath**, no ficheros del disco. Con una ruta absoluta dejaría de funcionar el empaquetado en `.jar`, la ejecución en otra máquina y en otro sistema operativo, y cualquier despliegue en contenedor.

Capítulo 4 · Patrones

4.1. Una clase que implemente `Diccionario`, reciba otro `Diccionario` por constructor y delegue en él, registrando cuando `dameSignificados()` devuelva vacío:

```

public class DiccionarioQueRegistraFallos implements Diccionario {
    private final Diccionario original;
    public DiccionarioQueRegistraFallos(Diccionario original) { this.original = original; }

    public Optional<List<String>> dameSignificados(String palabra) {
        Optional<List<String>> r = original.dameSignificados(palabra);
        if (r.isEmpty()) logger.info("No encontrada: " + palabra);
        return r;
    }
    // el resto de métodos delegan sin más
}

```

Se enchufa donde se construye el diccionario: en

`SuministradorDeDiccionariosEnBBDD.dameDiccionarioDe()`, envolviendo el objeto antes de devolverlo.

4.2. Ejemplos: **seguridad** (comprobar permisos antes de cada operación), **auditoría** (quién hizo qué y cuándo, por obligación legal) y **trazabilidad** (seguir una petición a través de varios servicios). Implementadas dentro de cada método: se repiten en cientos de sitios, es fácil olvidarlas en uno —y ése será justo el agujero de seguridad—, y no se pueden desactivar sin tocar el código.

4.3. A favor de unificar: es el mismo dato y hoy se mantienen dos ficheros. En contra: acopla el ciclo de vida del cliente al del servidor; añadir un campo en el servidor obligaría a recompilar y redespigar el cliente. **Si los desarrollan empresas distintas la respuesta es clarísima: separadas.** Un módulo compartido significa un acuerdo de versiones entre dos organizaciones.

4.4. (1) `@RestControllerAdvice` para el tratamiento de errores; (2) los repositorios de Spring Data, cuya implementación se genera en tiempo de arranque; (3) `@Transactional`, que envuelve el método en la apertura y confirmación de la transacción. Los tres son proxies creados por el framework.

4.5.

```

List<Palabra> findByIdioma_CodigoAndPalabraStartingWith(String codigo, String prefijo);
long countByIdioma_Codigo(String codigo);
Optional<Palabra> findByPalabraIgnoreCaseAndIdioma_CodigoIgnoreCase(String p, String c);

```

Capítulo 5 · Pruebas

5.1. Salen cinco pruebas, una por caso. No una sola con cinco aserciones porque **JUnit para en la primera aserción que falla**: con una sola prueba, si falla el primer caso no llegas a saber si los otros cuatro funcionan. Además el informe indica exactamente qué comportamiento se ha roto.

5.2. Creando una clase que implemente `PalabraRepository` y devuelva una lista fija de palabras —o usando Mockito—, y pasándosela al constructor de `DiccionarioEnBBDD`. No hace falta base de datos porque la clase **no la construye**: la recibe.

5.3.

```
@Test
void noHayaSignificadosSiLaPalabraNoExiste() {
    DiccionarioEnFichero d = new DiccionarioEnFichero("es", new HashMap<>());
    Optional<List<String>> r = d.dameSignificados("inexistente");
    Assertions.assertNotNull(r);
    Assertions.assertTrue(r.isEmpty());
}
```

Protege el **LSP**: todas las implementaciones de `Diccionario` deben comportarse igual ante el mismo caso.

5.4. La cobertura mide qué líneas se **ejecutan**, no si se **comprueban**. Una prueba puede recorrer una línea sin verificar su resultado, o verificar sólo el caso favorable. Medidas complementarias: revisión de código por pares, pruebas de mutación (que alteran el código y comprueban que alguna prueba falla) y exigir una prueba que reproduzca cada error corregido.

5.5. No sería unitaria porque ejercita varias capas a la vez: el enrutamiento HTTP, el controlador y el negocio. Es una **prueba de integración**, y en Spring se escriben con `@SpringBootTest` (levanta el contexto completo) o `@WebMvcTest` (levanta sólo la capa web), acompañadas de `MockMvc` para lanzar peticiones simuladas.

Capítulo 6 · Arquitecturas

6.1. En la V1: desarrollo reempaqueta, el usuario reinstala, operaciones publica y avisa, soporte no sabe qué versión tiene cada uno, y conviven varias apariencias distintas. En la V5 con interfaz web: desarrollo despliega, y **nadie más hace nada**; el siguiente usuario que recargue el navegador ya lo ve. El cambio de coste es de varios implicados a uno solo.

6.2. División razonable: tienda, agenda de citas, peluquería y recursos humanos. Datos compartidos: la ficha de cliente y la de mascota. **El punto más delicado es precisamente ése**: si cada sistema guarda su propia copia del cliente, se desincronizan; si todos consultan a un único servicio de clientes, ese servicio se convierte en un punto único de fallo. Es la decisión de diseño de fondo de cualquier descomposición.

6.3. Ejemplos válidos: (a) soportar millones de usuarios concurrentes —haría falta caché, réplicas y repensar `palabrasSimilares`—; (b) permitir que los usuarios propongan definiciones —haría falta autenticación, permisos y moderación, que hoy no existen—; (c) funcionar sin conexión —hoy el cliente depende por completo del servidor—.

6.4. Debe contener: que el coste inicial mayor es cierto y está asumido; que el gasto real de un producto está en los años de mantenimiento; y un número del propio proyecto, como que pasar de ficheros a base de datos costó un módulo nuevo y **una línea** modificada, cuando en la arquitectura anterior habría obligado a redistribuir la aplicación a todos los usuarios.

6.5. Recibiría instrucciones de presentación visual —negritas, márgenes, tamaños— que no puede convertir en voz, y tendría que extraer los datos analizando el marcado, con un resultado frágil. Del servicio

REST recibe `{"palabra":..., "significados":[...]}`, que puede leer directamente.

Capítulo 7 · HTTP y REST

7.1. Hoy se distinguen **por el cuerpo**: si el idioma no existe llegan todos los campos a `null`; si existe pero la palabra no, llega `idioma` relleno. Alternativas mejores: usar códigos distintos — `404` para palabra no encontrada y `400` o `404` con un campo `codigoDeError` explícito—, o separar la comprobación del idioma en una llamada previa a `/diccionarios/{idioma}`.

7.2.

Operación	Verbo y ruta	Éxito	Errores
Alta	<code>POST</code> <code>/diccionarios/{idioma}/{palabra}</code> <code>}</code>	<code>201</code> + recurso creado	<code>404</code> idioma no existe · <code>409</code> la palabra ya existe
Modificación	<code>PUT</code> <code>/diccionarios/{idioma}/{palabra}</code> <code>}</code>	<code>200</code> + recurso	<code>404</code> no existe
Borrado	<code>DELETE</code> <code>/diccionarios/{idioma}/{palabra}</code> <code>}</code>	<code>204</code> sin cuerpo	<code>404</code> no existe

7.3. Tres problemas: usa `GET` para una operación **destructiva**, cuando `GET` debe ser seguro y sin efectos —un rastreador o un navegador podrían invocarlo—; pone la **acción en la URL** en lugar del recurso; y pasa por parámetros de consulta lo que son partes de la identidad del recurso. Lo correcto es `DELETE /diccionarios/es/melon`.

7.4. El cliente Java se entera **en ejecución**: compila igual y `getSignificados()` devuelve `null`. El cliente Angular, también en ejecución: el campo llega `undefined`. Estrategia correcta: **añadir el campo nuevo sin quitar el antiguo**, publicar los dos durante un tiempo, avisar a los consumidores y retirar el viejo después. Es versionado de API.

7.5. Tres llamadas: 120 ms. Una: 40 ms. Es **3 veces más caro**, 80 ms extra por búsqueda. En una sesión de 50 búsquedas son **4 segundos** de espera adicional acumulada, sin contar la carga extra en el servidor.

Capítulo 8 · Tecnologías heredadas

8.1. Un módulo adaptador que hable RMI con el sistema heredado y exponga un API REST hacia fuera. El resto del sistema sólo conoce REST. Es el patrón **Adapter**, y en este contexto se le llama a menudo *fachada de modernización* o *strangler facade*, porque permite ir sustituyendo el sistema antiguo por partes.

8.2. La petición SOAP ronda los **330 bytes**; la información equivalente en REST viaja en la URL, unos **30 bytes**. Aproximadamente **11 veces más**, unos 300 bytes de sobrecoste por petición. Con un millón de peticiones diarias: unos **300 MB al día** sólo de sobre XML, sin contar la respuesta.

8.3. Con *Service Locator*, el componente **pide** su dependencia por un nombre; si el nombre está mal escrito el fallo aparece en ejecución, cuando se ejecuta esa línea. Con **inyección de dependencias**, el componente sólo declara qué necesita y el contenedor lo resuelve **al arrancar**: si falta algo, la aplicación no llega a levantarse. Fallar pronto y en el sitio correcto es una ventaja enorme en producción.

8.4. El *Session Bean* contenía lógica de negocio sin estado: hoy lo cumple

`SuministradorDeDiccionariosEnBBDD`, anotado con `@Component`. El *Entity Bean* representaba una fila de una tabla: hoy son las entidades `Idioma`, `Palabra` y `Significado`, anotadas con `@Entity`.

8.5. Porque **nadie lo usaba ya**: eliminar algo que ningún proyecto moderno utiliza no rompe nada en la práctica. La API de colecciones, en cambio, la usa absolutamente todo el ecosistema. En términos del capítulo 1, ambos serían cambios de versión **MAYOR**, pero el coste real de un *breaking change* no depende sólo de la norma: depende de cuánta gente dependa de lo que se rompe.

Capítulo 9 · Spring

9.1. La aplicación **falla al arrancar** con un error de tipo `NoUniqueBeanDefinitionException`, en el momento de construir el contexto. Soluciones: `@Primary` sobre la implementación que debe ganar por defecto —útil cuando hay una opción claramente principal— o `@Qualifier("nombre")` en el punto de inyección —necesario cuando distintos consumidores quieren implementaciones distintas—.

9.2.

```
@Configuration
public class SuministradorConfiguration {
    @Bean
    public SuministradorDeDiccionarios suministrador(
        @Value("${diccionarios.servidor:http://localhost:8080}") String rutaServidor) {
        return new SuministradorDeDiccionariosEnServicioWeb(rutaServidor);
    }
}
```

9.3. Porque a `SuministradorDeDiccionariosEnBBDD` lo construye **Spring** —es un punto de entrada al negocio y necesita que le inyecten los repositorios—, mientras que `DiccionarioEnBBDD` lo construye **el propio suministrador** con `new`, una vez por idioma y en tiempo de ejecución. Anotarlo no serviría: Spring no sabría qué idioma pasarle.

9.4. `@SpringBootApplication` busca componentes en el paquete de la clase y sus subpaquetes. Al mover la clase a `com.curso.arranque`, el paquete `com.curso.diccionarios` deja de estar por debajo. Soluciones: mantener `scanBasePackages = {"com.curso.diccionarios"}` en la anotación, o devolver la clase a un paquete que sea raíz común de todos los componentes.

9.5. Cinco ejemplos: arrancar el servidor de aplicaciones (antes: instalar y configurar WebSphere o Tomcat); registrar las rutas HTTP (antes: `web.xml`); convertir objetos a JSON (antes: código a mano); localizar dependencias (antes: JNDI); crear las tablas de la base de datos (antes: scripts SQL a mano).

Capítulo 10 · Persistencia

10.1. Una relación de una palabra consigo misma. Con una entidad intermedia:

```
@Entity
@Table(name = "sinonimos",
        uniqueConstraints = @UniqueConstraint(columnNames = {"palabra_id", "sinonimo_id"}))
public class Sinonimo {
    @Id @GeneratedValue private int id;
    @ManyToOne private Palabra palabra;
    @ManyToOne private Palabra sinonimo;
}
```

La restricción de que una palabra no sea sinónimo de sí misma **no se puede expresar con una anotación JPA**: hay que validarla en el código —o con una restricción `CHECK` en la base de datos—. Es un buen ejemplo de que el ORM no lo cubre todo.

10.2. (a) `List<Palabra> findByIdioma_CodigoOrderByPalabraAsc(String codigo);` — derivada. (b) `long countByIdioma_Codigo(String codigo);` — derivada, una por idioma; para todos de golpe hace falta `@Query` con `GROUP BY`. (c) `List<Palabra> findByPalabraContaining(String texto);` — derivada. (d) Necesita `@Query`: `SELECT p FROM Palabra p WHERE p.significados IS EMPTY`.

10.3. Además de tardar el doble, cambia el comportamiento ante un fallo. **Con** `@Transactional`, si la carga falla a mitad se deshace todo y la base de datos queda **vacía y coherente**; al reiniciar, el `count() > 0` es falso y se vuelve a intentar la carga completa. **Sin** la anotación, quedan cargadas las palabras insertadas hasta el fallo; al reiniciar, `count() > 0` es cierto y el cargador **no hace nada**, dejando el sistema con un diccionario incompleto de forma permanente. El segundo escenario es mucho peor porque es silencioso.

10.4. Hay que cambiar: la dependencia del driver en el `pom.xml` y las propiedades de conexión en `application.properties`. **Cero líneas de código Java**. Ésa es exactamente la promesa del patrón DAO y de un ORM: el almacenamiento cambia y la lógica de negocio no se entera.

10.5. Porque la misma palabra puede existir legítimamente en varios idiomas. Válidas: `(casa, es)` y `(casa, en)` —"casa" existe en español y es un préstamo en inglés—. Rechazadas: dos filas `(casa, es)`, o dos `(house, en)`.

Capítulo 11 · Programación funcional

11.1. La versión imperativa ronda las 20-25 líneas frente a 9: bucle sobre todas las palabras, dos `if` de filtrado, construcción de una lista auxiliar, ordenación con `Collections.sort` y un segundo bucle para cortar en diez y extraer el texto. El criterio de legibilidad es personal, pero conviene fijarse en que la versión con *streams* **se lee en el mismo orden en que ocurren las cosas**, mientras que la imperativa mezcla el qué con el cómo.

11.2.

```

long a = palabras.stream().filter(p -> p.startsWith("a")).count();

Optional<String> b = palabras.stream().max(Comparator.comparingInt(String::length));

List<String> c = palabras.stream()
    .flatMap(p -> p.getSignificados().stream())
    .distinct()
    .collect(Collectors.toList());

```

11.3. `casa` → `caso` : **1** (sustituir la `a` final por `o`). `casa` → `causa` : **1** (insertar una `u`).

`melón` → `manzana` : **5**. Con el filtro de longitud y `DISTANCIA_MAXIMA_ADMISIBLE = 2` : los dos primeros pares se calculan (difieren en 0 y 1 caracteres de longitud); el tercero **también se calcularía**, porque `melón` tiene 5 letras y `manzana` 7, y la diferencia es exactamente 2. Se descartaría después, en el segundo filtro.

11.4. Medido sobre el diccionario real de 20.000 palabras: para una palabra de seis letras y un margen de 2, sobreviven **13.241 palabras (66,2 %)**, es decir se ahorra un **33,8 %** del trabajo. Con margen 3 sobreviven el 80,4 %. La longitud media del diccionario es de 7,6 caracteres, y el grueso está entre 5 y 10 letras, por eso el filtro descarta menos de lo que la intuición sugiere: es más eficaz con palabras muy cortas o muy largas.

11.5.

```

public static final int DIFERENCIA_MAXIMA_DE_LONGITUD = 3;
public static final int DISTANCIA_MAXIMA_ADMISIBLE = 2;

```

La de longitud debe ser **mayor o igual** que la de distancia: si se descartara por longitud algo que podría tener una distancia aceptable, se perderían sugerencias válidas. Con longitud 2 y distancia 2, como está hoy, el filtro es correcto pero más restrictivo de lo necesario.

Capítulo 12 · Inteligencia artificial

Los cinco ejercicios de este capítulo son de criterio y no tienen respuesta única. Puntos que debería tocar una buena respuesta:

12.1. Errores típicos que buscar en pruebas generadas: que no cubran `null` ni la cadena vacía; que verifiquen lo que el código hace en lugar de lo que **debería** hacer; y aserciones vacías o triviales que pasan siempre.

12.2. La respuesta con contexto debería usar los nombres reales del proyecto, respetar sus convenciones y encajar con las clases existentes. La respuesta sin contexto tenderá a inventar nombres y a proponer una solución genérica y aislada.

12.3. Preguntas obligadas: ¿qué contrato de tratamiento de datos hay con el proveedor?, ¿se usan los datos para entrenar?, ¿hay datos personales o credenciales en el repositorio?, ¿lo permite el contrato con

el cliente? Negativa clara si hay datos personales, secretos o código sujeto a acuerdos de confidencialidad.

12.4. Debe explicar que el contrato de interfaz de usuario estaba diseñado para un flujo dirigido por la aplicación, mientras que en una interfaz gráfica manda el usuario; y que el hecho de que dos implementaciones independientes llegaran a la misma conclusión indica que la abstracción de negocio estaba bien planteada y la de interfaz no del todo.

12.5. El ejercicio se evalúa por la calidad del argumento, no por la decisión. Una crítica bien fundada podría señalar, por ejemplo, la caché de idiomas del cliente web, el uso de `text-transform` en CSS para ocultar que los datos llegan en mayúsculas, o la ausencia de pruebas en el código generado.

Parte II · El proyecto

II.1. En la V1: alguien añade el fichero, un desarrollador reempaqueta, se publica la versión, se avisa a los usuarios y **cada usuario reinstala**; hasta que el último no lo haga, conviven respuestas distintas. En la V5: se añade el módulo del idioma, se despliega el servidor y **todos los clientes lo ven al instante**, sin tocar nada. Uno o dos intervinientes frente a todos los usuarios de la organización.

II.2. Opción A, tocando los ficheros: renombrar `es.grande.txt` a otro nombre que no colisione y decidir cuál de los dos es "el español". Opción B, tocando el código: que el código de idioma no se deduzca del nombre del fichero sino de un fichero de metadatos o de una cabecera dentro del propio `.txt`. La A es inmediata y no resuelve la causa; la B elimina el problema de raíz pero obliga a cambiar `CargadorDeDatos` y el formato de los diccionarios. En un sistema con recorrido, la B.

II.3. Bastaría con un único método en `Diccionario` que devolviera un resultado completo —significados o sugerencias— en una sola llamada, y que `Aplicacion` lo usara en lugar de encadenar `existe`, `dameSignificados` y `palabrasSimilares`. Sí se puede hacer sin romper a nadie: se **añade** el método nuevo como `default` y se dejan los antiguos en su sitio, marcados como obsoletos. Es el mismo mecanismo del capítulo 3.

II.4. Módulo nuevo `diccionarios-en-servicio-externo` que implemente `SuministradorDeDiccionarios` y `Diccionario`. Se reutiliza absolutamente todo lo demás y se cambia **una línea** en la factoría o una dependencia en el `pom.xml`. La caché debe ir **dentro del módulo nuevo**, no en la aplicación: es un detalle del proveedor caro, no del negocio. Conviene además decidir qué hacer cuando el servicio externo no responde y hay datos en caché caducados.

II.5. Un orden defendible: (1) pruebas del servidor y del acceso a datos, porque sin ellas cualquier otro cambio es arriesgado; (2) arreglar `palabrasSimilares`, que es el fallo con más probabilidad de tumbar el sistema en producción; (3) actualizar H2 o sustituirla, por las vulnerabilidades; (4) unificar el contrato documentado con el real, que es barato y evita errores en los clientes; (5) el idioma fantasma `ES.GRANDE`, que es visible pero inofensivo. Lo esencial del ejercicio es el razonamiento riesgo/esfuerzo, no el orden concreto.

Apéndice E · Correcciones respecto a las notas de clase

Este manual se ha escrito a partir de las notas tomadas durante las sesiones. Esas notas son un material de trabajo excelente —de ellas sale prácticamente todo el contenido— pero se redactaron sobre la marcha y contienen algunos datos imprecisos.

Se recogen aquí por una razón práctica: **los alumnos conservan sus propios apuntes**, y en estos puntos concretos el manual dice algo distinto de lo que se dijo en clase. Conviene saber cuáles son y por qué.

E.1. Edsger Dijkstra

En las notas	Dato correcto
"Edgar Dijkstra"	Edsger W. Dijkstra
Premio Turing en 1971	Premio Turing en 1972
El principio SoC sale de <i>The Humble Programmer</i>	<i>The Humble Programmer</i> es su conferencia Turing de 1972; la formulación de la separación de intereses aparece en un texto posterior, de 1974

The Humble Programmer sigue siendo una lectura muy recomendable y el espíritu de la cita de clase es exacto: la humildad del programador frente a la complejidad.

E.2. El origen de Kotlin

En las notas: Google encargó a JetBrains la creación de un lenguaje nuevo, que fue Kotlin.

Lo que ocurrió: JetBrains empezó a desarrollar Kotlin en **2010 por iniciativa propia**, para sus propias herramientas, y lo publicó como software libre en 2012. Google no lo encargó: lo **adoptó** como lenguaje oficialmente soportado para Android en 2017, y como lenguaje preferente en 2019.

El fondo del argumento de clase se sostiene igual —Google redujo su dependencia de Java tras el litigio con Oracle— pero la relación causa-efecto era otra.

E.3. El origen de Node.js

En las notas: Google extrajo de Chromium el intérprete de JavaScript para poder ejecutar aplicaciones fuera del navegador, y eso es Node.js.

Lo que ocurrió: Node.js lo creó **Ryan Dahl en 2009**, y lo hizo **sobre** el motor V8, que sí es de Google y sí procede de Chrome. Pero Node.js no es un proyecto de Google ni salió de Chromium: es un proyecto independiente que reutilizó una pieza de Google.

E.4. Fechas de las versiones de Java

Varias fechas de la cronología de clase están desplazadas un año. La secuencia correcta de las versiones citadas:

Versión	En las notas	Fecha correcta
Java 1.0	"finales de los 90"	enero de 1996
Java 5	2005	septiembre de 2004
Java 10	2017	marzo de 2018
Java 11	2018	septiembre de 2018 ✓
Java 12	2018	marzo de 2019
Java 21	2024	septiembre de 2023
Java 22	2025	marzo de 2024

Lo que sí es exacto: el cambio de cadencia a seis meses a partir de la versión 9 (2017) y que la 27 está prevista para septiembre de 2026.

E.5. La compra de Sun por Oracle

En las notas: Oracle compra Sun Microsystems en 2009.

Precisión: la operación se **anunció en abril de 2009** y se **completó en enero de 2010**. La fecha de clase es la del anuncio, que es la que suele citarse.

E.6. El conflicto por la licencia de Java

En las notas: Oracle anunció que cobraría por la máquina virtual (25 \$/año para particulares, 50 \$/core para empresas) y, tras conversaciones con Google, convirtió la JVM en una especificación abierta.

Lo que ocurrió, con más precisión:

- Java ya tenía una especificación abierta y un proceso de estandarización (el JCP) desde mucho antes; eso no fue una concesión de aquel momento.
- Lo que Oracle introdujo, a partir de 2019, fue un **modelo de suscripción para el uso comercial de su propia distribución del JDK**, con precios por procesador en servidores y por usuario en puestos de trabajo. Las cifras concretas de las notas no se corresponden con las tarifas publicadas.
- El desenlace no vino de una negociación con Google, sino de la **consolidación de OpenJDK** y de la aparición de distribuciones gratuitas mantenidas por otras compañías (Eclipse Temurin, Amazon Corretto, Azul Zulu, Red Hat, Microsoft).
- El conflicto con Google fue un **litigio judicial de once años** (2010-2021) por el uso de las APIs de Java en Android, que el Tribunal Supremo de Estados Unidos resolvió a favor de Google.

La conclusión práctica de clase se mantiene intacta: **hoy nadie está obligado a pagar por ejecutar Java**, pero descargar el JDK de la web de Oracle e instalarlo en los servidores de una empresa puede tener implicaciones de licencia.

E.7. La creación de la Web

En las notas: Tim Berners-Lee crea la Web en 1990 y se hace pública en 1993.

Precisión: la propuesta es de **1989**, el primer sitio funcionando de **1991**, y en **1993** el CERN liberó la tecnología al dominio público. La fecha de 1993 de las notas es correcta y es la más relevante.

E.8. El contrato del API REST

En las notas y en el material de referencia: cuando el idioma no existe, el servicio devuelve `{}`.

Lo que devuelve realmente:

```
{"idioma":null,"palabra":null,"significados":null,"similares":null}
```

La causa es que Jackson serializa también los campos a `null`. Además, el campo `idioma` se devuelve **tal y como lo escribió el cliente** (`es`), no normalizado a mayúsculas (`ES`) como se almacena en la base de datos.

No es un error de las notas sino una divergencia entre el contrato diseñado y el comportamiento implementado. Está recogida en el capítulo 7 y en la lista de limitaciones del capítulo 18.

E.9. El tamaño del diccionario de español

Durante el curso, el fichero `es.grande.txt` contenía **646.612 entradas**, y sobre esa cifra se hicieron las mediciones de rendimiento de la carga inicial (130 segundos, que bajaron a 65 con `@Transactional`).

El diccionario que se distribuye hoy con el proyecto tiene **20.000 palabras**, seleccionadas por frecuencia de uso, con las definiciones tomadas del Wikcionario y con su procedencia y licencias documentadas en el fichero `NOTICE.md` que las acompaña.

Las mediciones citadas en el manual se identifican como históricas allí donde aparecen. La lección técnica no cambia: el coste no estaba en insertar, estaba en confirmar cada inserción por separado.

Ninguna de estas correcciones afecta a los razonamientos de fondo del curso. Son datos de contexto: nombres, fechas y cifras. Se recogen aquí porque un material destinado a los alumnos debe poder consultarse sin reservas, y porque saber **dónde** un documento corrige a otro es parte de poder fiarse de los dos.